Goals for this talk

- Overview of CART, bagged trees, and random forest.
- Learn how to “read” a decision tree.
- Why bagging is useful and how random forests build upon the ideas of bagging.
- Why gradient boosting improves performance further.

Readings

- The Hands on Machine Learning with R book provides useful practical discussions on all three topics
  - Including how to fit, tune, and make predictions with the methods using multiple packages
  - See Chapters 9, 10, and 11

- Chapter 8 in An Introduction to Statistical Learning (ISL) provides additional discussion.

Tree-based models

- Non-parametric methods which partition the input space into smaller non-overlapping regions (rectangles).
- Simple models are fit within each region.
- We will focus on the Classification And Regression Tree (CART) algorithm.
  - Can be used for regression or classification!

CART – Binary recursive partitioning for regression

- Split the input space into two regions.
- Model the output as the AVERAGE value within each region.
- **Variable and split-point is chosen to achieve the best fit**.
- Split one or both regions into two more regions.
- Continue until a stopping rule is applied.

CART – regression model predictions

- Consider $\mathrm{m} = 1, \ldots, M$ regions: $R_1, R_2, \ldots, R_M$

- Define the constant output value within region R_m as:

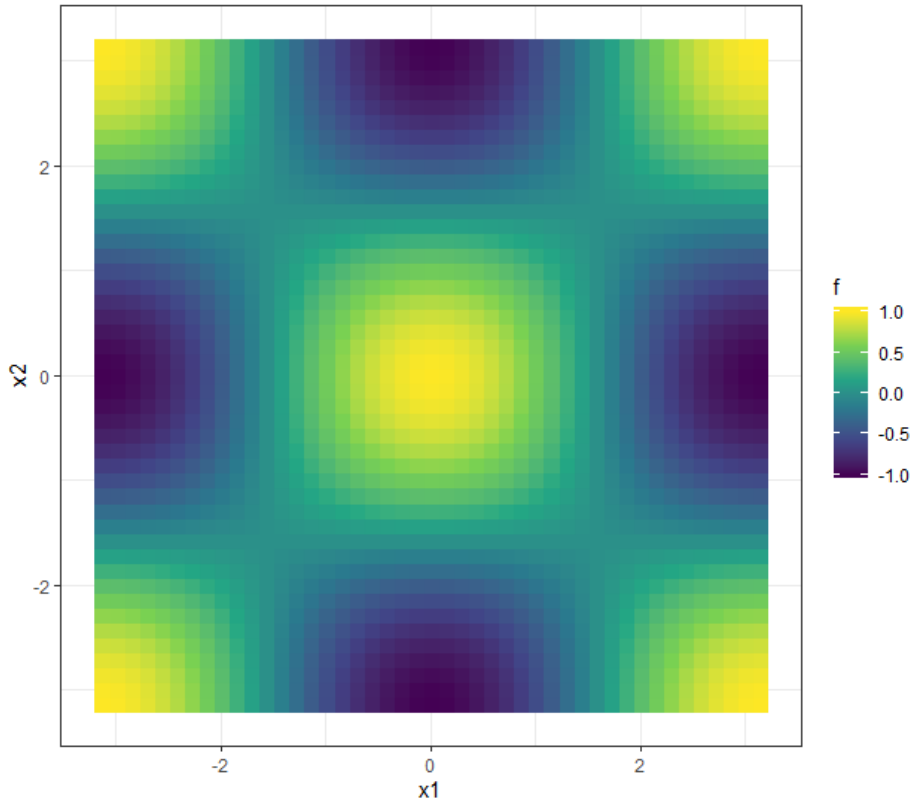
$$c_m = \mathrm{AVERAGE}(y_n \mid \mathbf{x}_n \in R_m)$$

- The CART prediction for a new input point $\mathbf{x}_*$ is:

$$f(\mathbf{x}_*) = \sum_{m=1}^{M} \left(c_m I(\mathbf{x}_* \in R_m) \right)$$

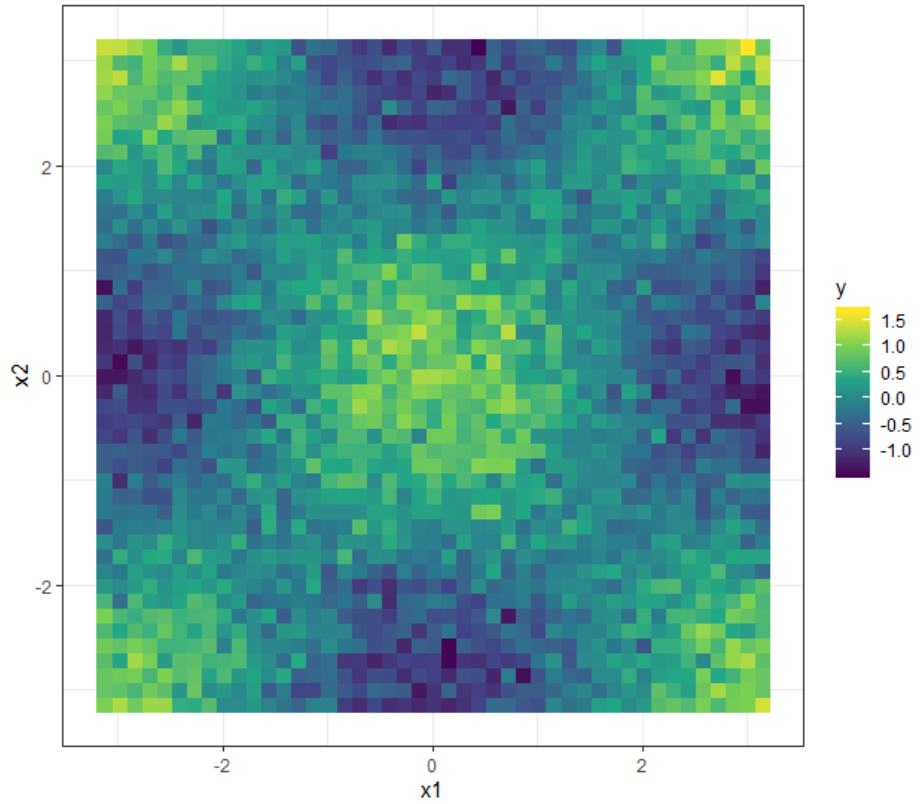
Indicator function: if $\mathbf{x}_*$ is within region R_m return 1, otherwise return 0

2D example – true response: $\cos(x_1)\cos(x_2)$



True noise-free function

$$f_n = \cos(x_{n,1})\cos(x_{n,2})$$

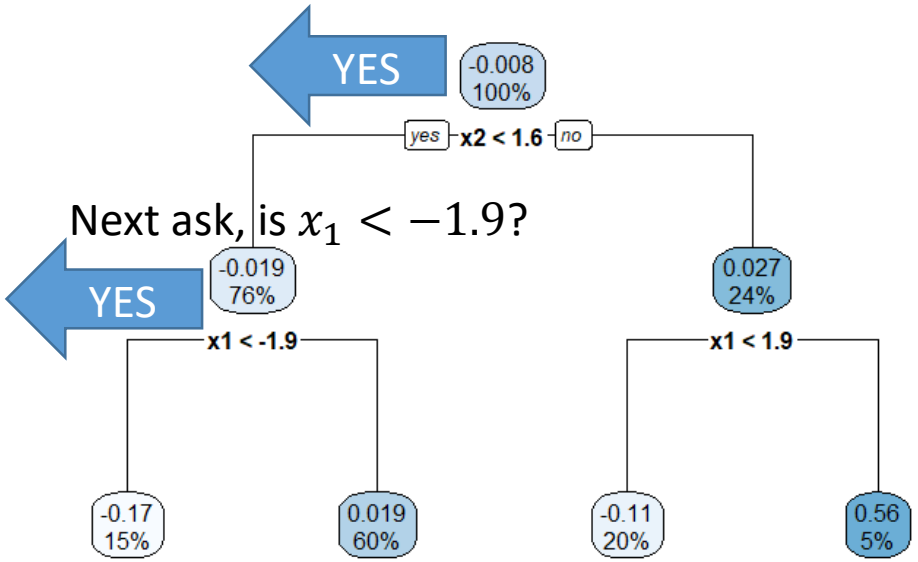


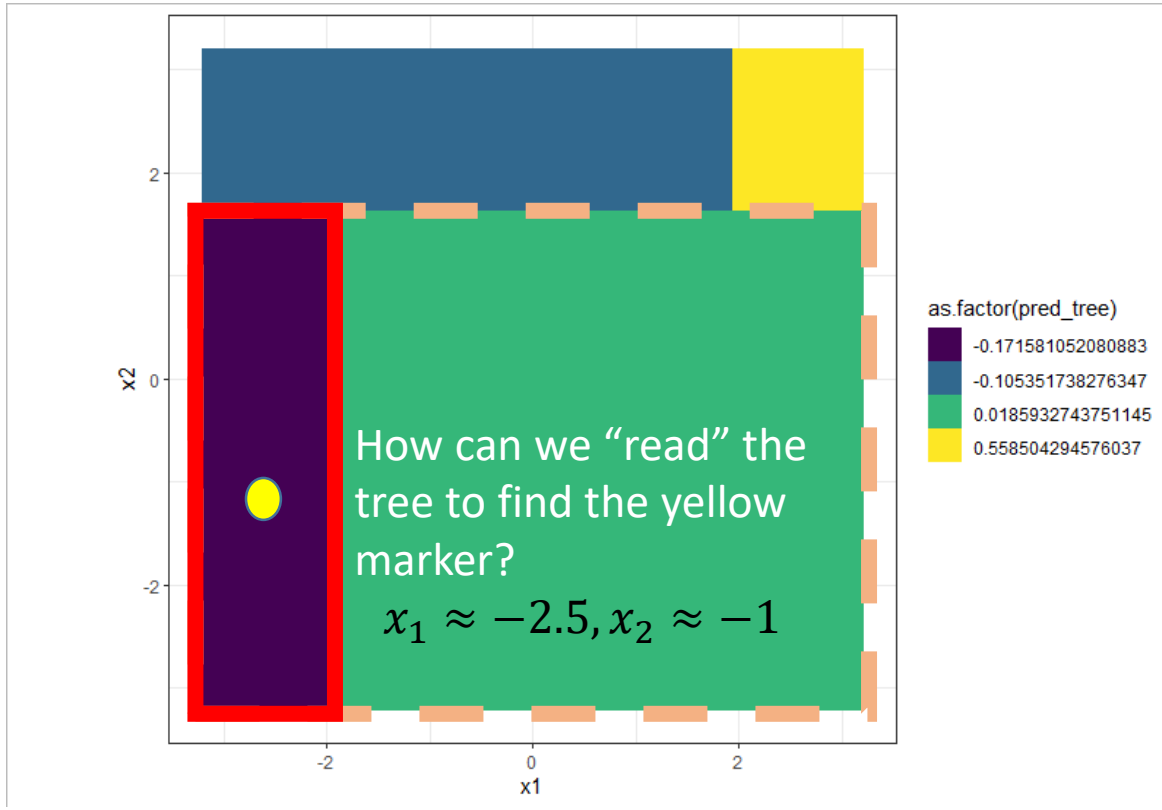
Noisy observations

$$y_n \sim \text{normal}(f_n, 0.25)$$

Fit a simple decision tree – interpretable results!

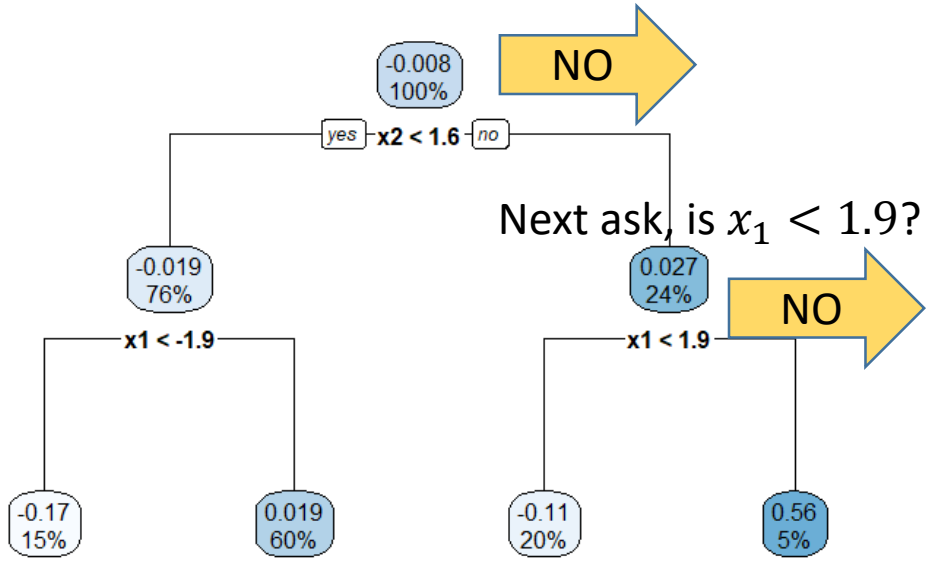
Start with the first split, is $x_2 < 1.6$?

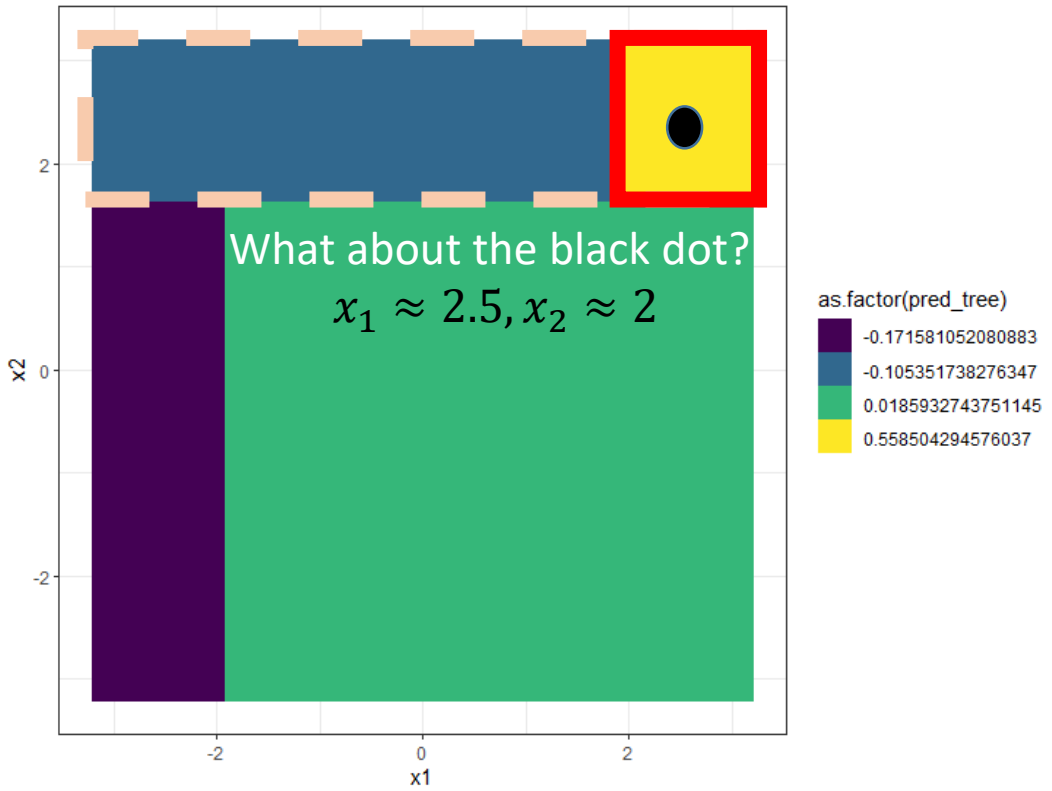




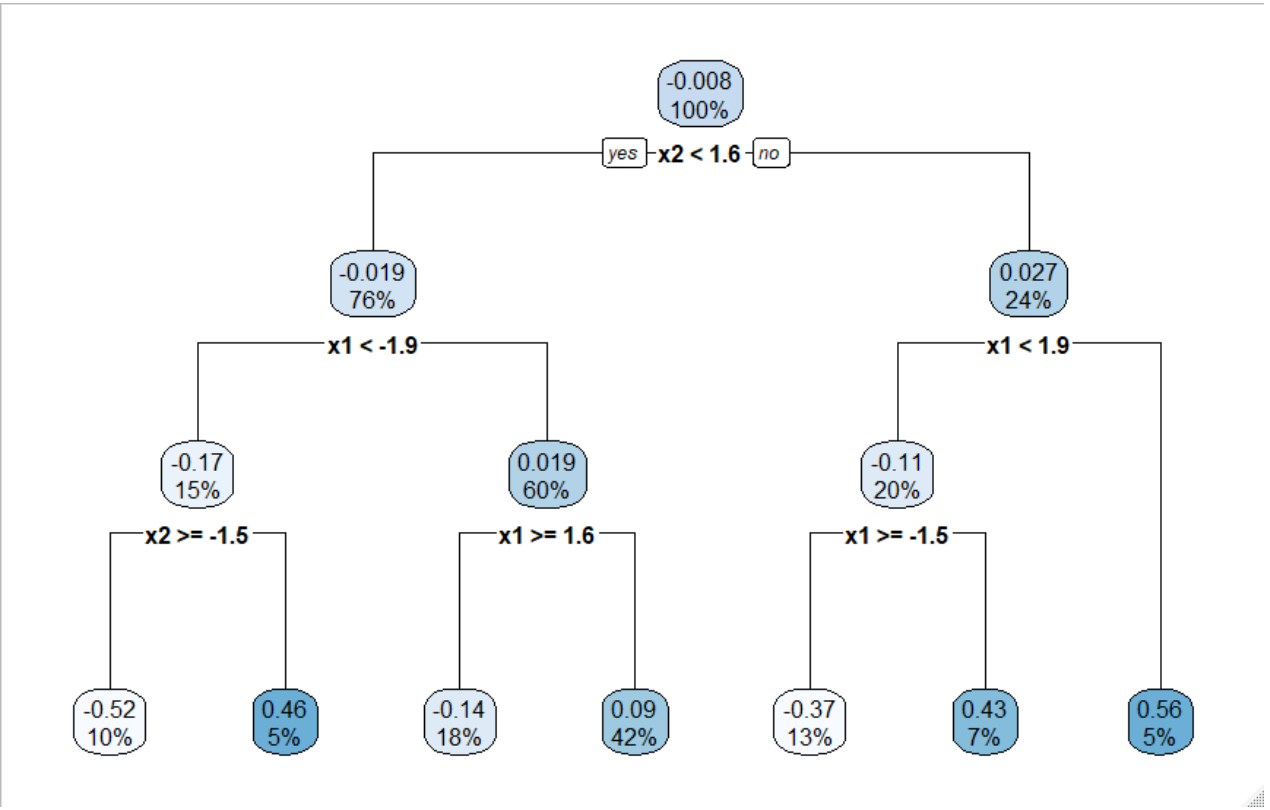
Fit a simple decision tree – interpretable results!

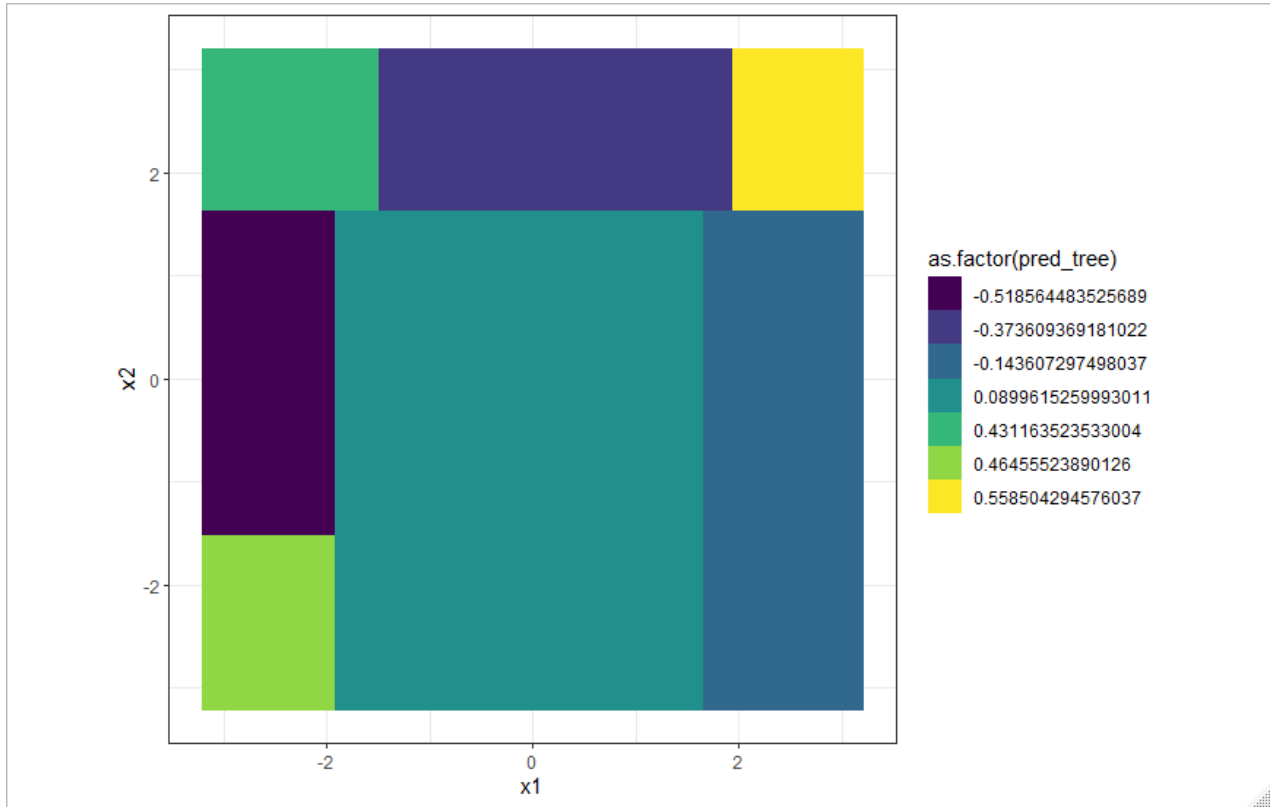
Start with the first split, is $x_2 < 1.6$?





Interpretations are the same even with a more complex tree!





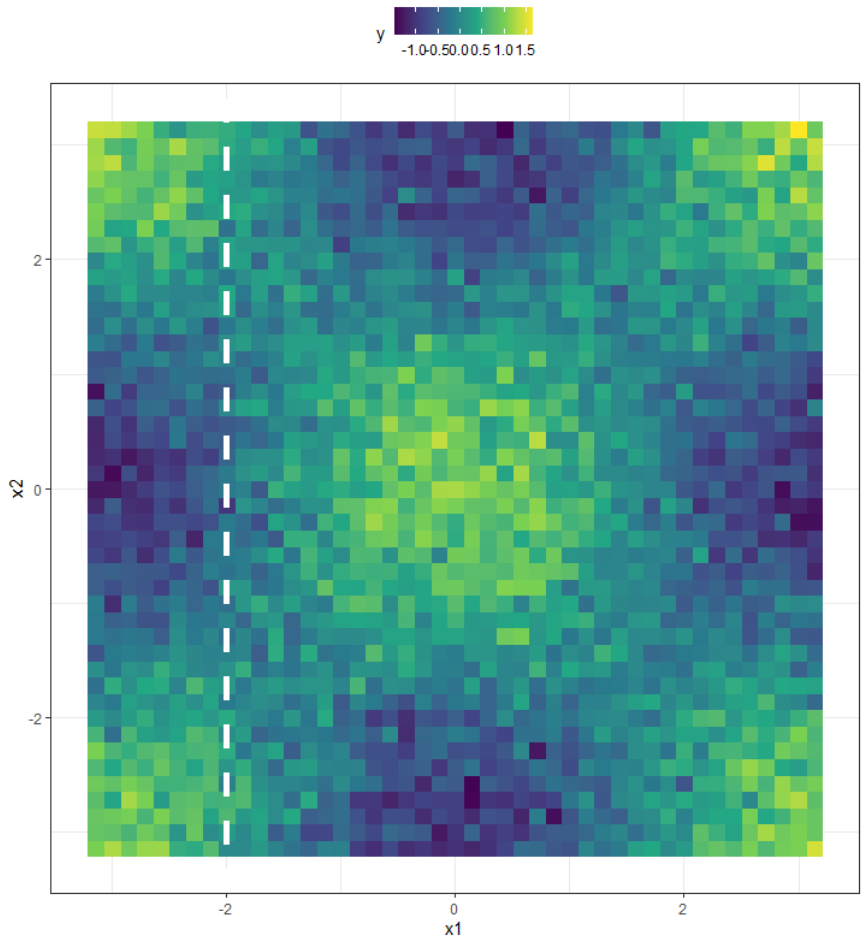
How are splits selected?

- Model searches **every distinct value** of **every input** to find the input, x_d, and value,$x_d = s_*$, that partitions the data into two regions that minimize the sum of squared errors (SSE).

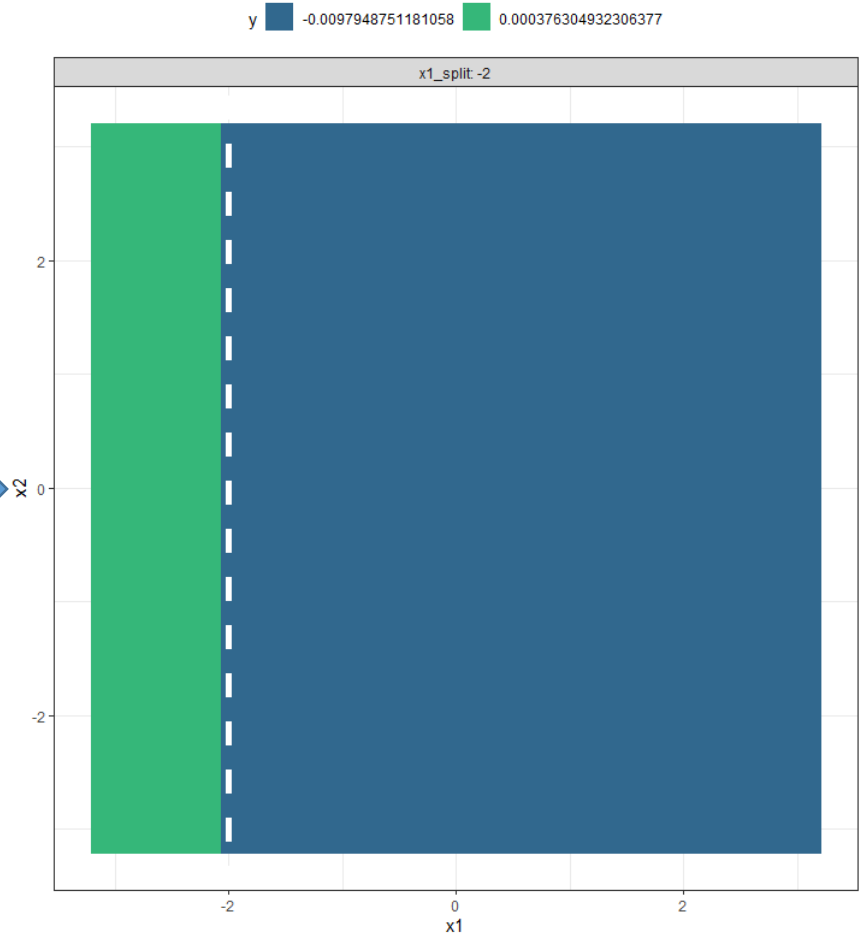
$$\min_{x_d, x_d = s_*} \left(\sum_{n \in R_1} \left((y_n - c_1)^2 \right) + \sum_{n \in R_2} \left((y_n - c_2)^2 \right) \right)$$

- c_1 is the average response in R_1, c_2 is the average response in R_2

Visualize how the top split is selected



Calculate average per region

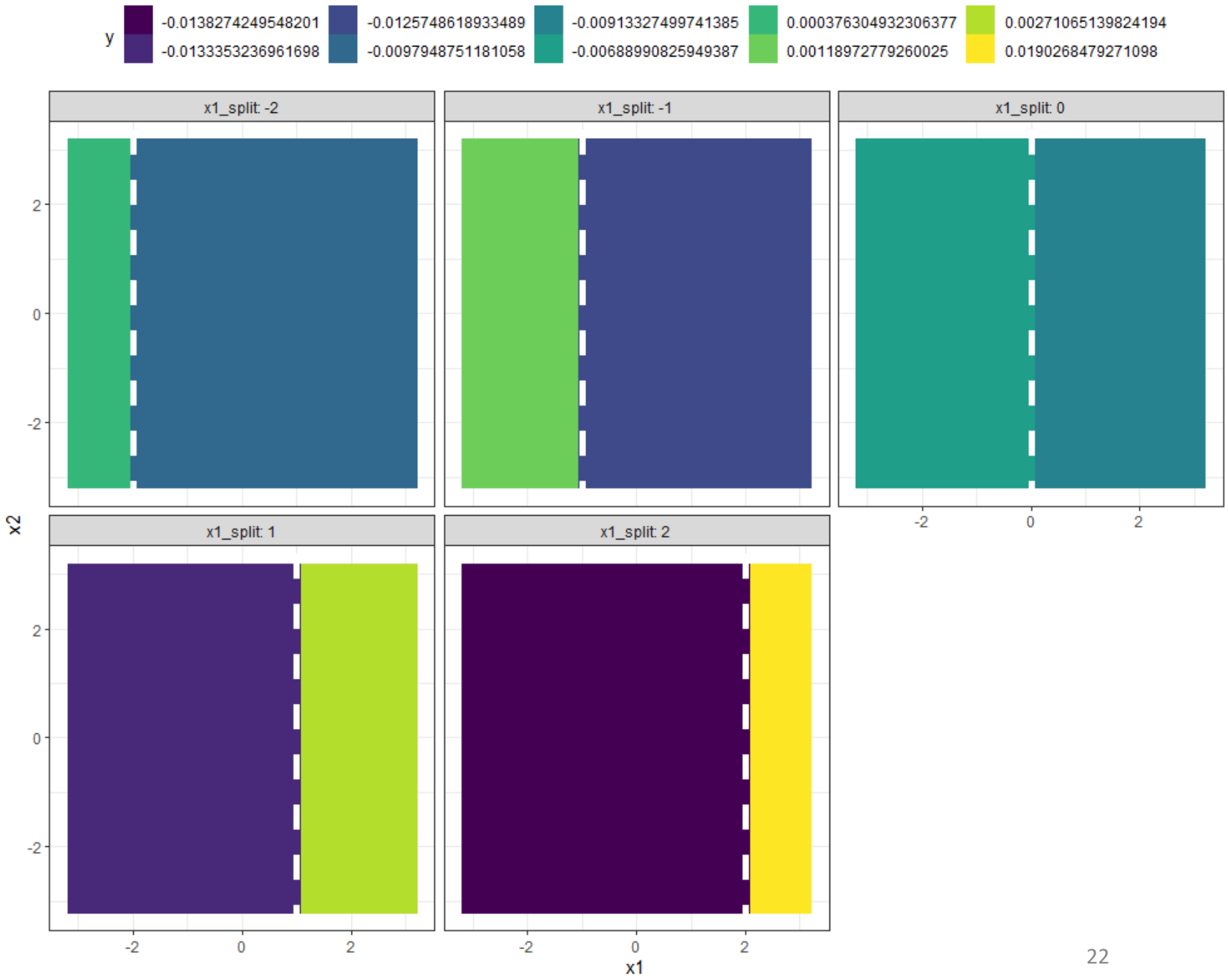


Select an input, specify a split point
Above figure shows $x_1 = -2$

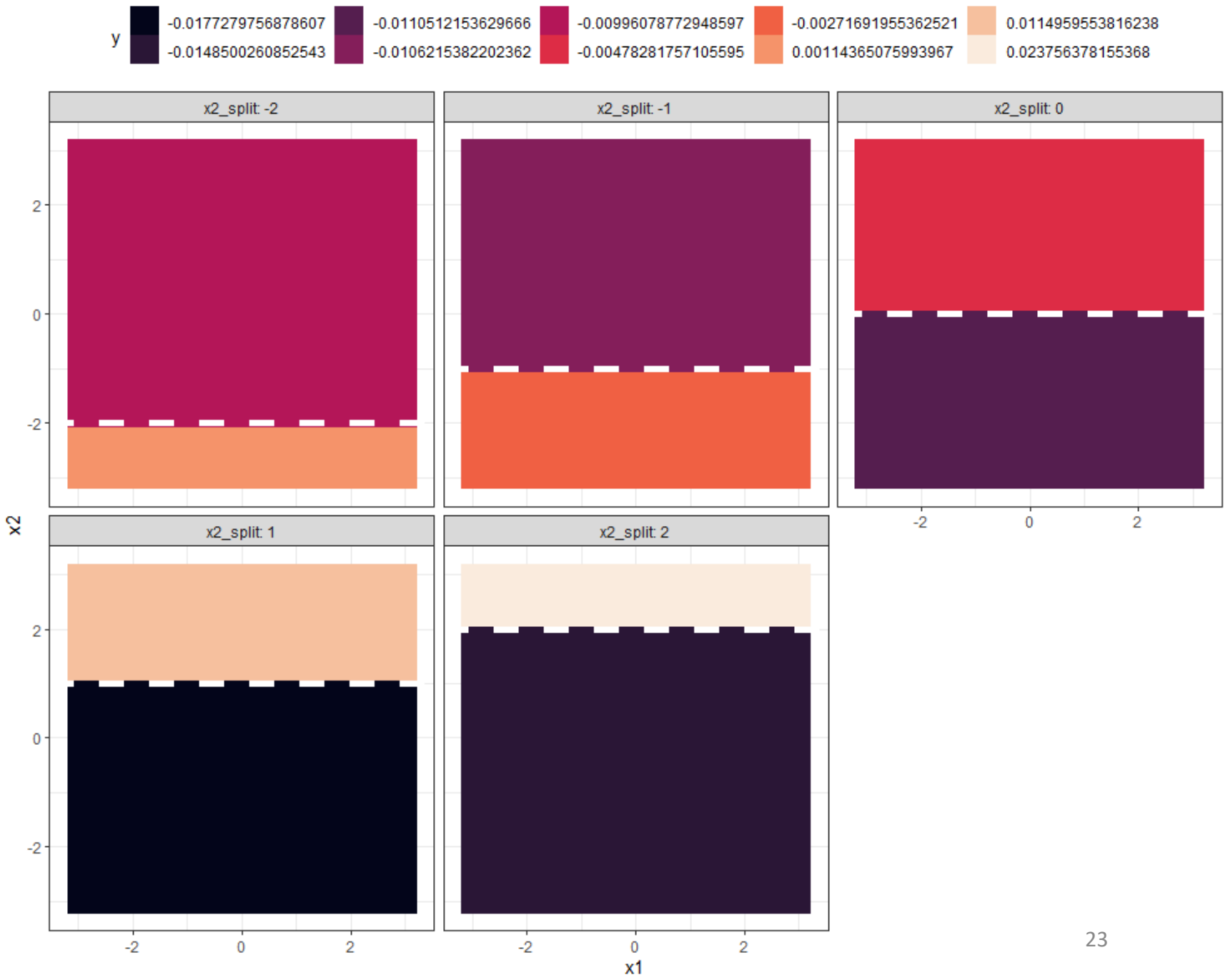
Change the value of split point associated with the input.

Calculate the average response in the TWO regions associated with each split point.

Repeat!!!

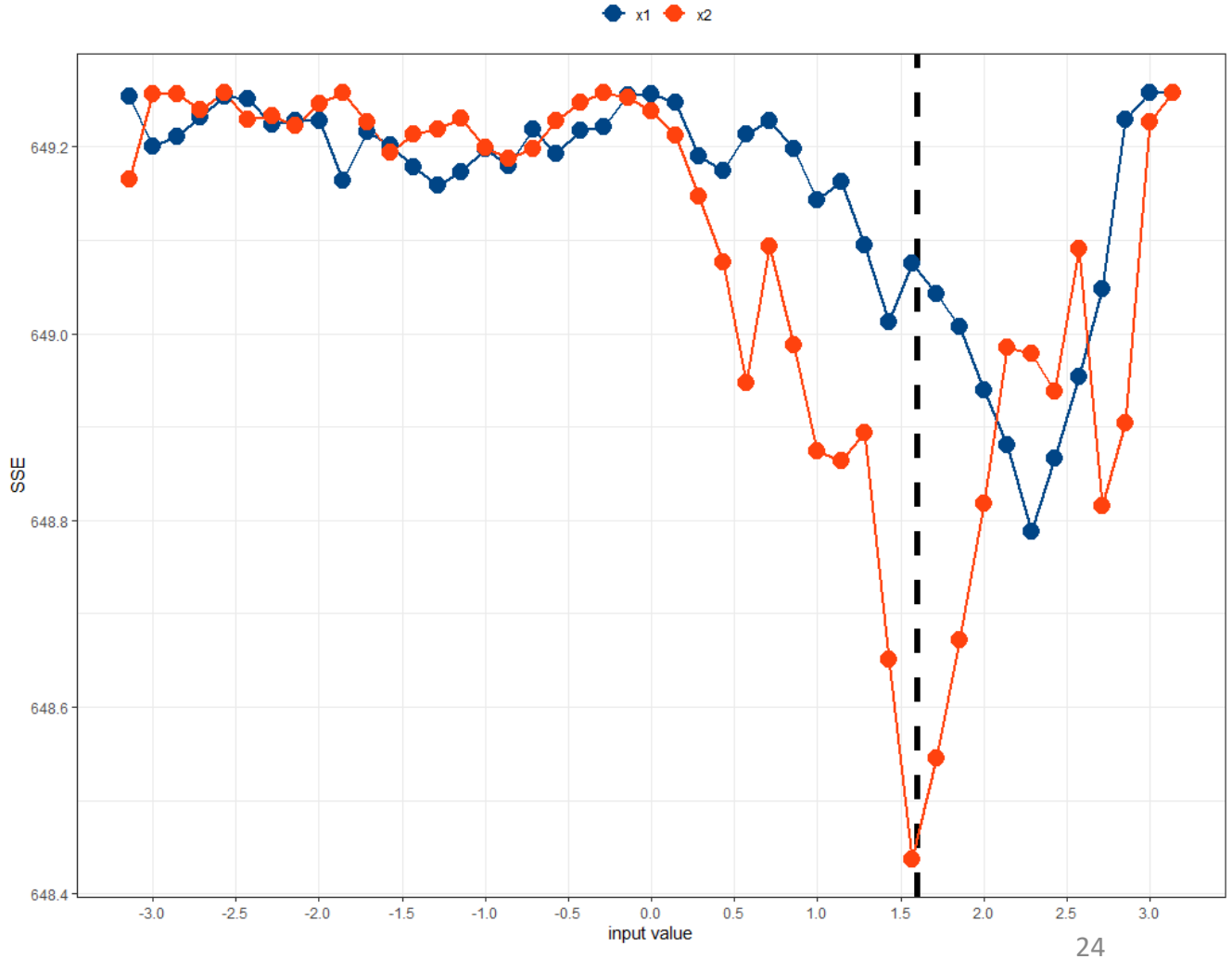


Repeat for every input!!!!!!!

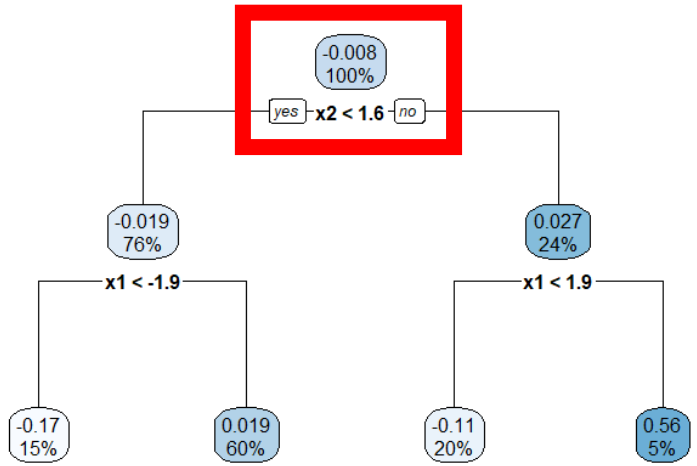


Select the input and the split point by minimizing the SSE!

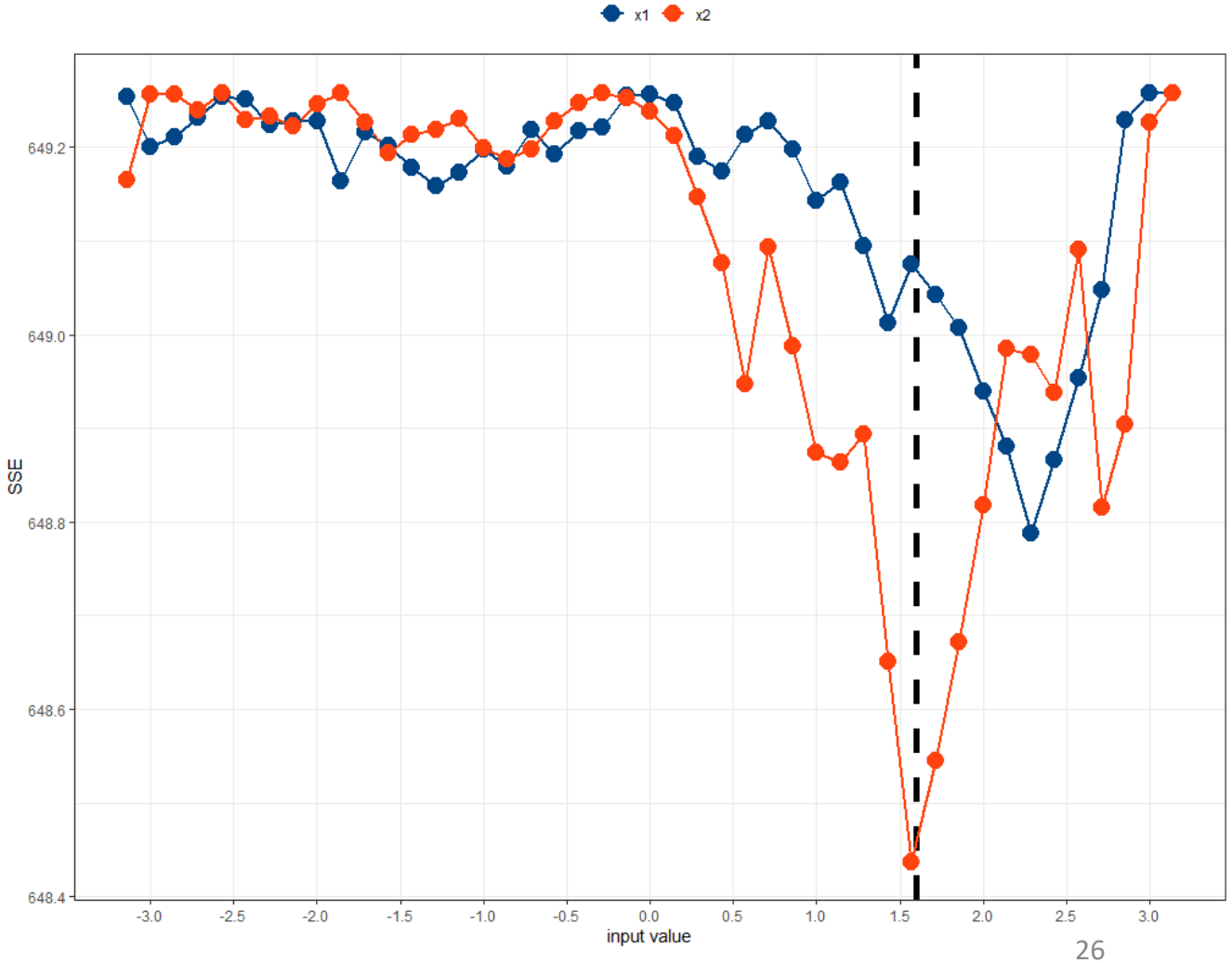
- Instead of trying just a few values per input, try every unique value!
- Calculate the SSE associated with each split point.
- The split value is chosen as that which MINIMIZES the SSE!



Select the input and the split point by minimizing the SSE!



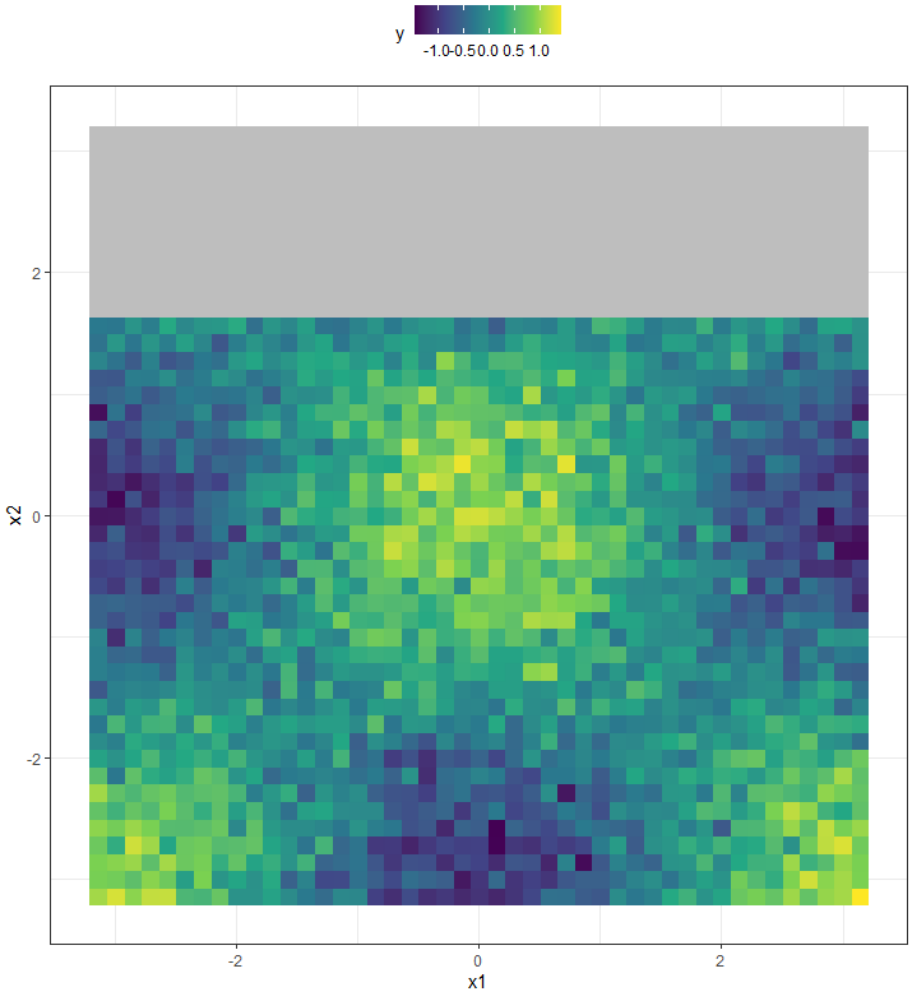
"Fitting" the tree is a SEARCH problem!!



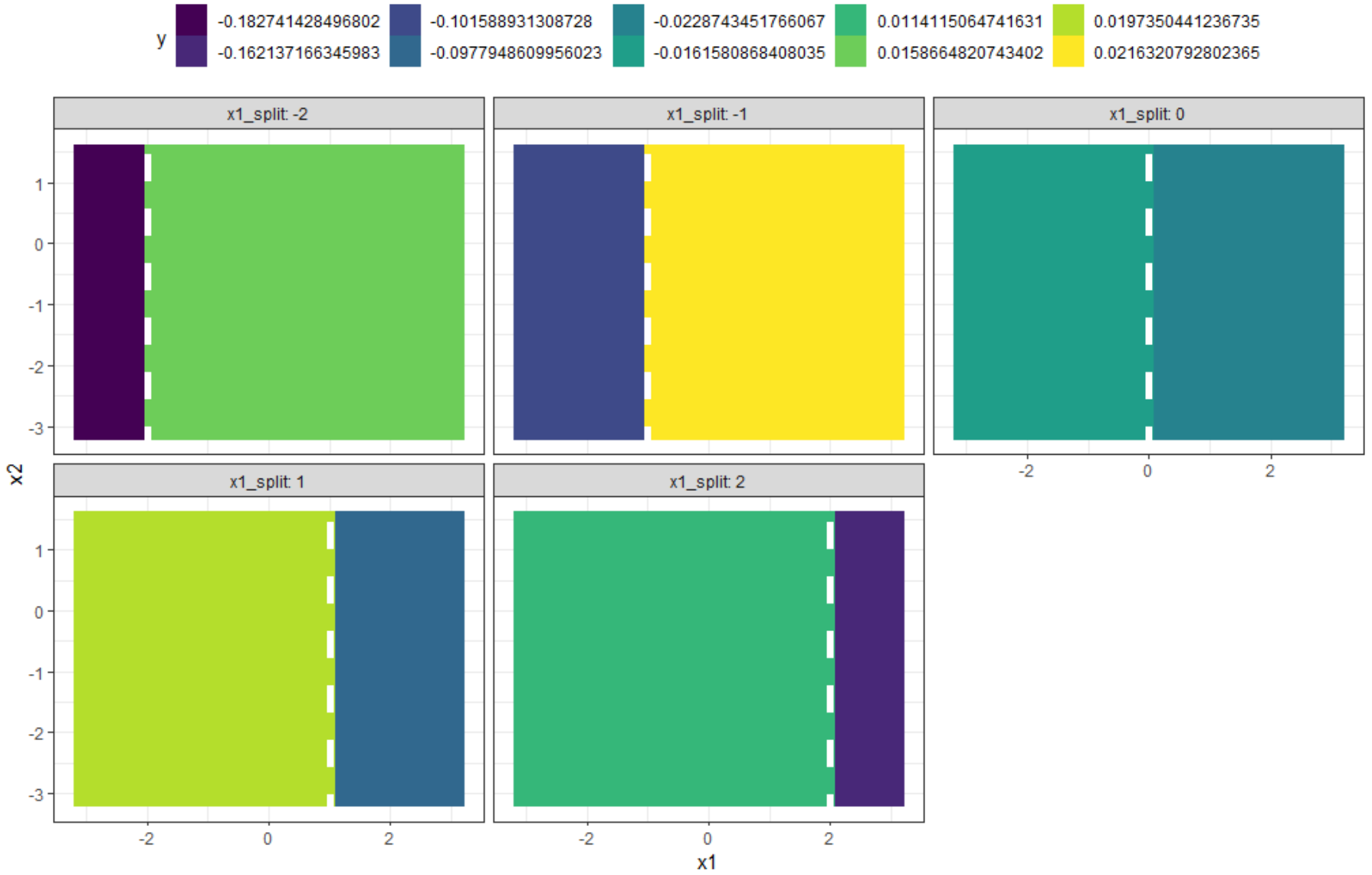
Subsequent splits are selected using the same process

- However, instead of working with the entire domain, the fitting is associated within the new regions.

- For example, the figure to the right shows the $x_2 < 1.6$ region.

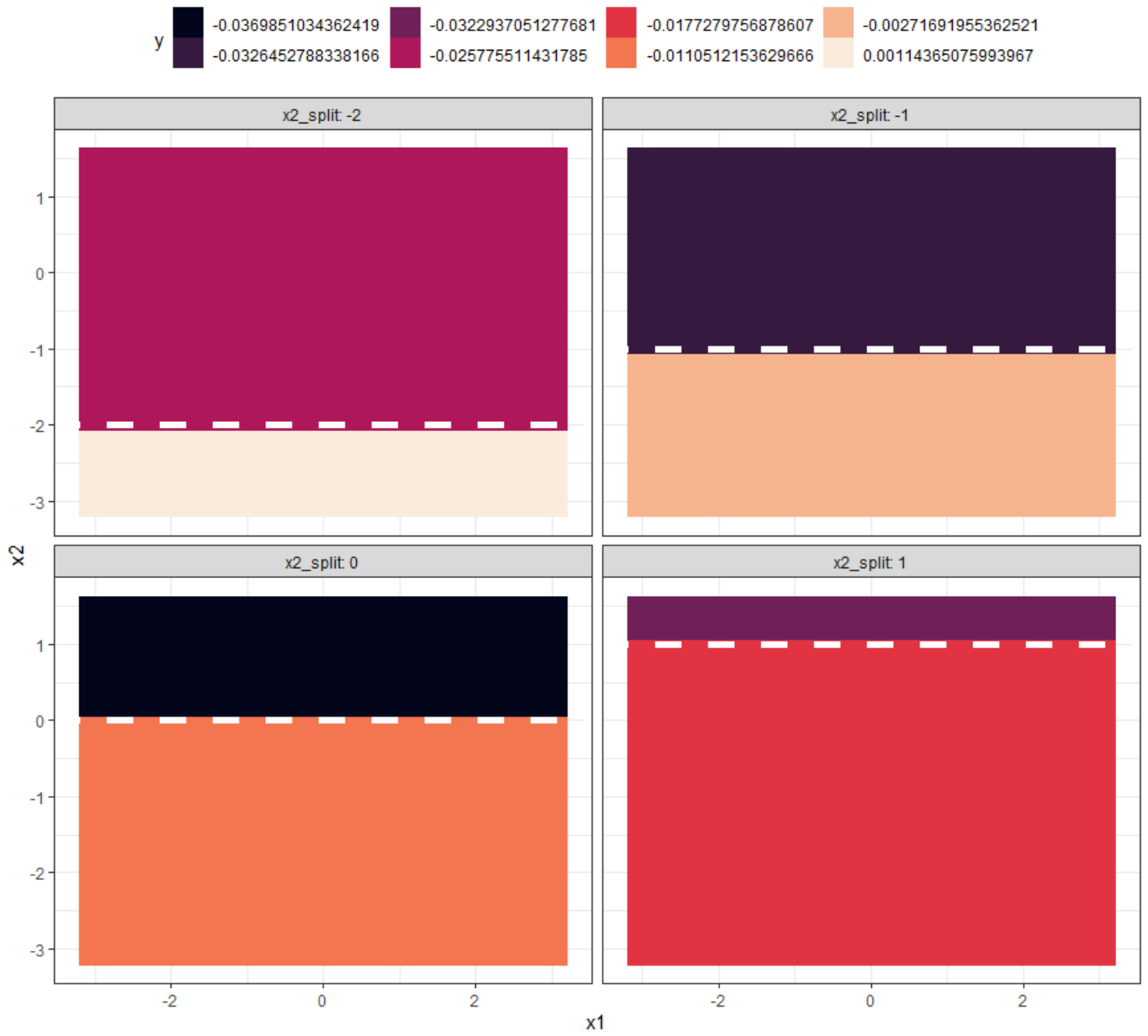


Split the $x_2 < 1.6$ region into 2 smaller zones!

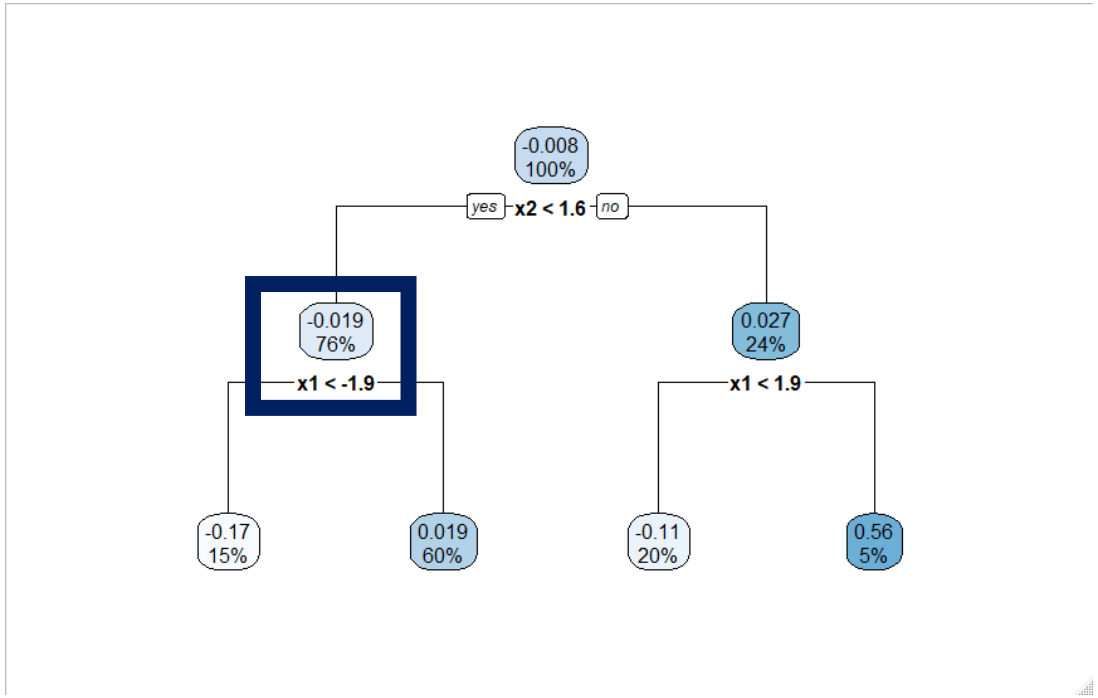


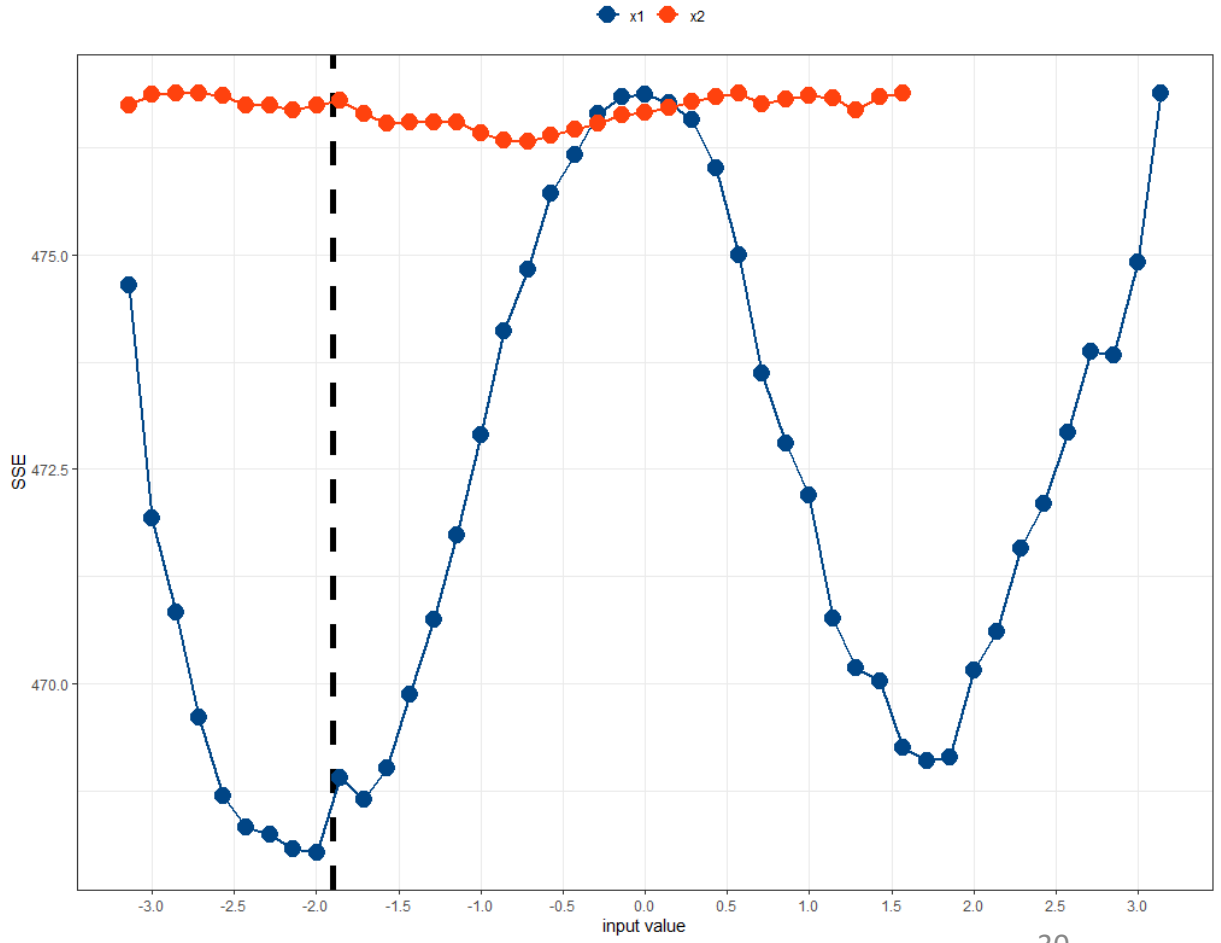
First, split by x_1

Then, split by x_2



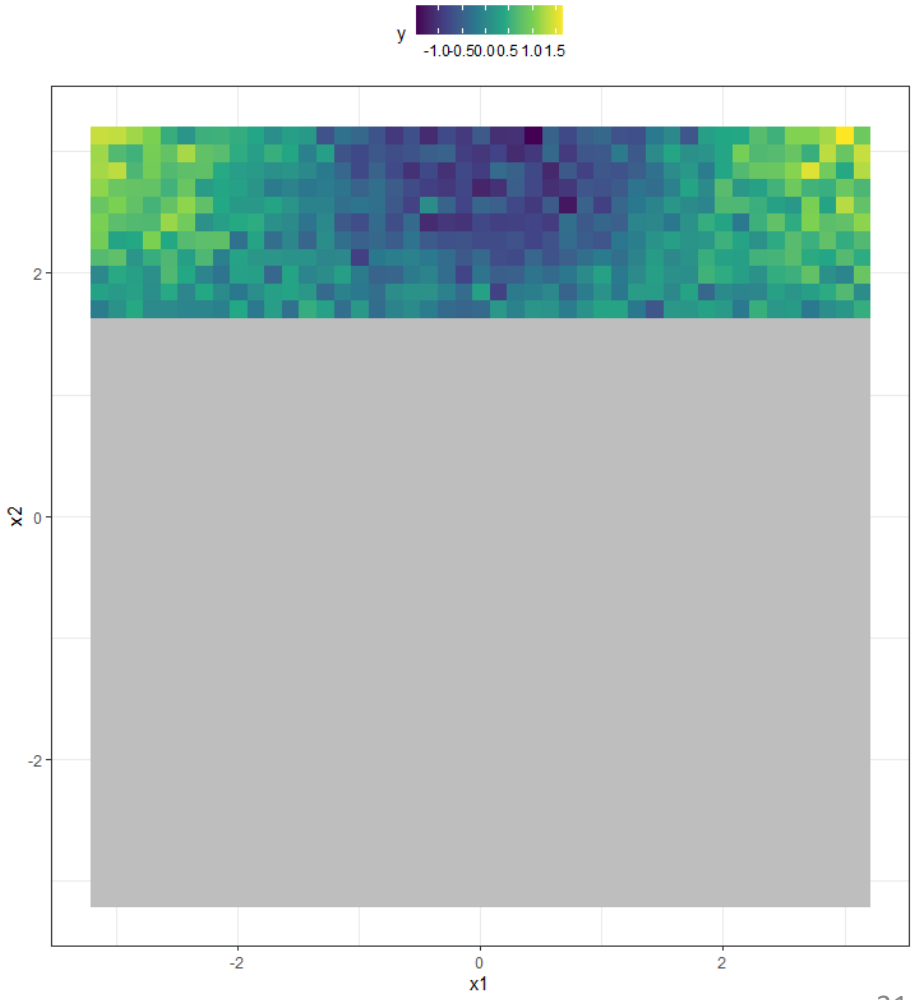
The next split is selected by minimizing the SSE within that region!



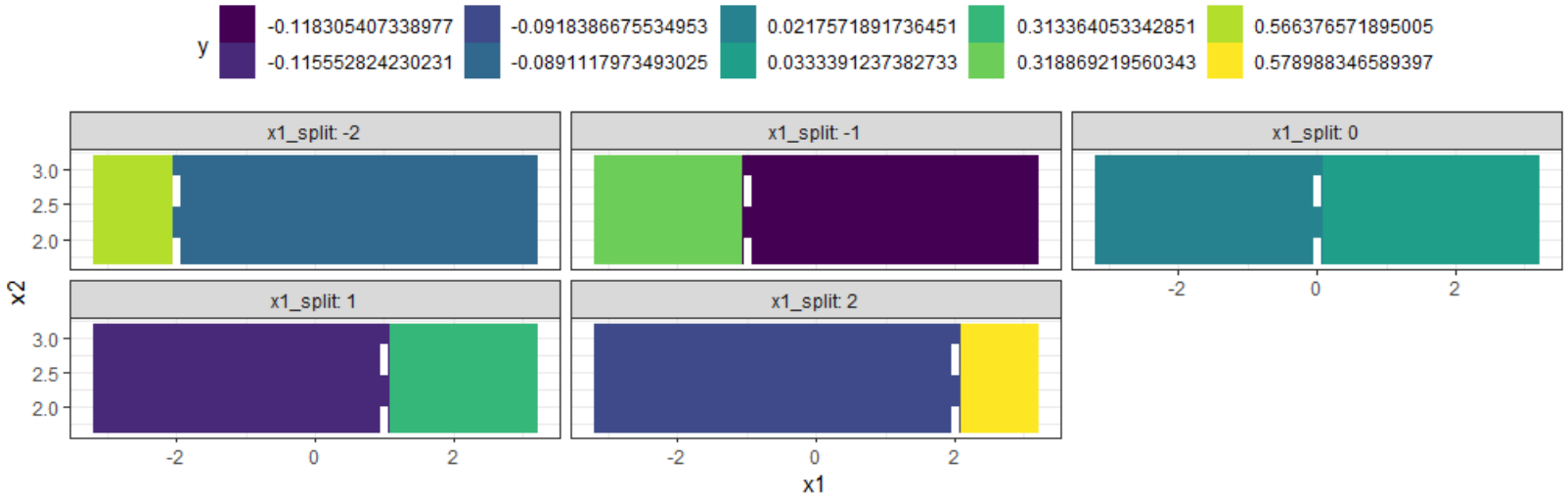


The second, smaller, region created by the top split is split following this same process

- The figure to the right shows the $x_2 \geq 1.6$ region.
- This smaller region only has access to a reduced number of unique values for the x_2 variable.
- That's ok! Split this region into two smaller zones!

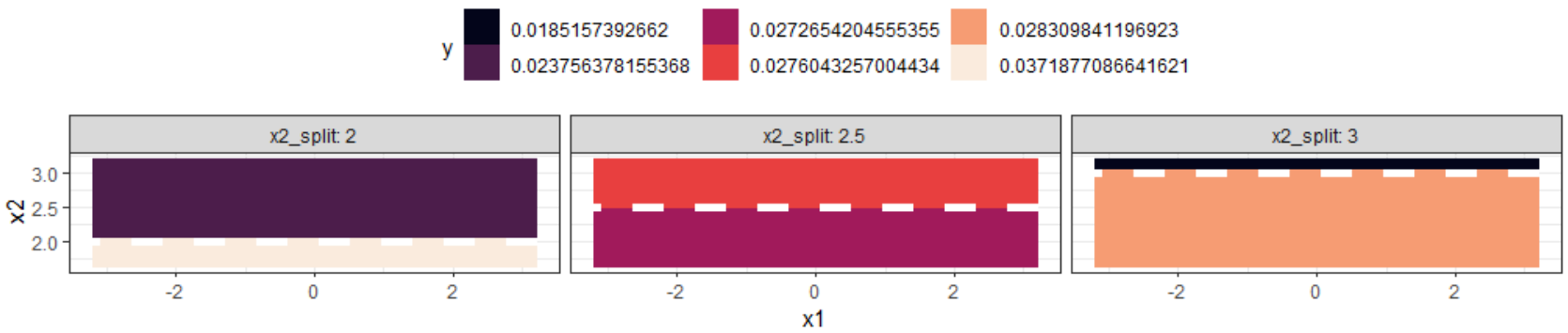


Split the $x_2 \geq 1.6$ region into 2 smaller zones!

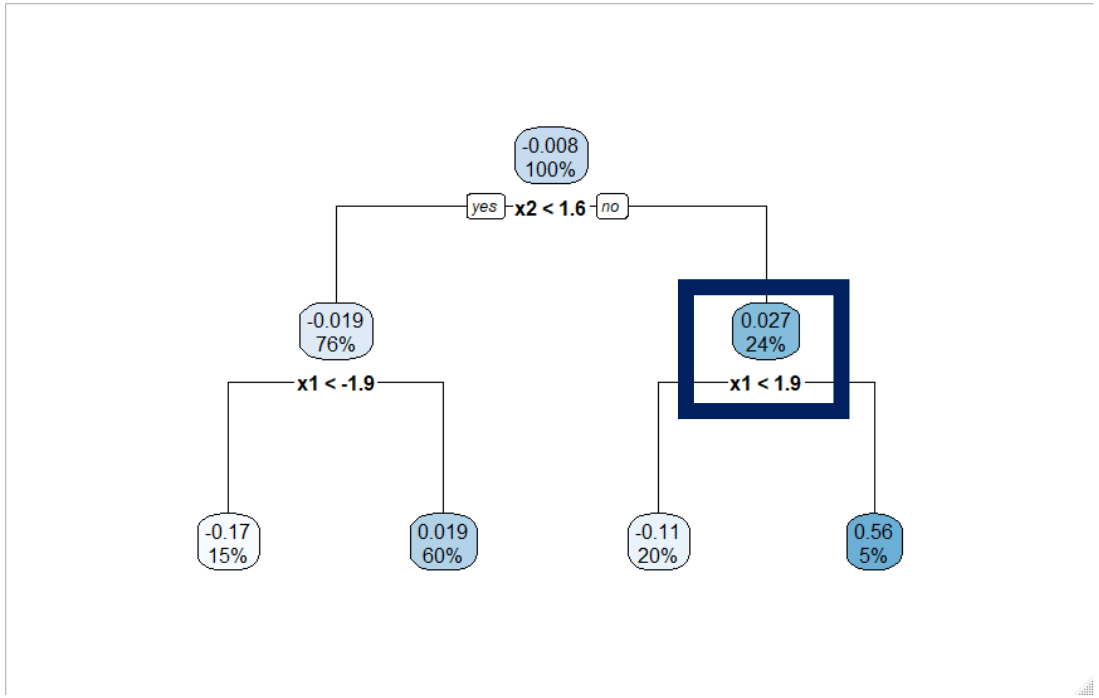


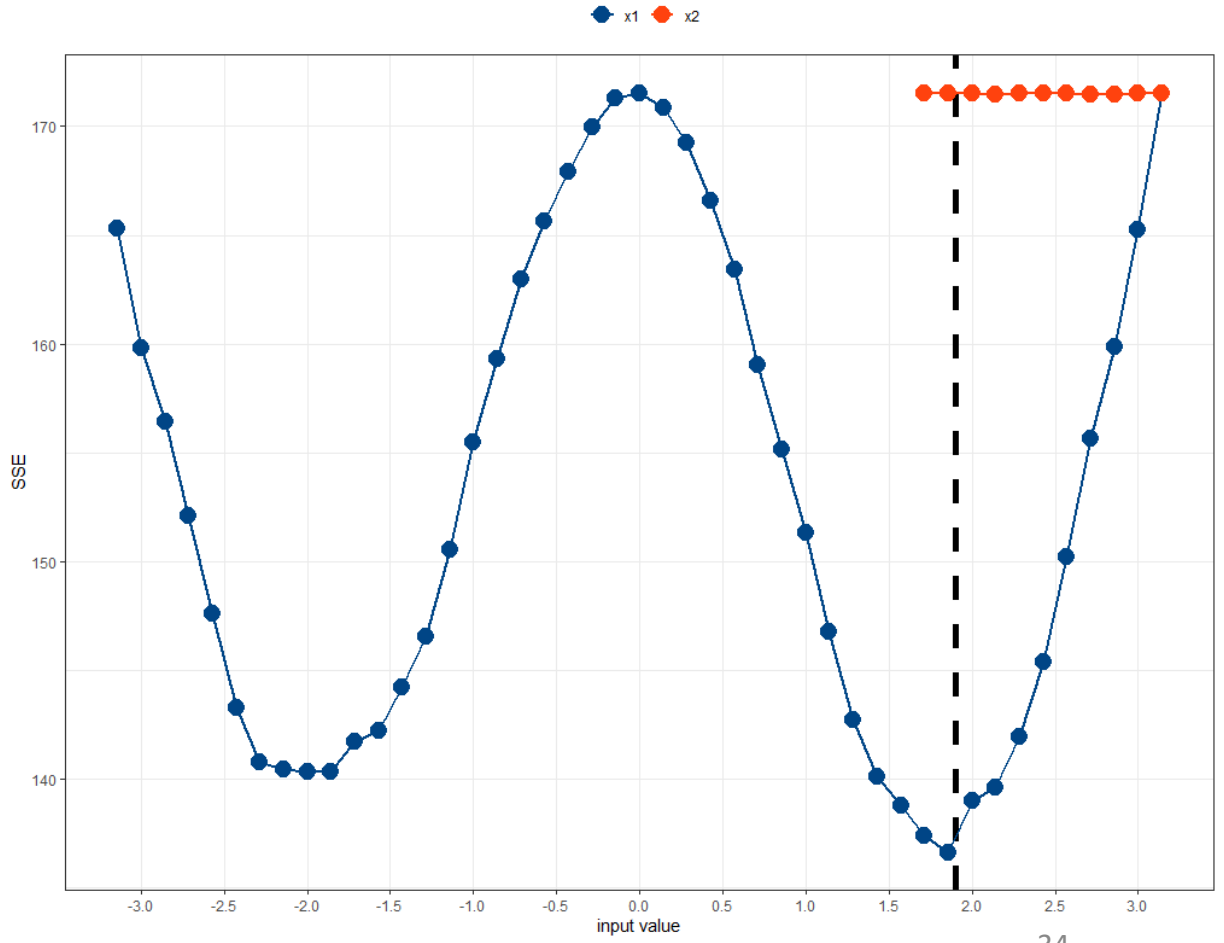
First, split by x_1

Then, split by x_2

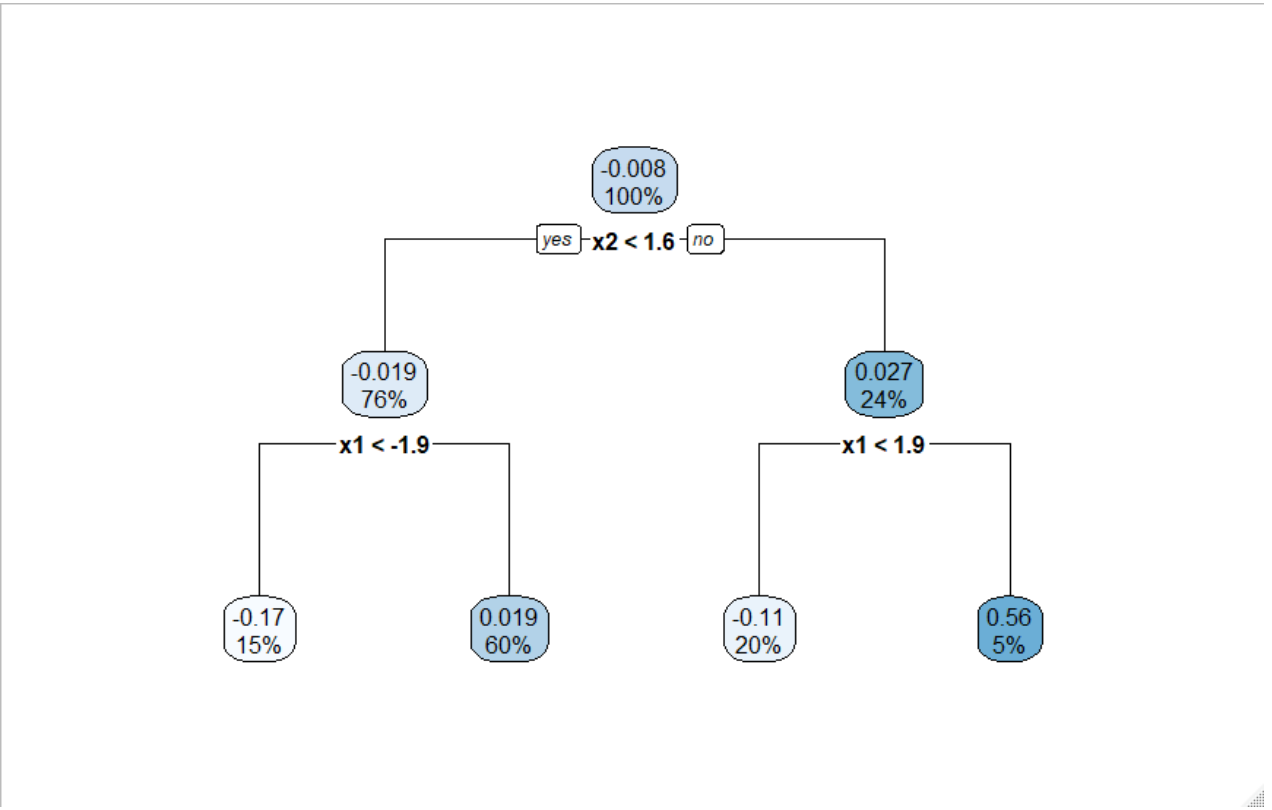


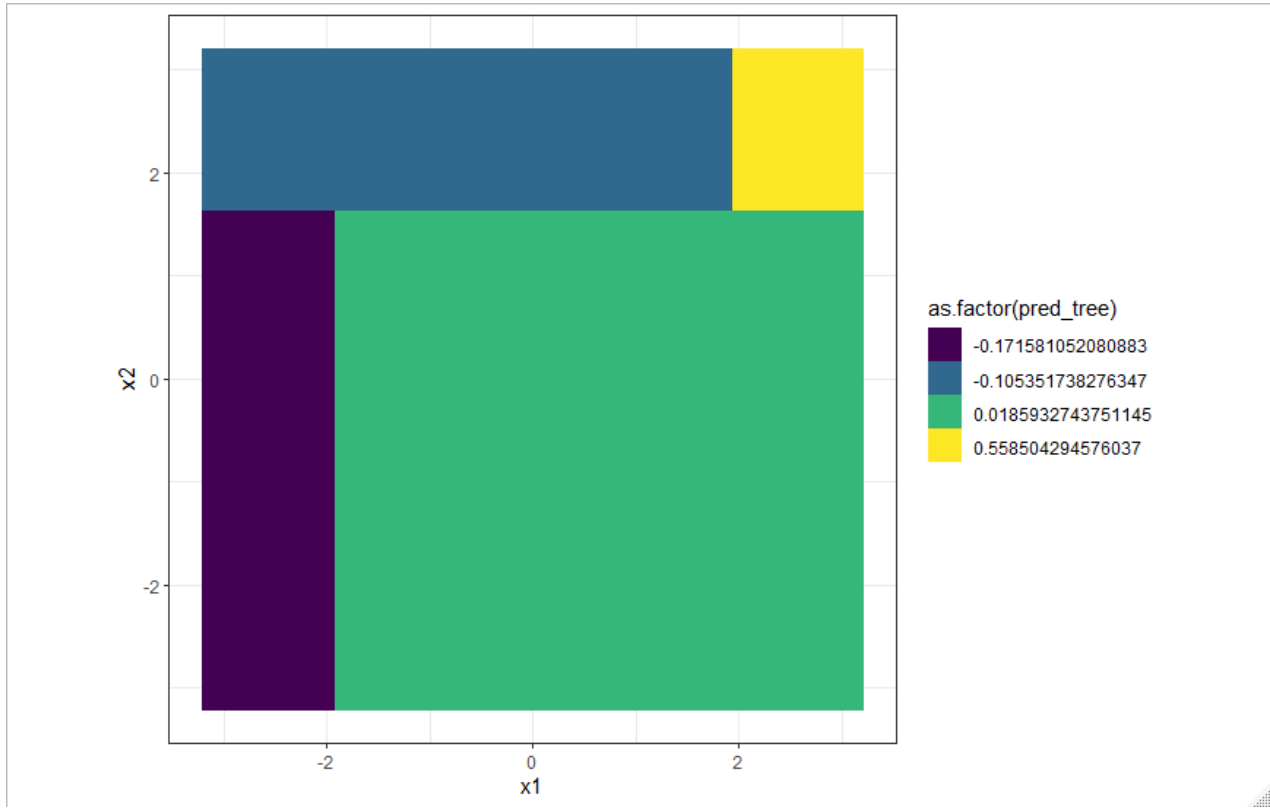
Splitting value selected by minimizing the SSE





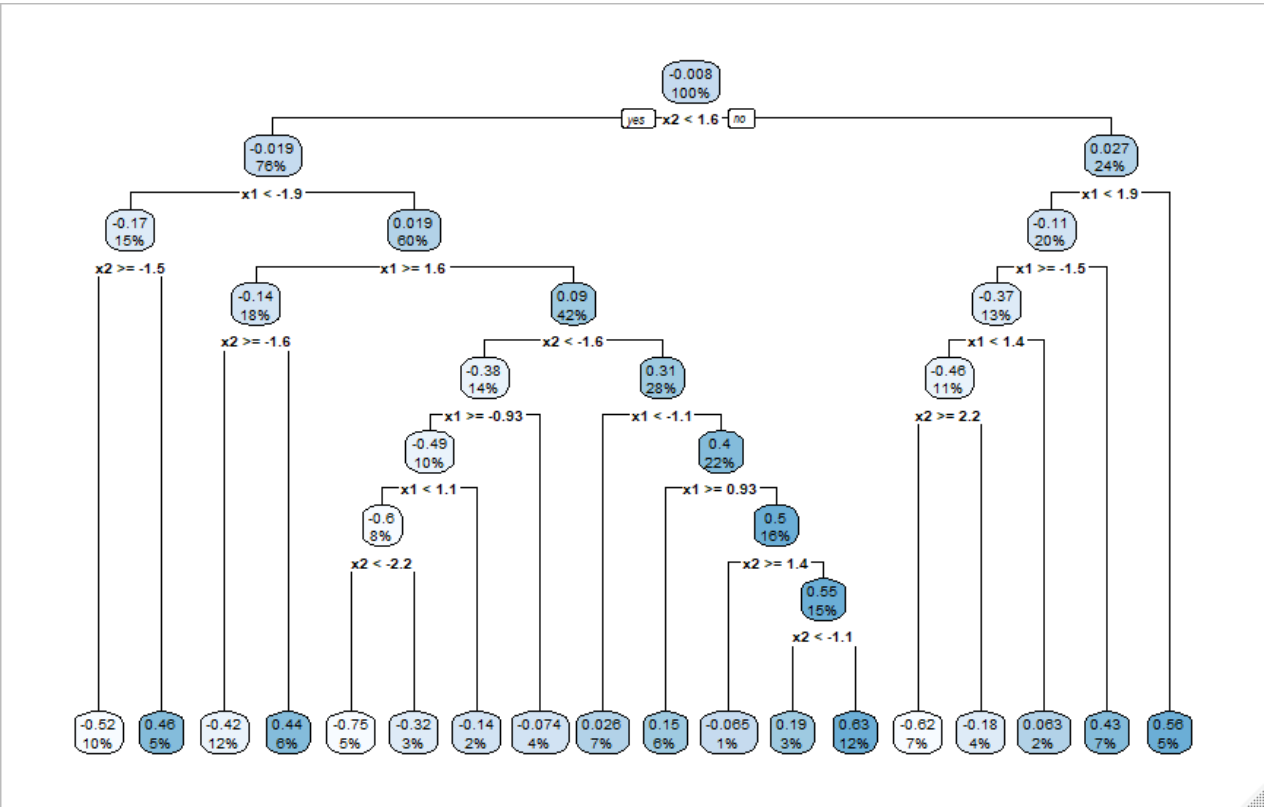
Splits are decided by finding the input value that improves the model's fit as much as possible!

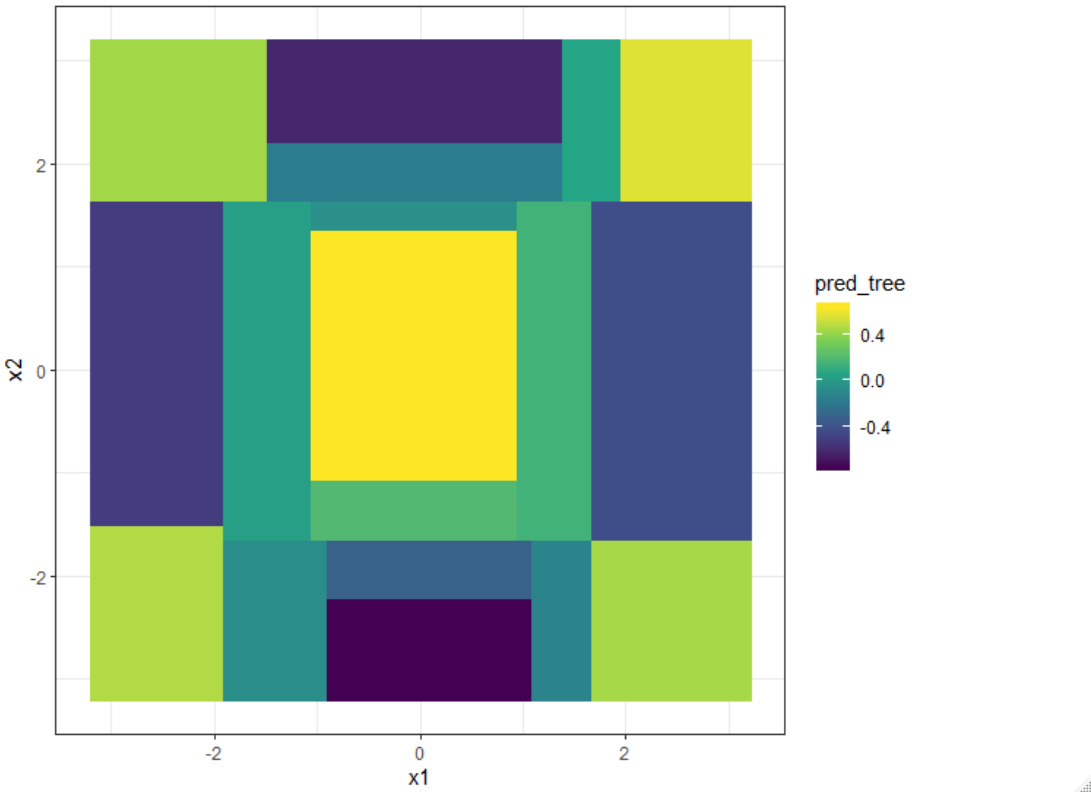




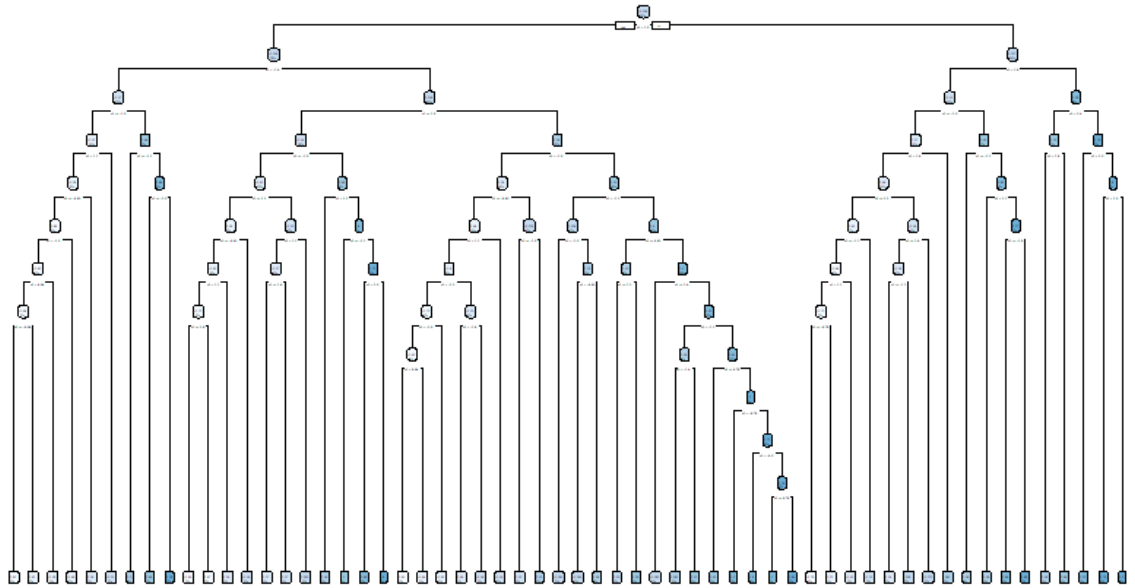
Splits are created RECURSIVELY!!

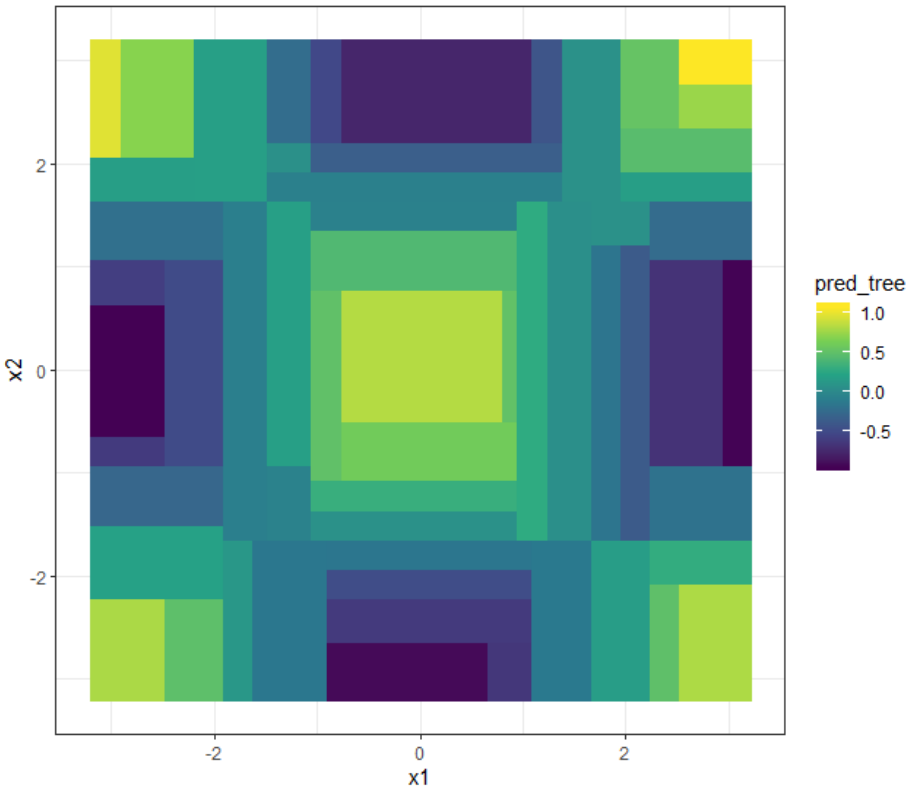
Continue “growing” the tree…



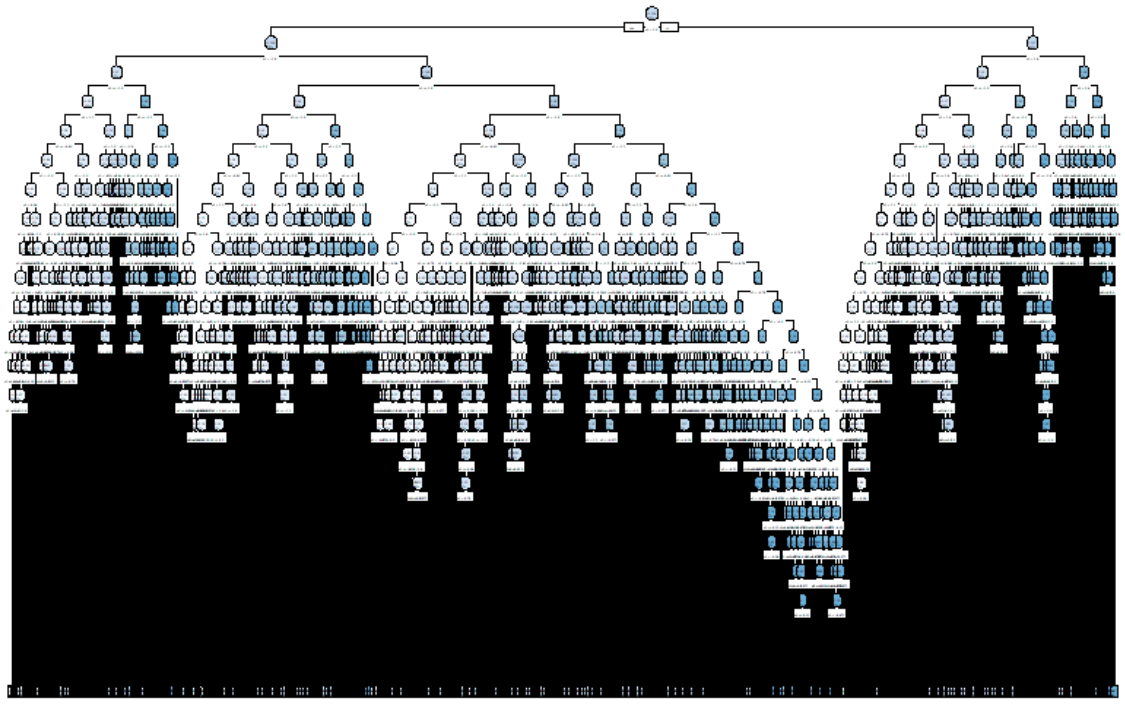


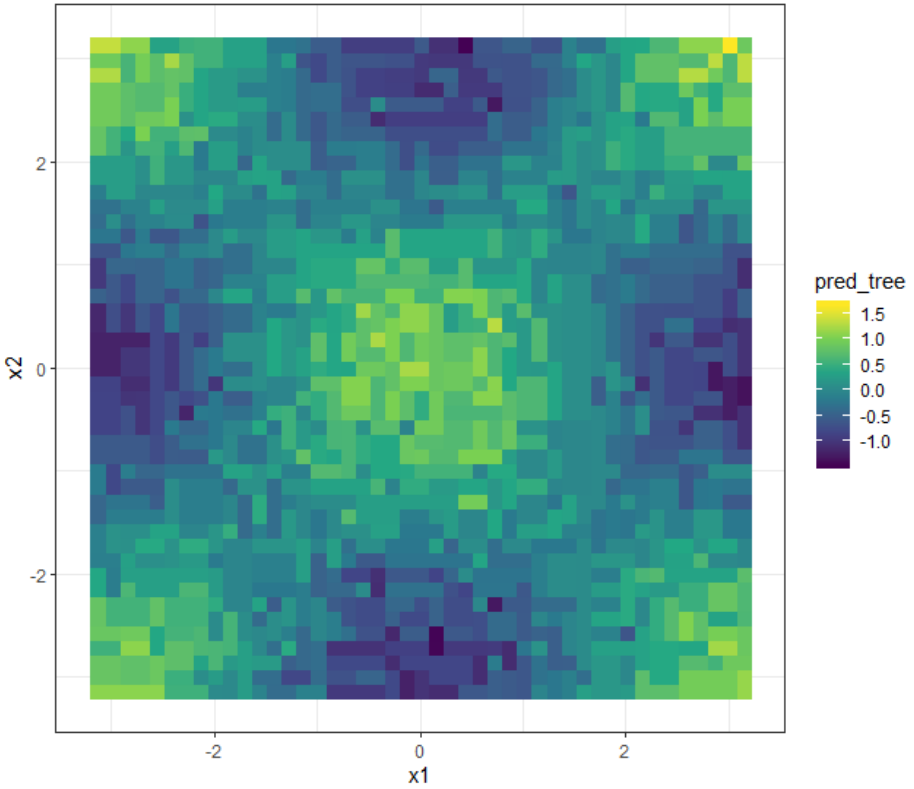
Continue “growing” the tree…





Continue “growing” the tree…





The tree is now so complex…it’s modeling noise!!

When should we stop growing the tree?

- Rather than picking the stopping criteria upfront, build a deep tree and then reduce it's depth by **pruning**.

- Find the optimal sub-tree through a **cost complexity criterion**.

- Penalize a tree based on the number of terminal nodes, $|T|$.

$$\min\left(\text{SSE} + c_p|T|\right)$$

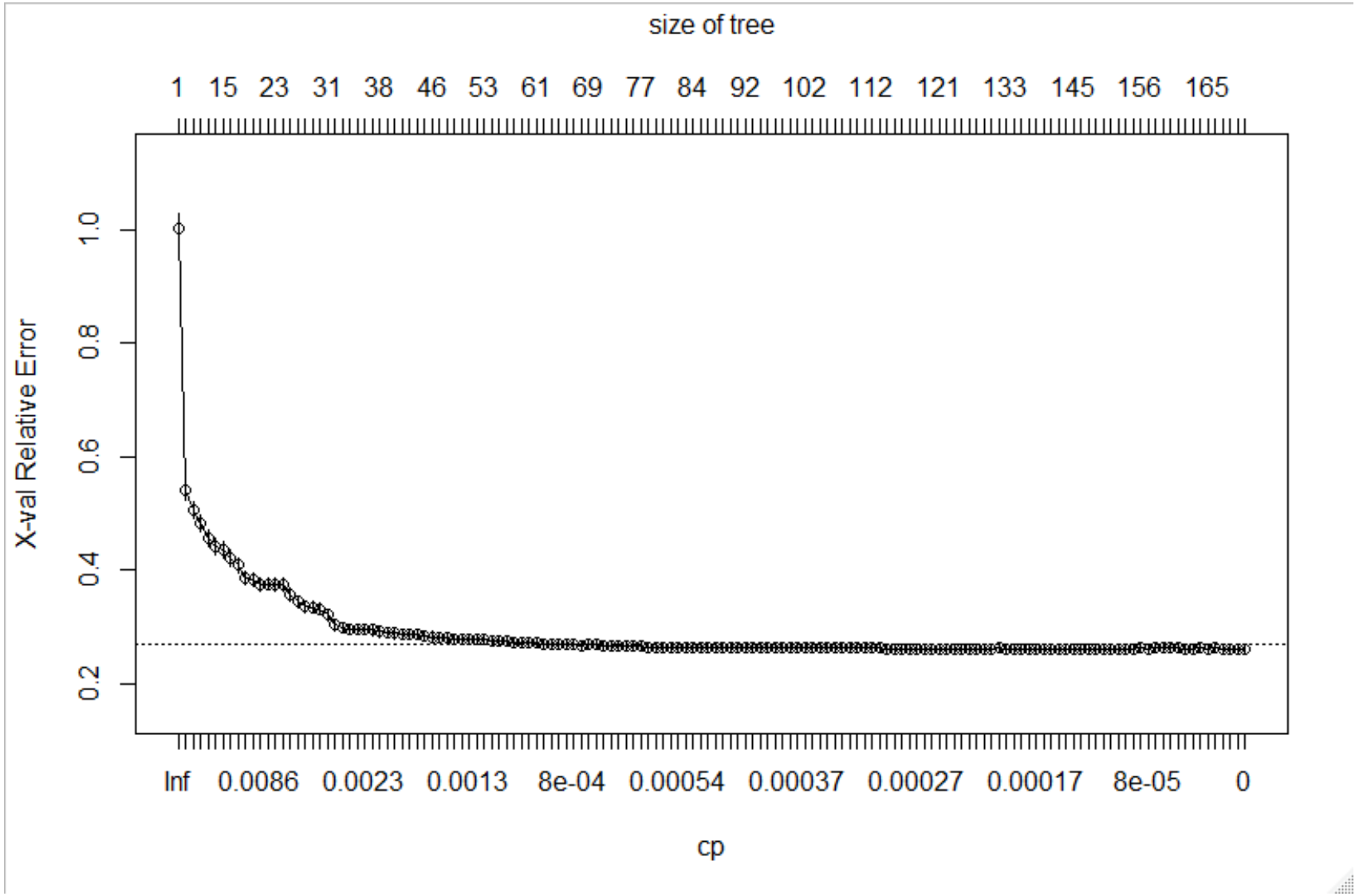
When should we stop growing the tree?

- Rather than picking the stopping criteria upfront, build a deep tree and then reduce it's depth by **pruning**.
- Find the optim **y criterion**.
- Penalize a tree des, $|T|$.

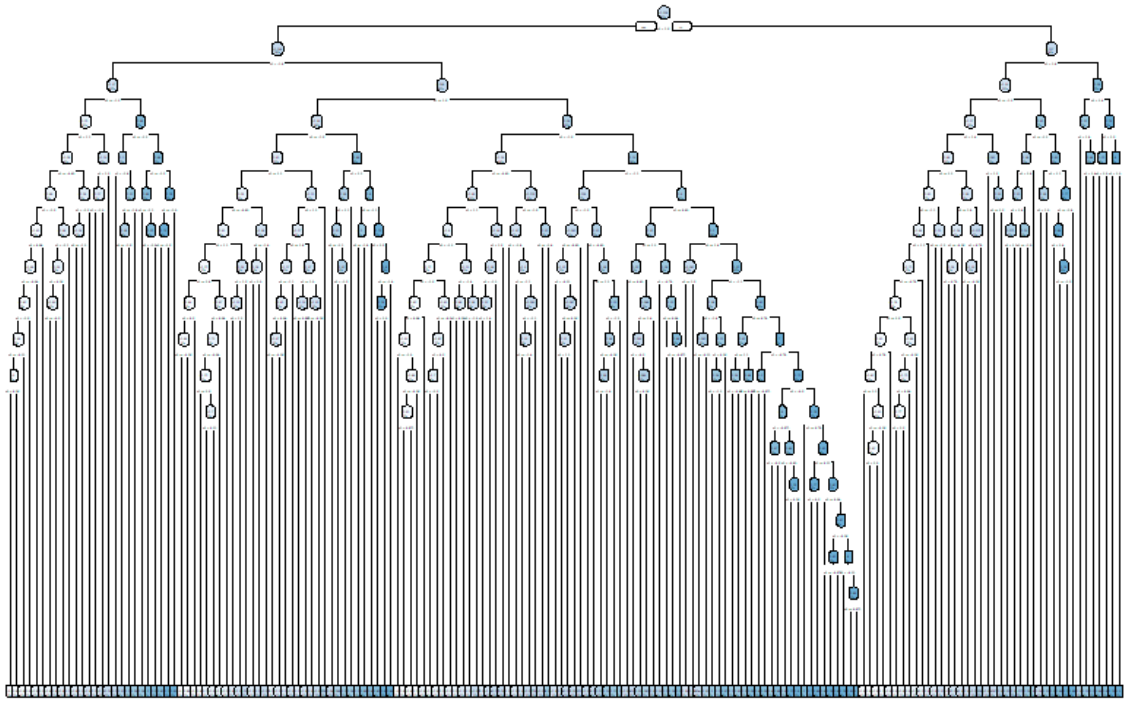
c_p is analogous to the penalty factor in Lasso regression!

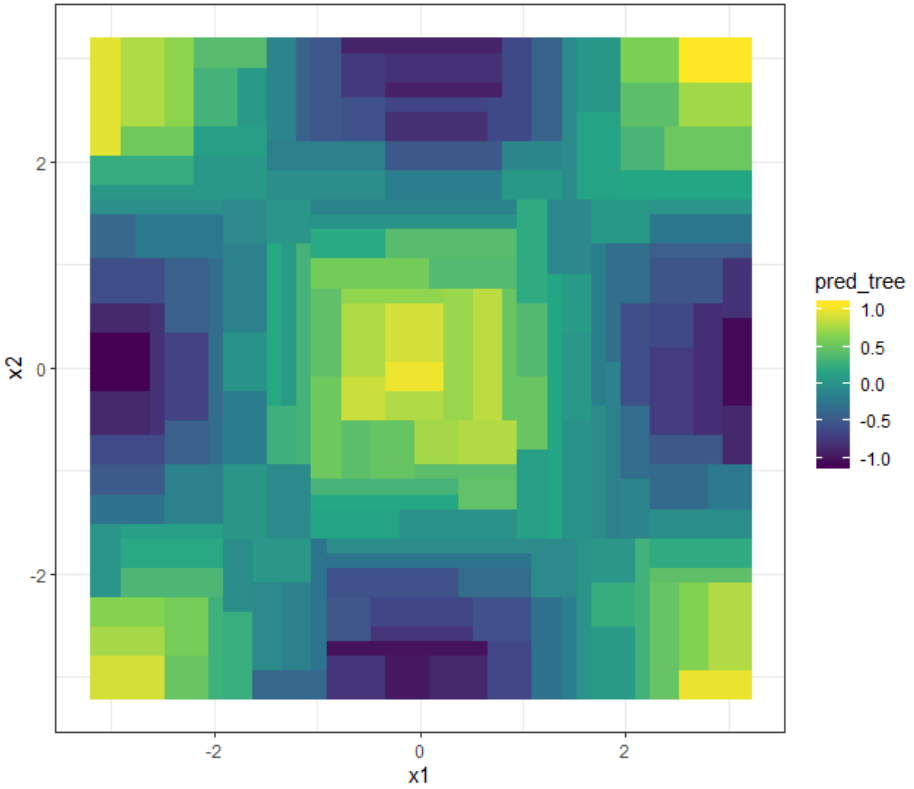
$$\min(\text{SSE} + c_p|T|)$$

Use cross-validation to tune the complexity parameter

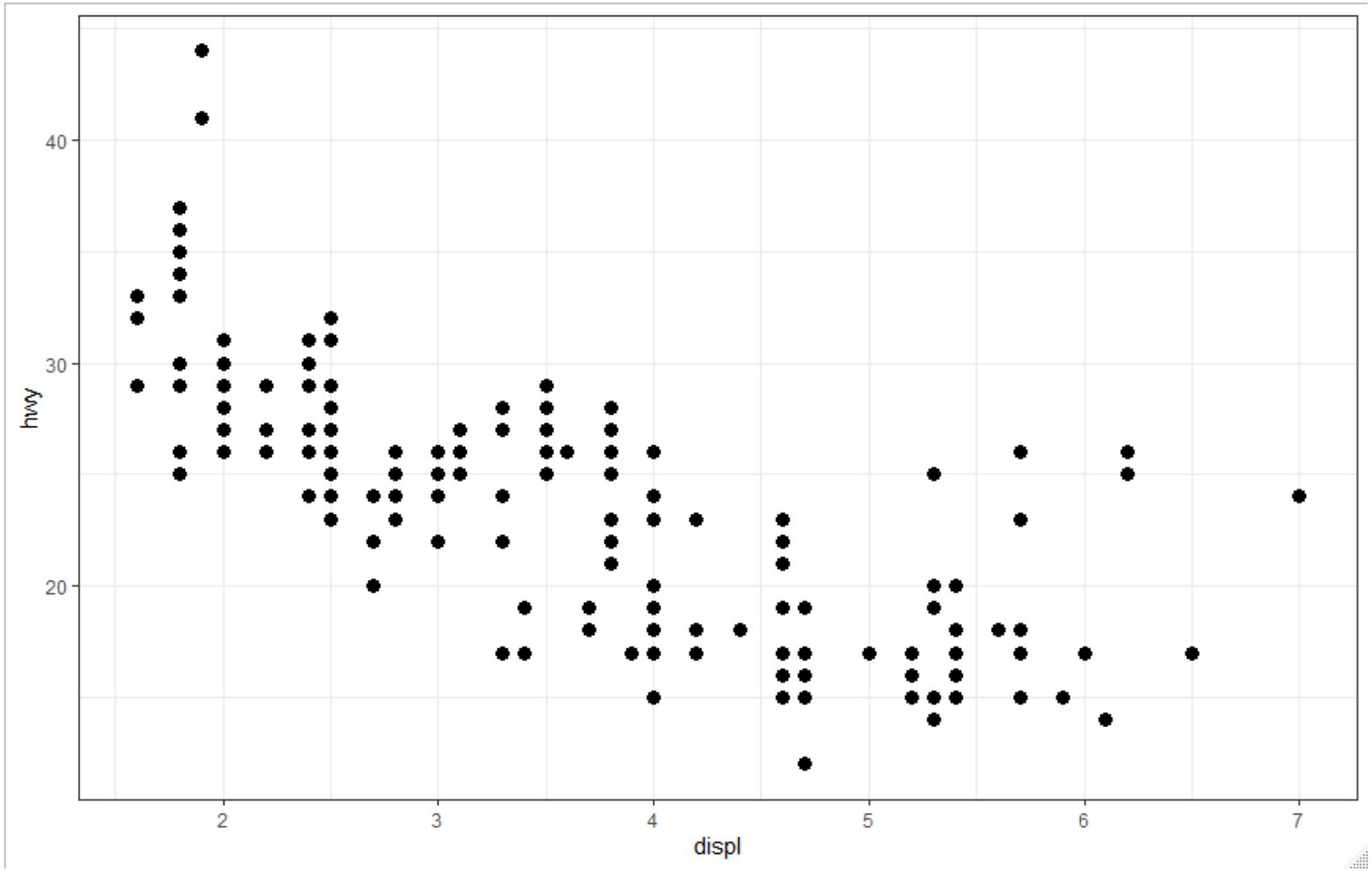


Optimally pruned sub-tree

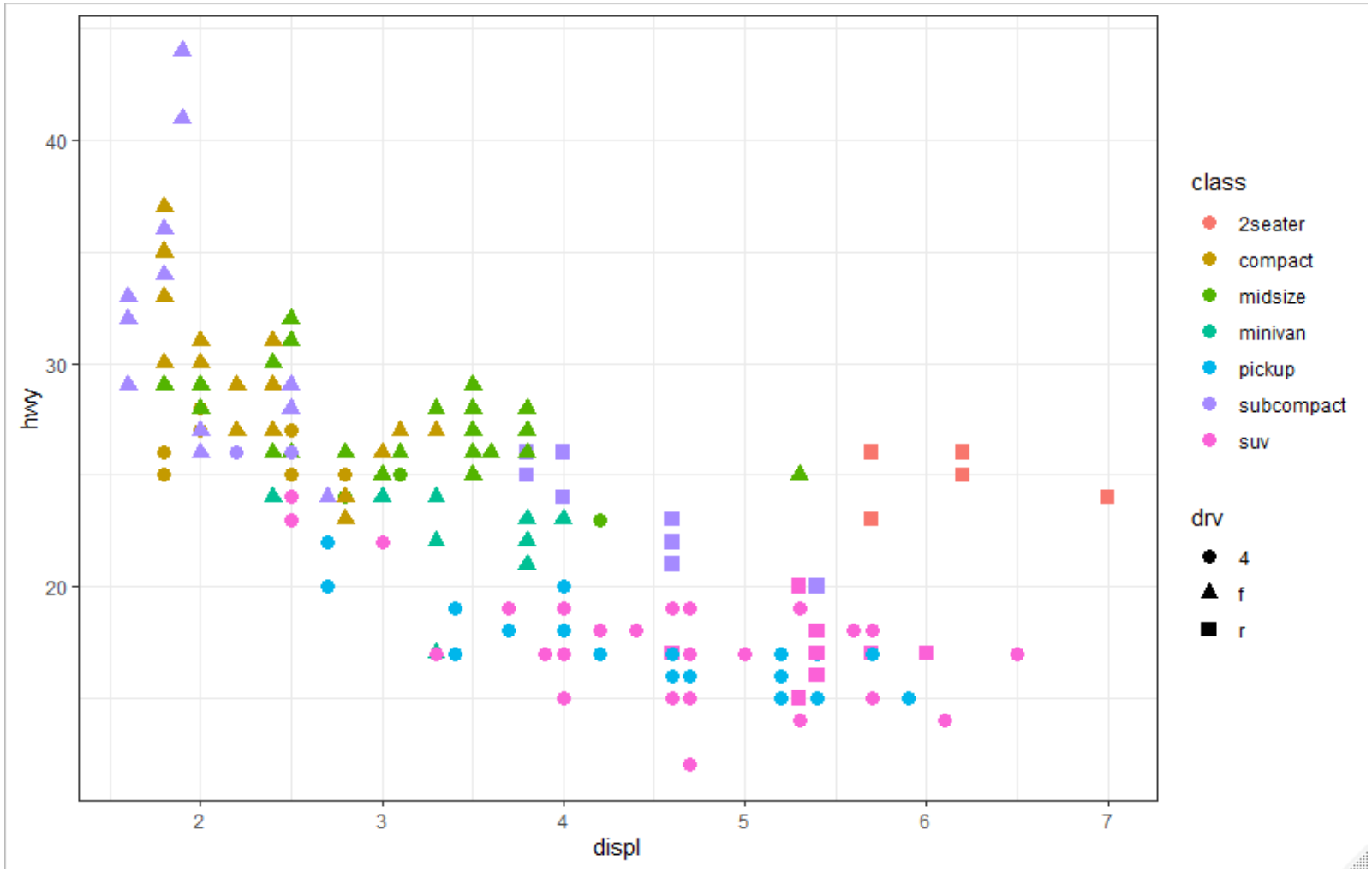




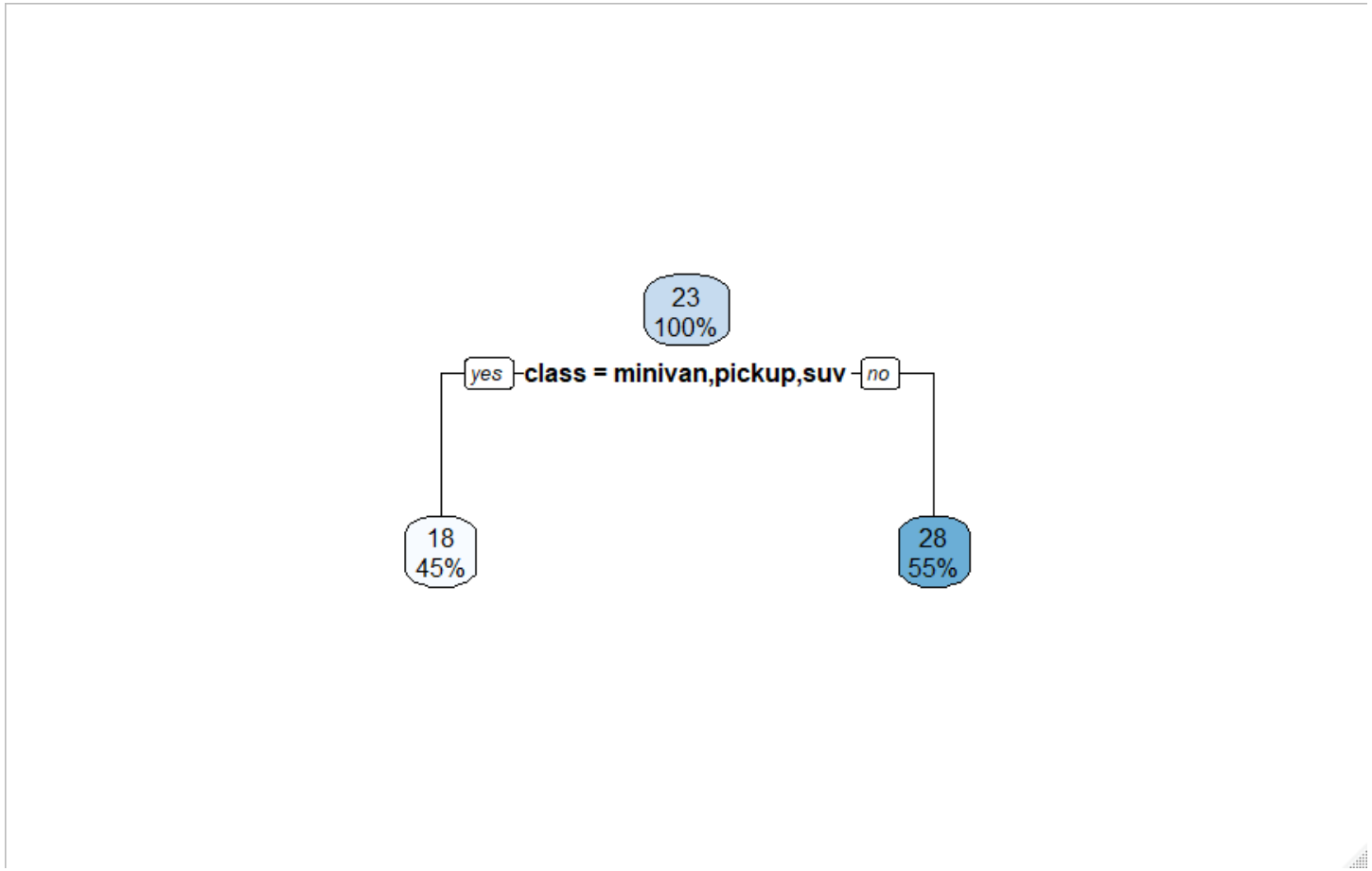
Let's see how it works again, but this time on the `mpg` data set



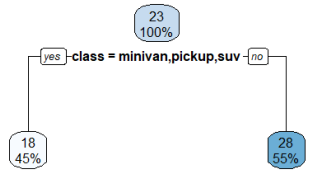
hwy ~ displ + class + drv



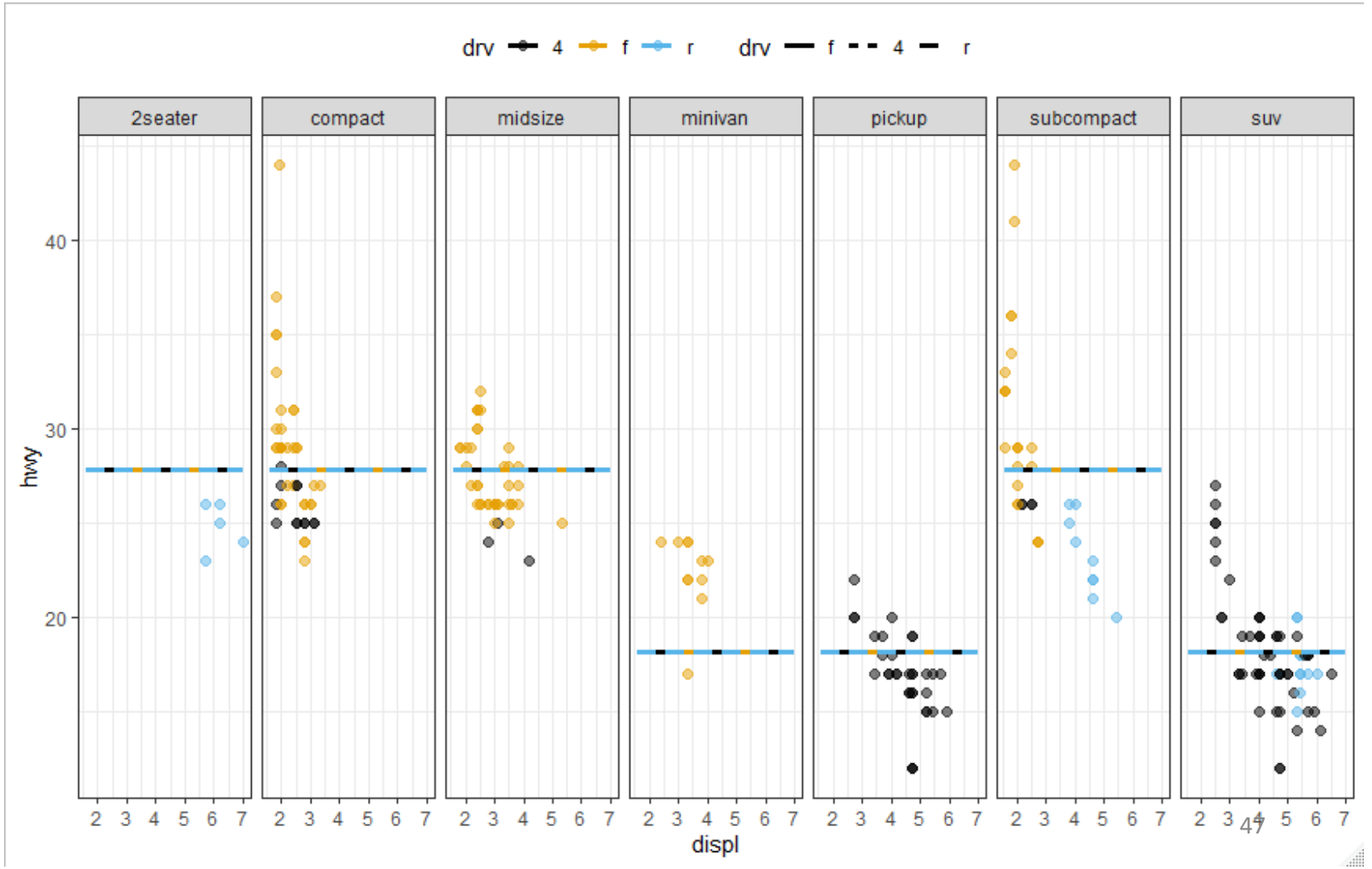
Build a simple decision tree with 1 split or 2 terminal nodes – a **stump**



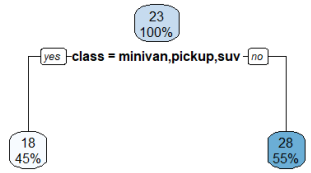
With this simple model `hwy` changes only based on `class`



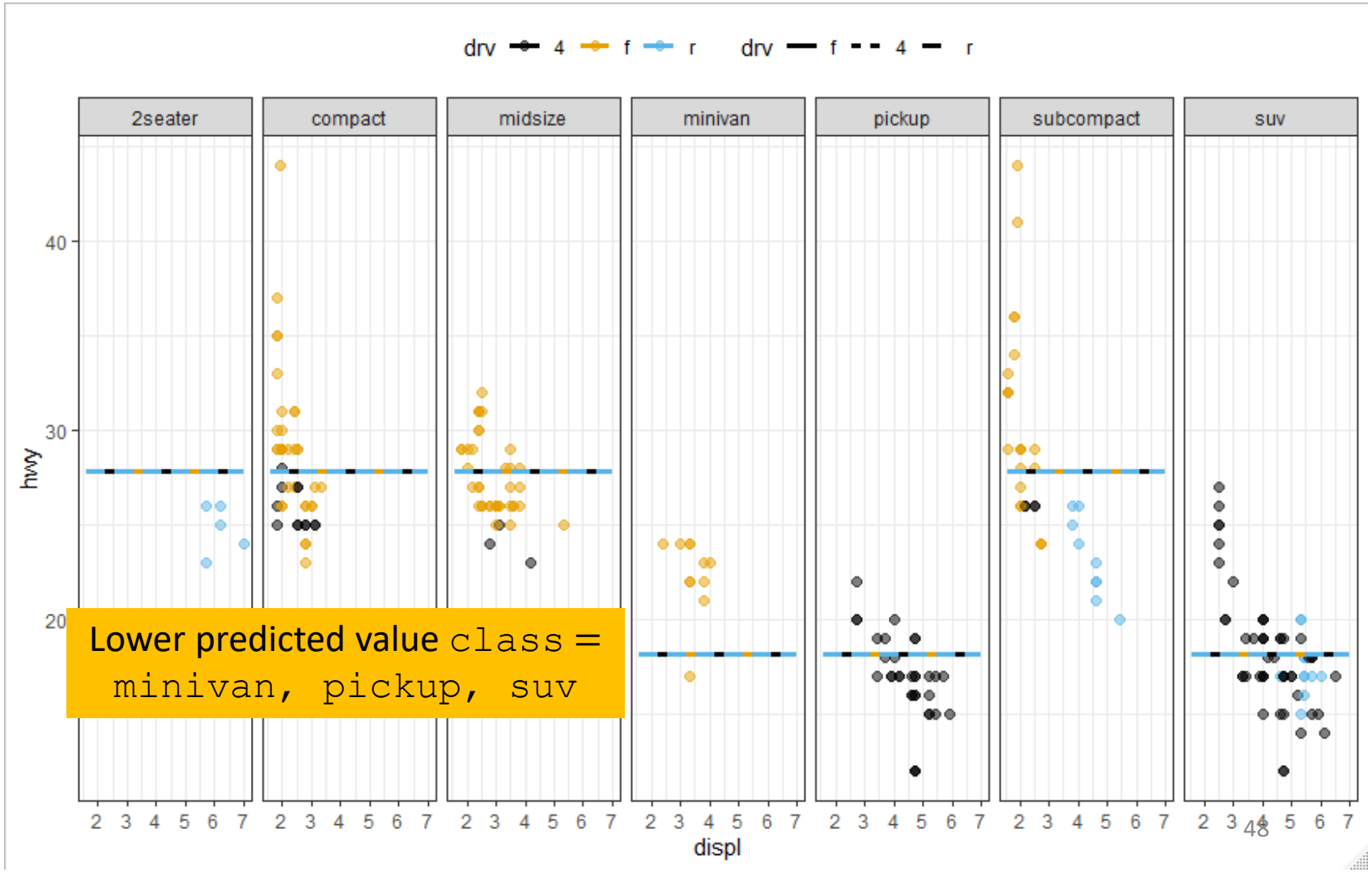
Just 2 predicted output values because there are only 2 terminal nodes (regions)



With this simple model `hwy` changes only based on `class`

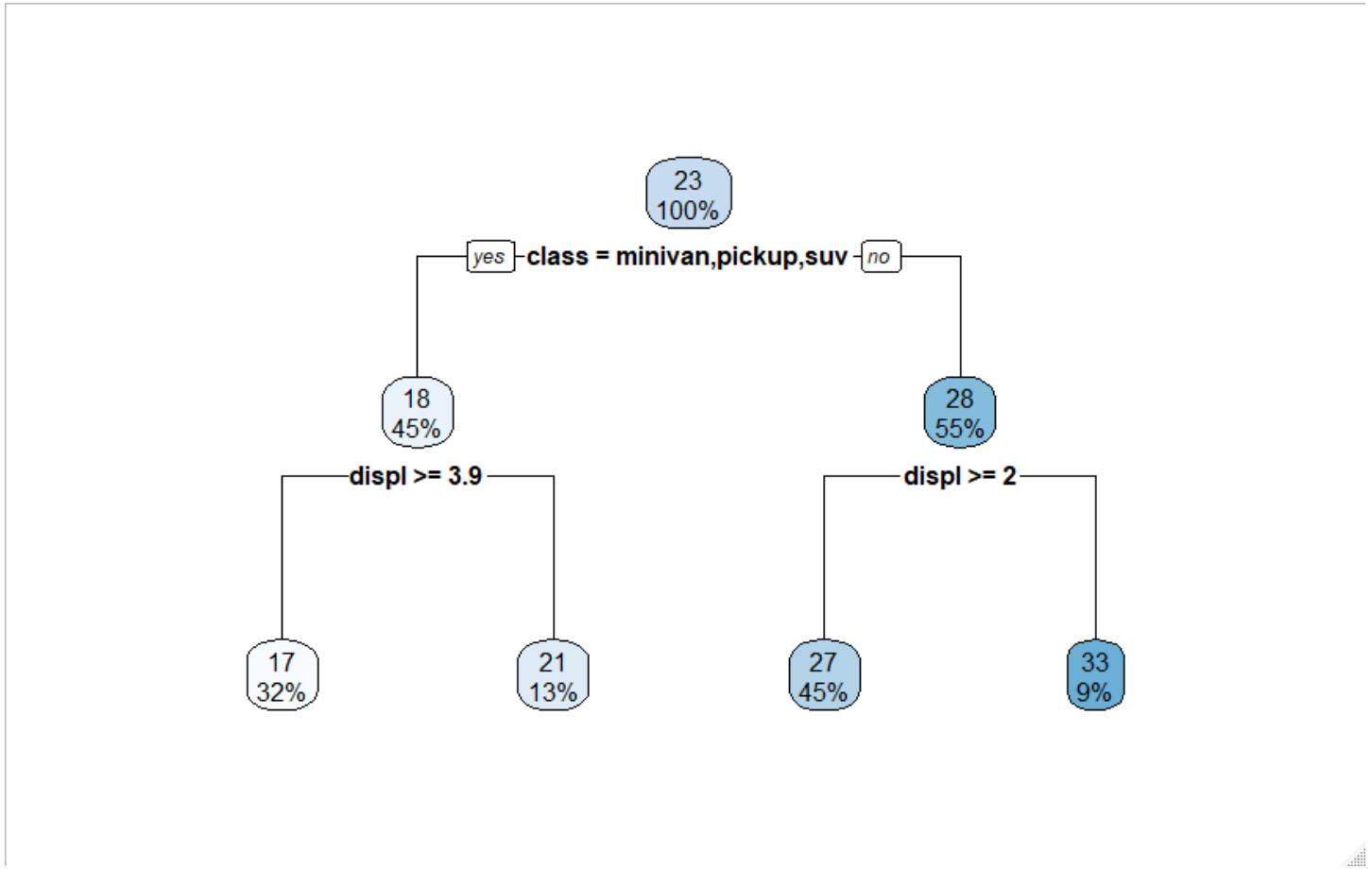


Higher predicted value `class` ≠ `minivan, pickup, suv`



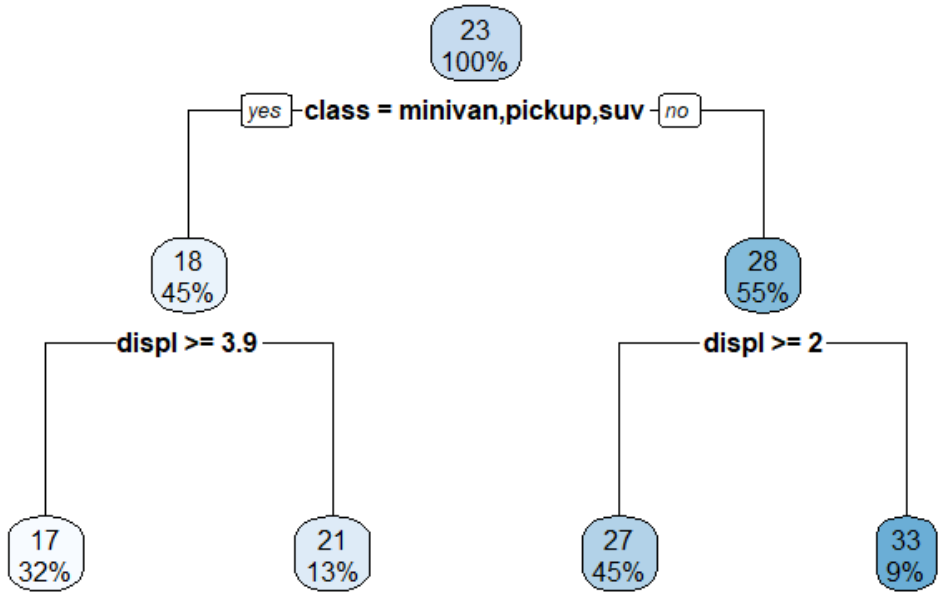
Lower predicted value `class` = `minivan, pickup, suv`

Grow the tree to 4 terminal nodes



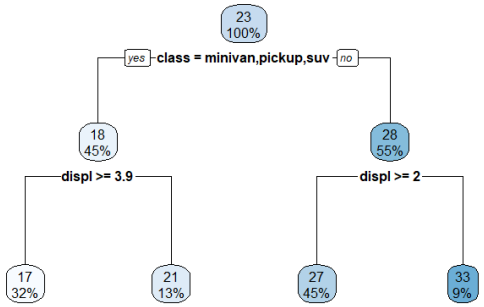
Grow the tree to 4 terminal nodes

Regions are split based on the value of `displ`

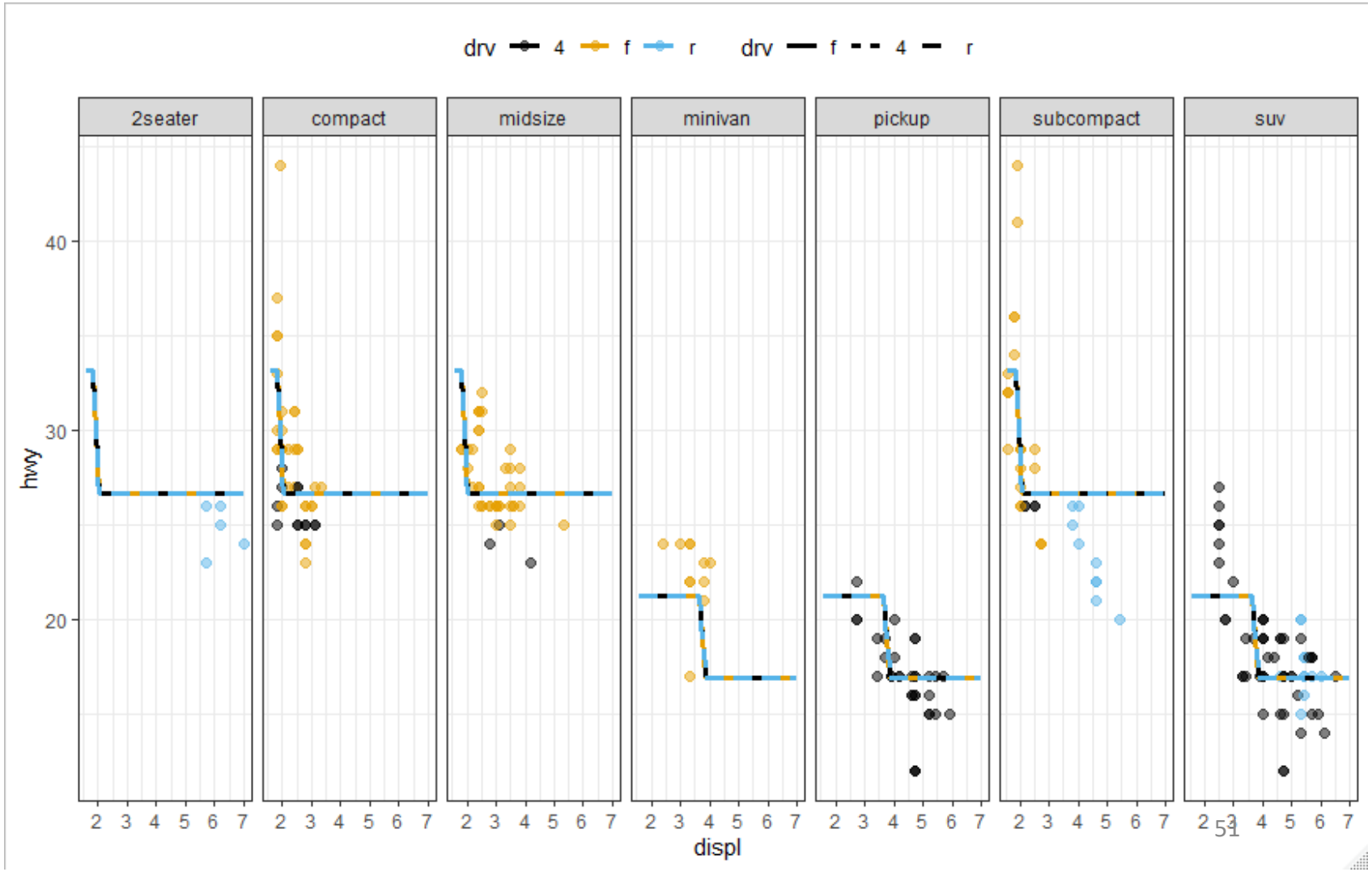


`displ` interacts with `class`!

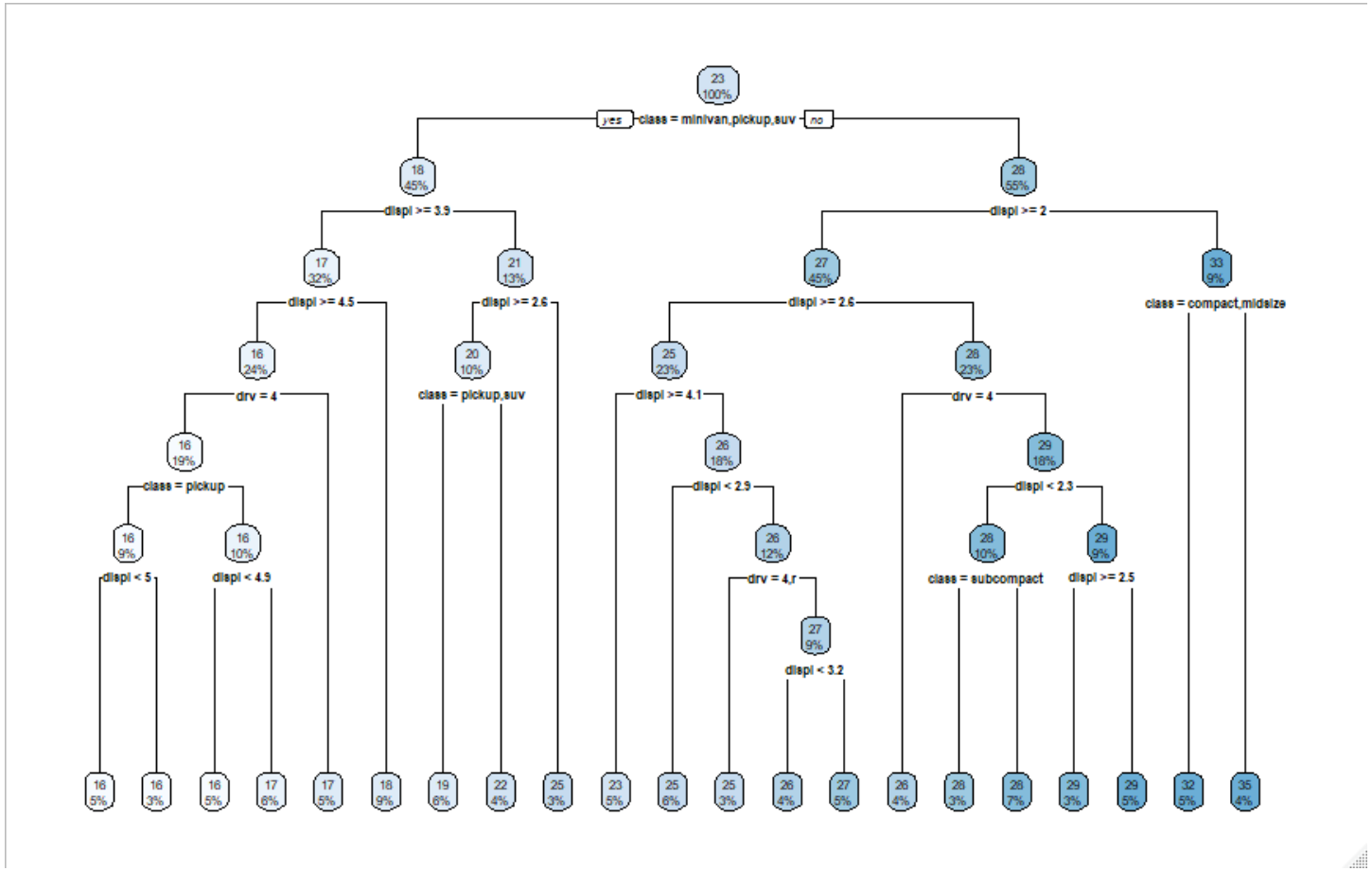
Because `displ` interacts with class, `hwy` changes with `displ` based on the `class`



Notice the `displ` split point (value) depends on the `class`

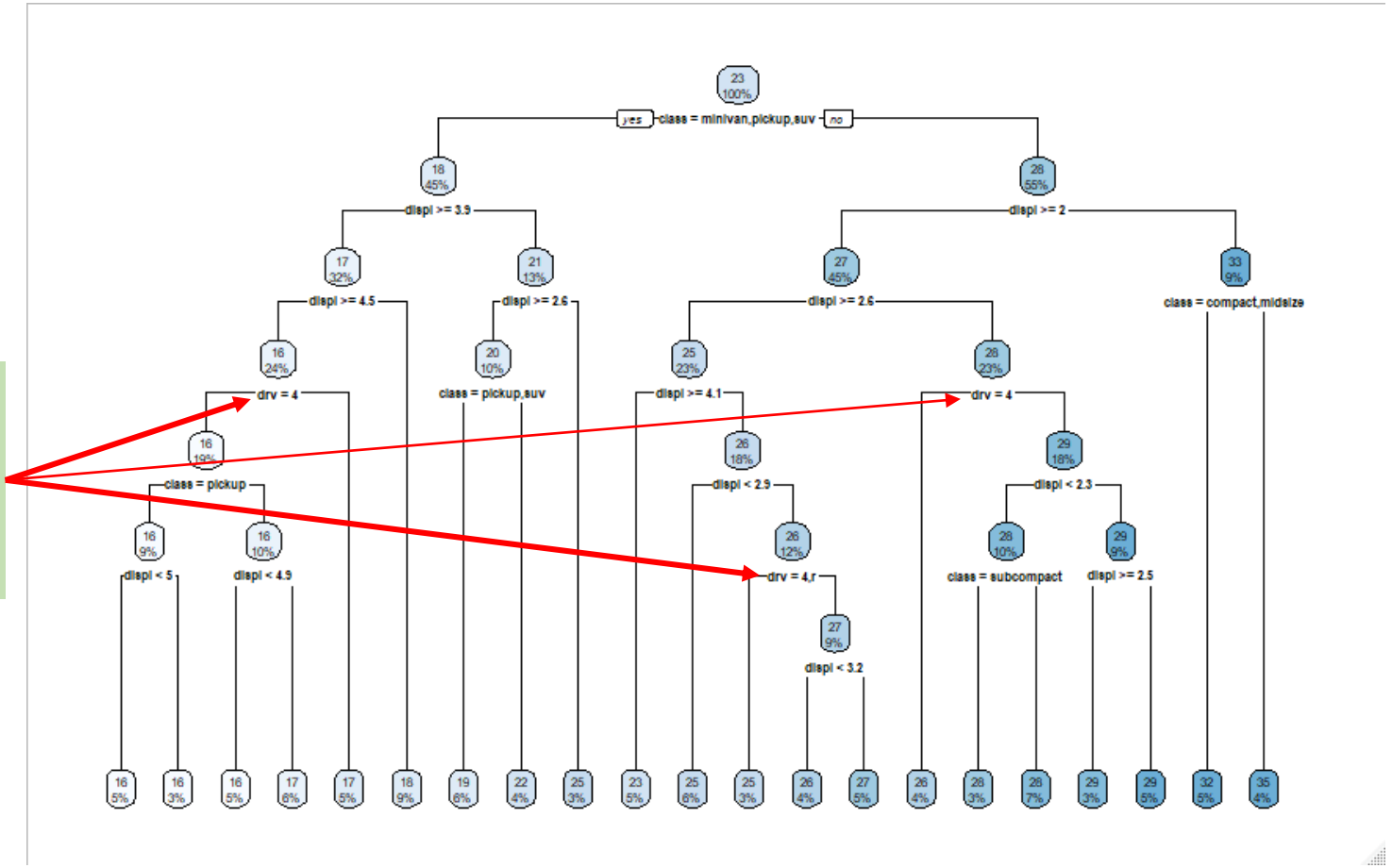


Next, fit a complex or deep tree

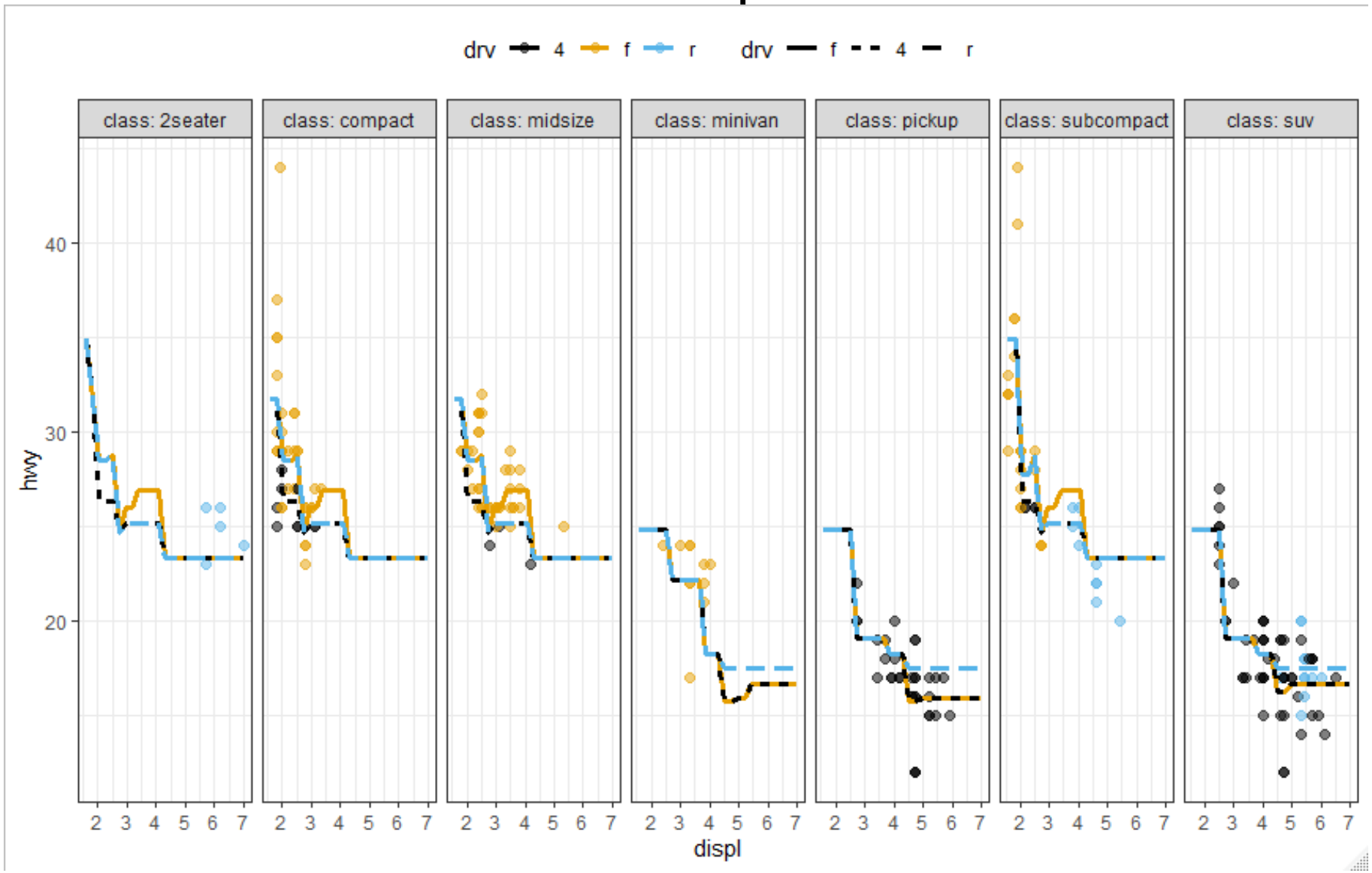


Next, fit a complex or deep tree

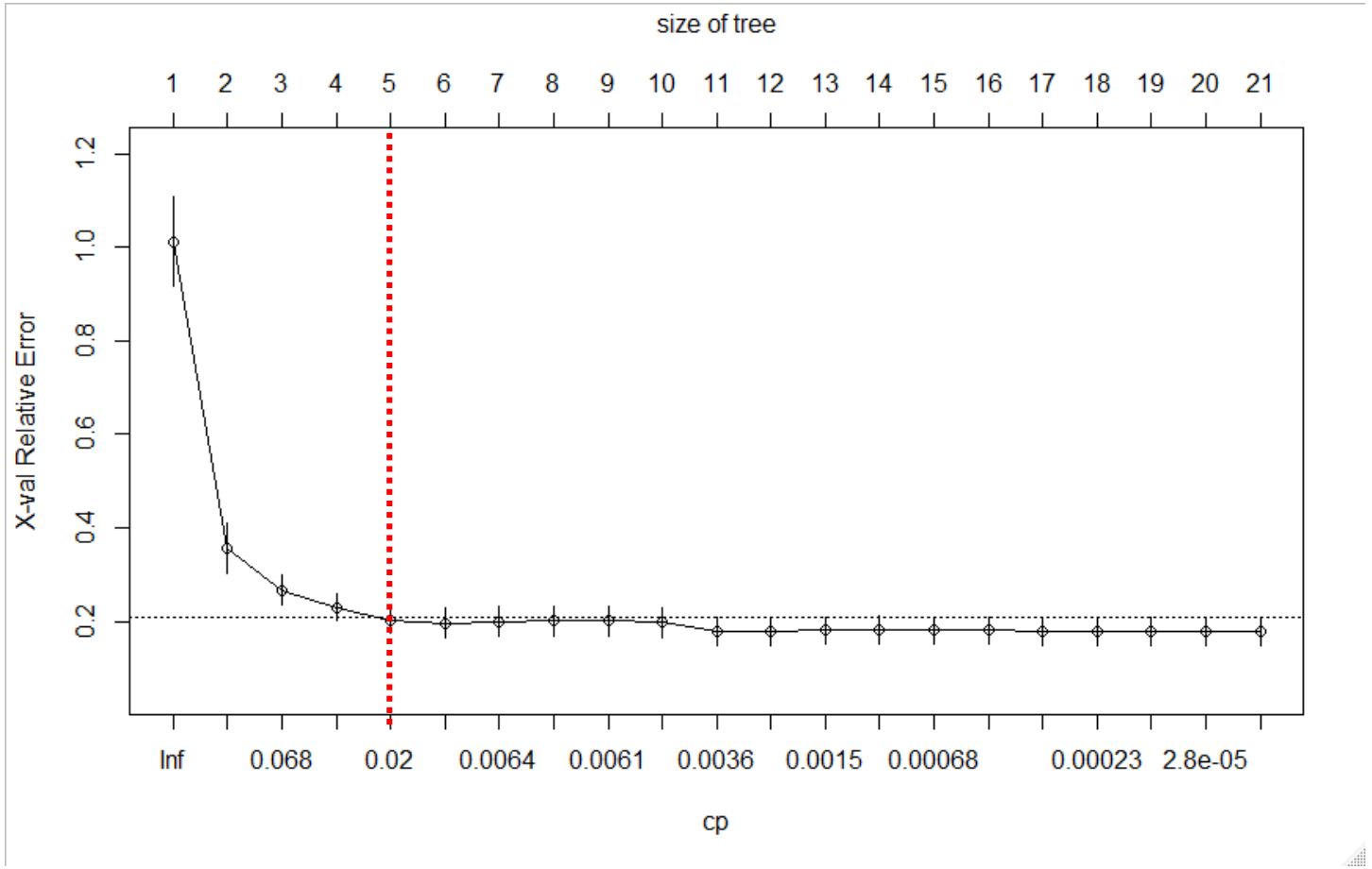
`drv` interacts with the other variables



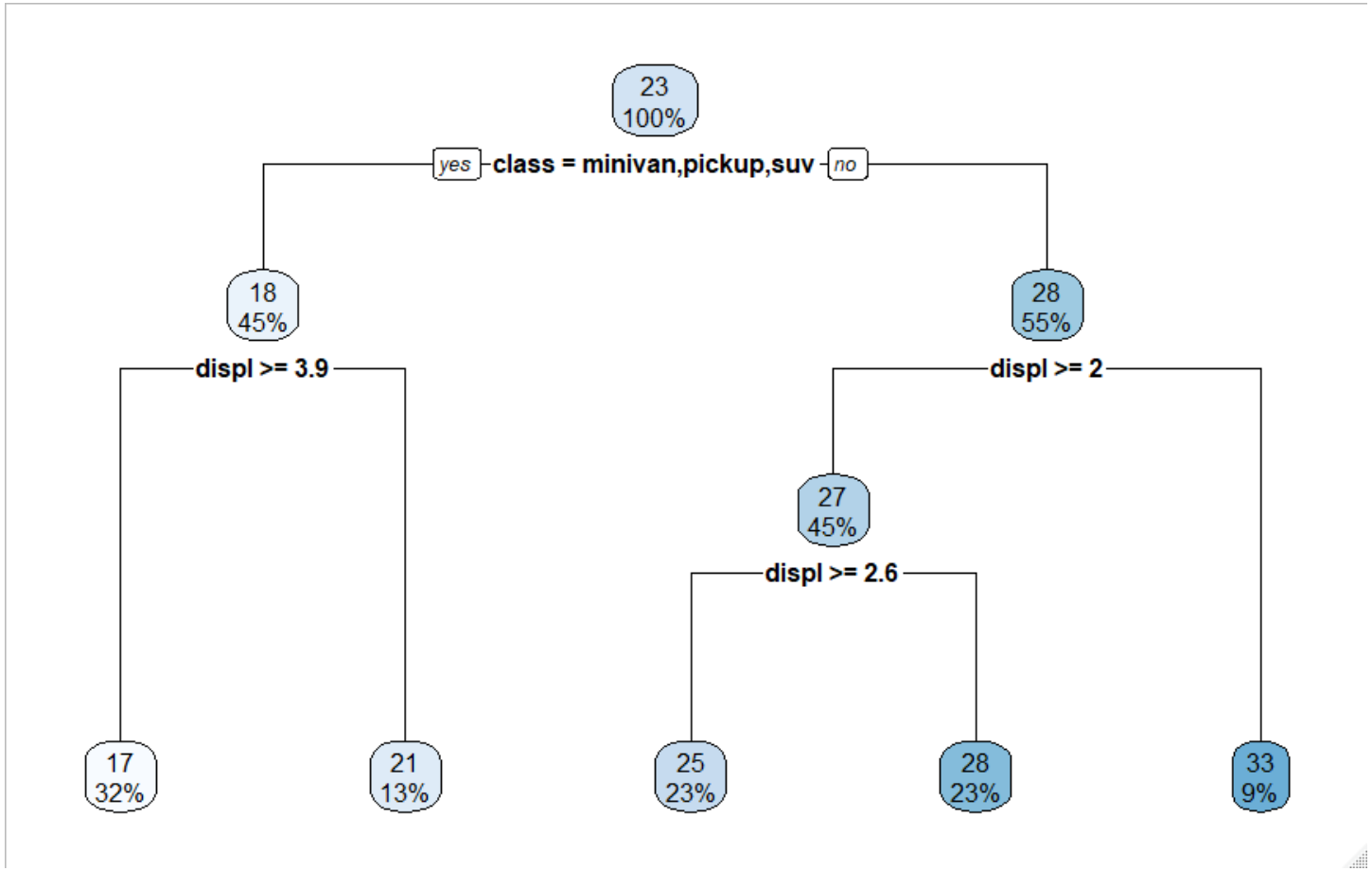
Predictions with the deep tree



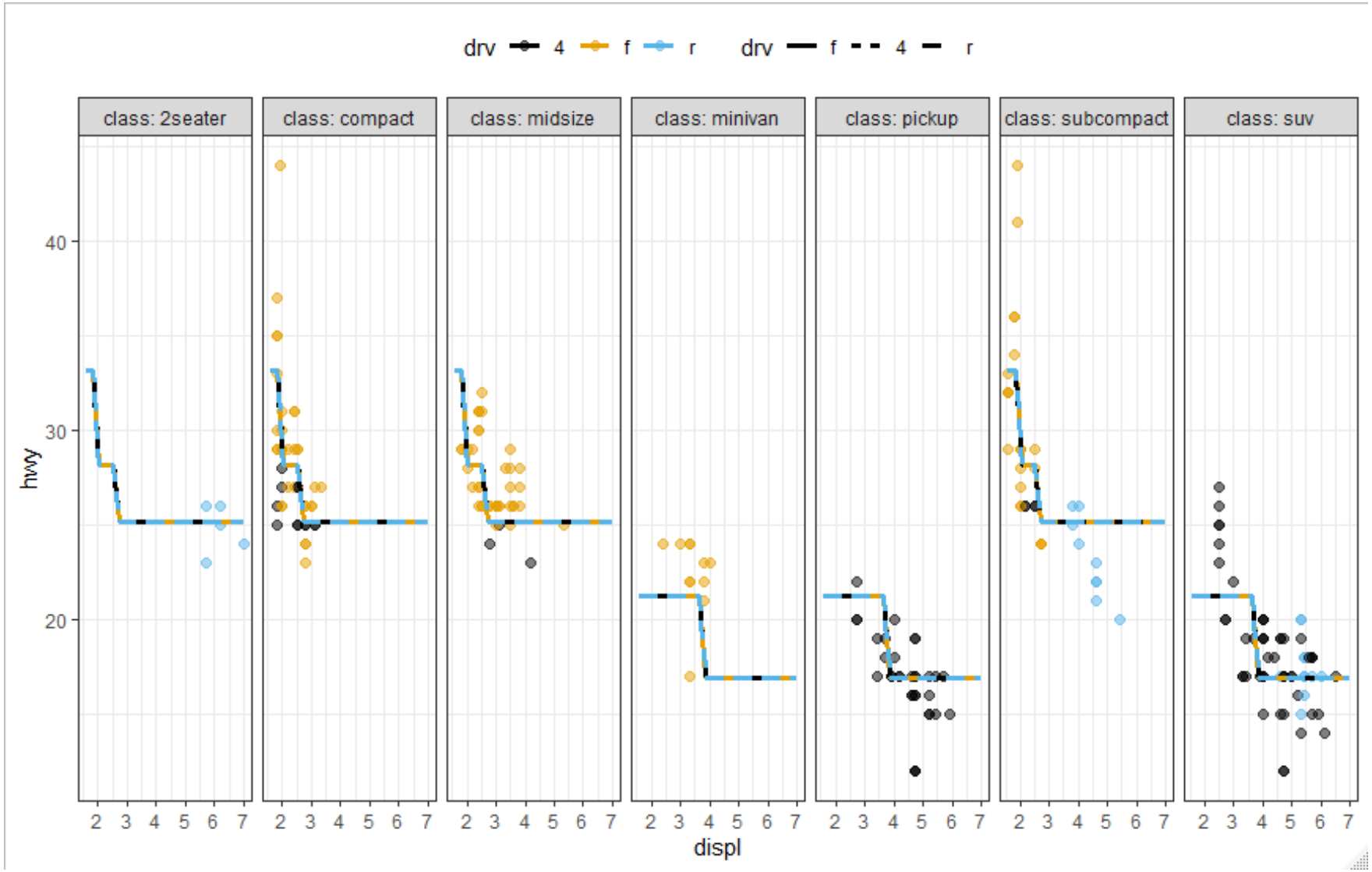
We “prune” the complex tree to hopefully guard against overfitting



The optimally pruned tree



Predictions from the optimally pruned tree



Classification trees

- The overall structure and concepts are similar to regression trees.
- **EXCEPT in the criteria for splitting and pruning!**
- We cannot use the SSE loss, we need to use a criteria more suitable for classification.

Define the proportion of the l-th class in node m

$$\hat{\mu}_{m,l} = \frac{1}{N_m} \sum_{\mathbf{x}_n \in R_m} I(y_n = l)$$

- N_m is the number of observations at node m.
- The above formula is just counting!

Define the proportion of the l-th class in node m

$$\hat{\mu}_{m,l} = \frac{1}{N_m} \sum_{\mathbf{x}_n \in R_m} I(y_n = l)$$

Indicator function: if $\mathbf{x}_n$ is within region R_m and $y_n = l$ return 1 otherwise return 0

- N_m is the number of observations at node m.
- The above formula is just counting!

Define the proportion of the l-th class in node m

$$\hat{\mu}_{m,l} = \frac{1}{N_m} \sum_{\mathbf{x}_n \in R_m} I(y_n = l)$$

- Classify the observations in node m based on the majority class within the node.

$$l(m) = \arg\max_{l} \hat{\mu}_{m,l}$$

Three different criteria for classification trees

Misclassification error

$$\frac{1}{N_m}\sum_{n\in R_M}\Big(I\big(y_n \neq l(m)\big)\Big) = 1 - \hat{\mu}_{m,l(m)}$$

Gini index

$$\sum_{l\neq l'}\big(\hat{\mu}_{m,l}\hat{\mu}_{m,l'}\big) = \sum_{l=1}^{L}\Big(\hat{\mu}_{m,l}\big(1-\hat{\mu}_{m,l}\big)\Big)$$

Cross-entropy

$$\sum_{l=1}^{L}\big(\hat{\mu}_{m,l}\log\big[\hat{\mu}_{m,l}\big]\big)$$

Three different criteria for classification trees

Misclassification error

$$\frac{1}{N_m}\sum_{n\in R_M}\left(I\left(y_n \neq l(m)\right)\right) = 1 - \hat{\mu}_{m,l(m)}$$

Typically used for pruning

Gini index

$$\sum_{l\neq l'}\left(\hat{\mu}_{m,l}\hat{\mu}_{m,l'}\right) = \sum_{l=1}^{L}\left(\hat{\mu}_{m,l}\left(1-\hat{\mu}_{m,l}\right)\right)$$

Cross-entropy

$$\sum_{l=1}^{L}\left(\hat{\mu}_{m,l}\log\left[\hat{\mu}_{m,l}\right]\right)$$

How can we extend or improve the decision tree?

- Even with pruning, decision trees have **high variance**.
- The deeper (less pruned) a tree is the more **sensitive** the model is to the training set.
- Can we **exploit** that behavior?

We previously discussed cross-validation as a technique for tuning and comparing models

- We generated separate random data splits of training and hold-out test sets.

- Provided a way to estimate the generalization error via the cross-validation hold-out set error.

We will now use another resampling technique: the Bootstrap.

- We will resample the dataset **WITH REPLACEMENT**.

- Let's visualize how this works with the small `mtcars` data set.

- The `mtcars` consists of just 32 rows (observations).

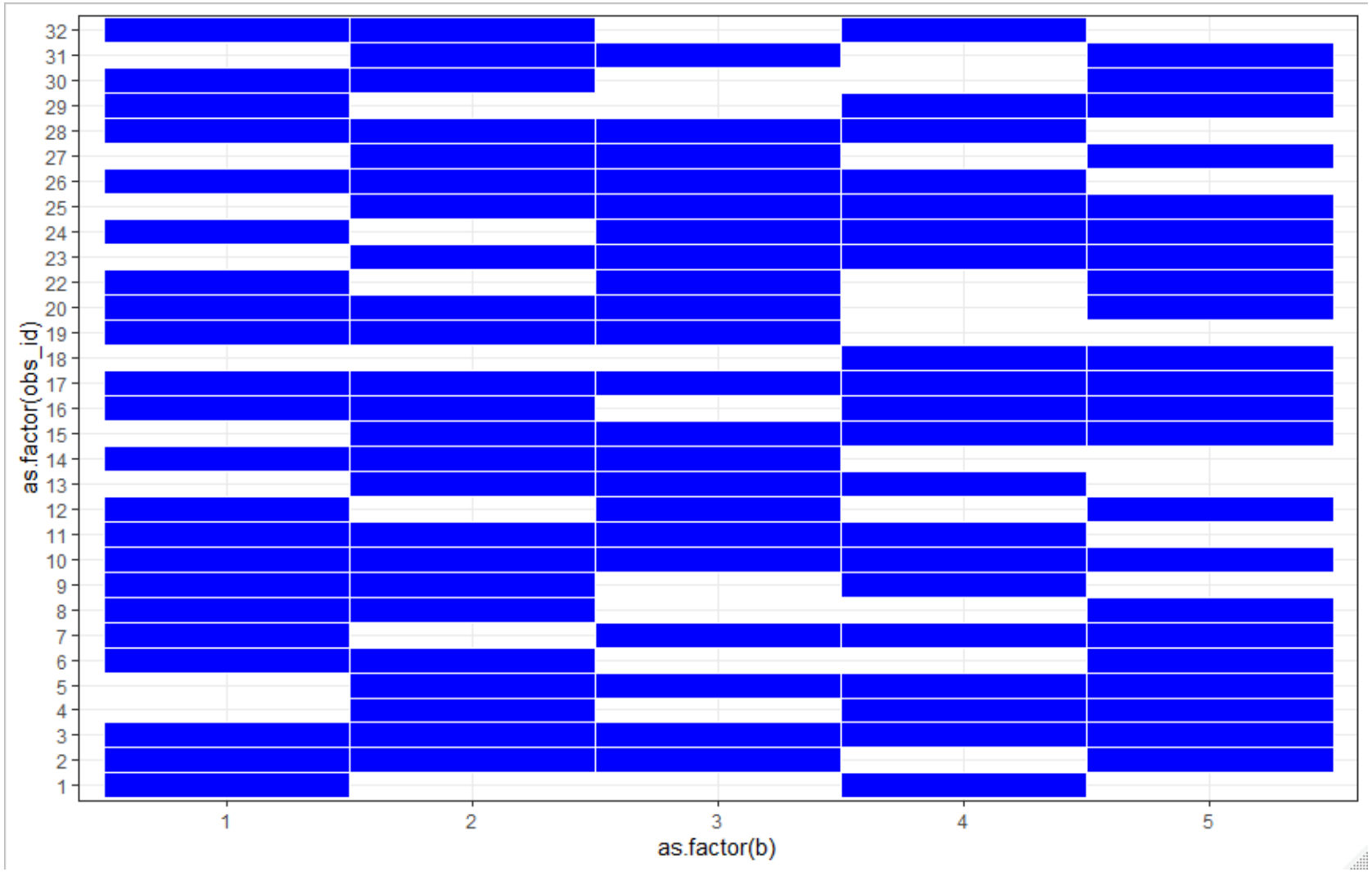
Generate 5 bootstrap samples

`obs_id` corresponds to a particular row in the data set.

`b` corresponds to a bootstrap id.

A blue rectangle represents the observation (row) is within a particular bootstrap sample.

A white space therefore means the row is **NOT** within a particular bootstrap sample.



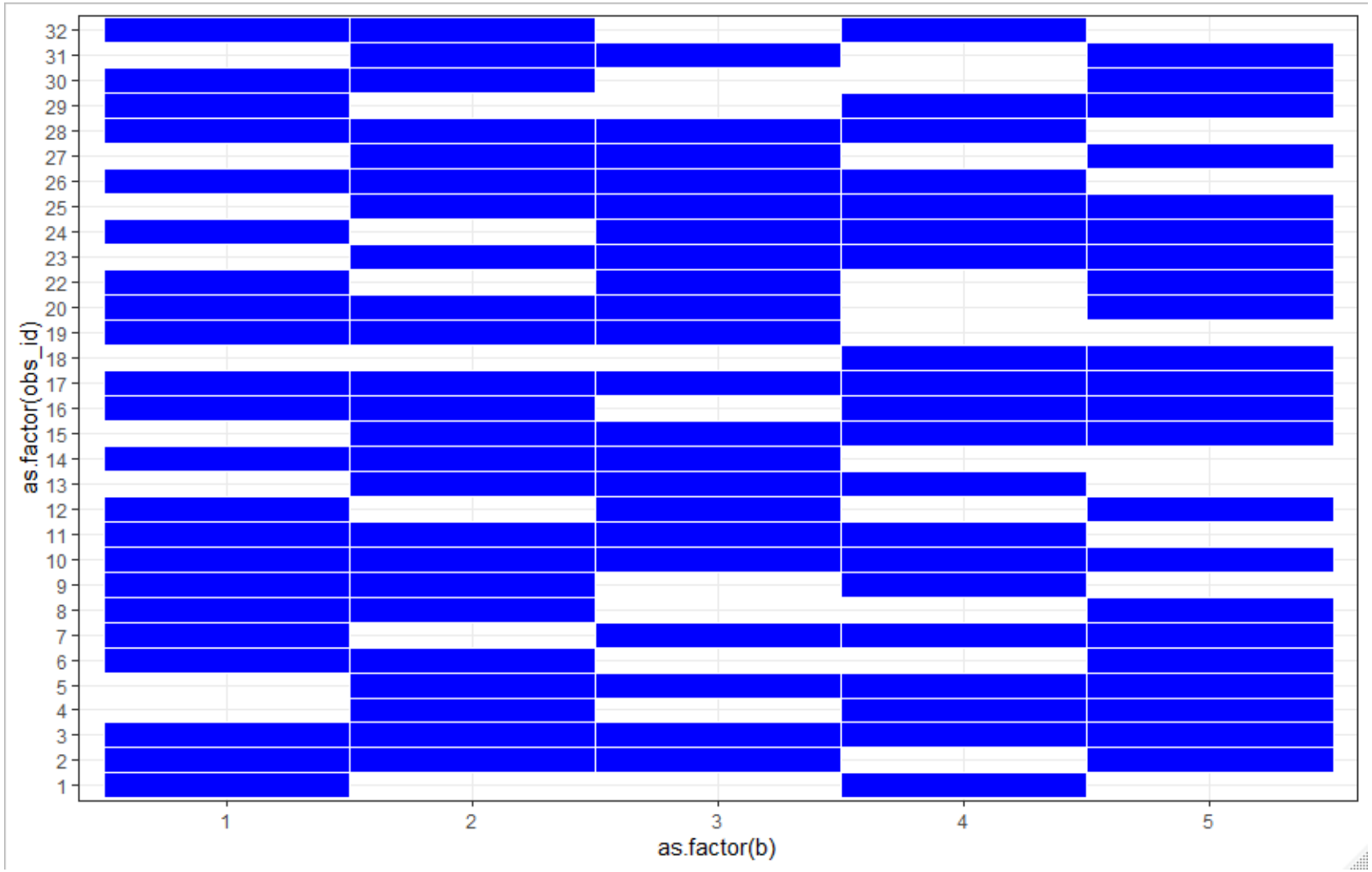
How does this look compared to cross-validation?

`obs_id` corresponds to a particular row in the data set.

`b` corresponds to a bootstrap id.

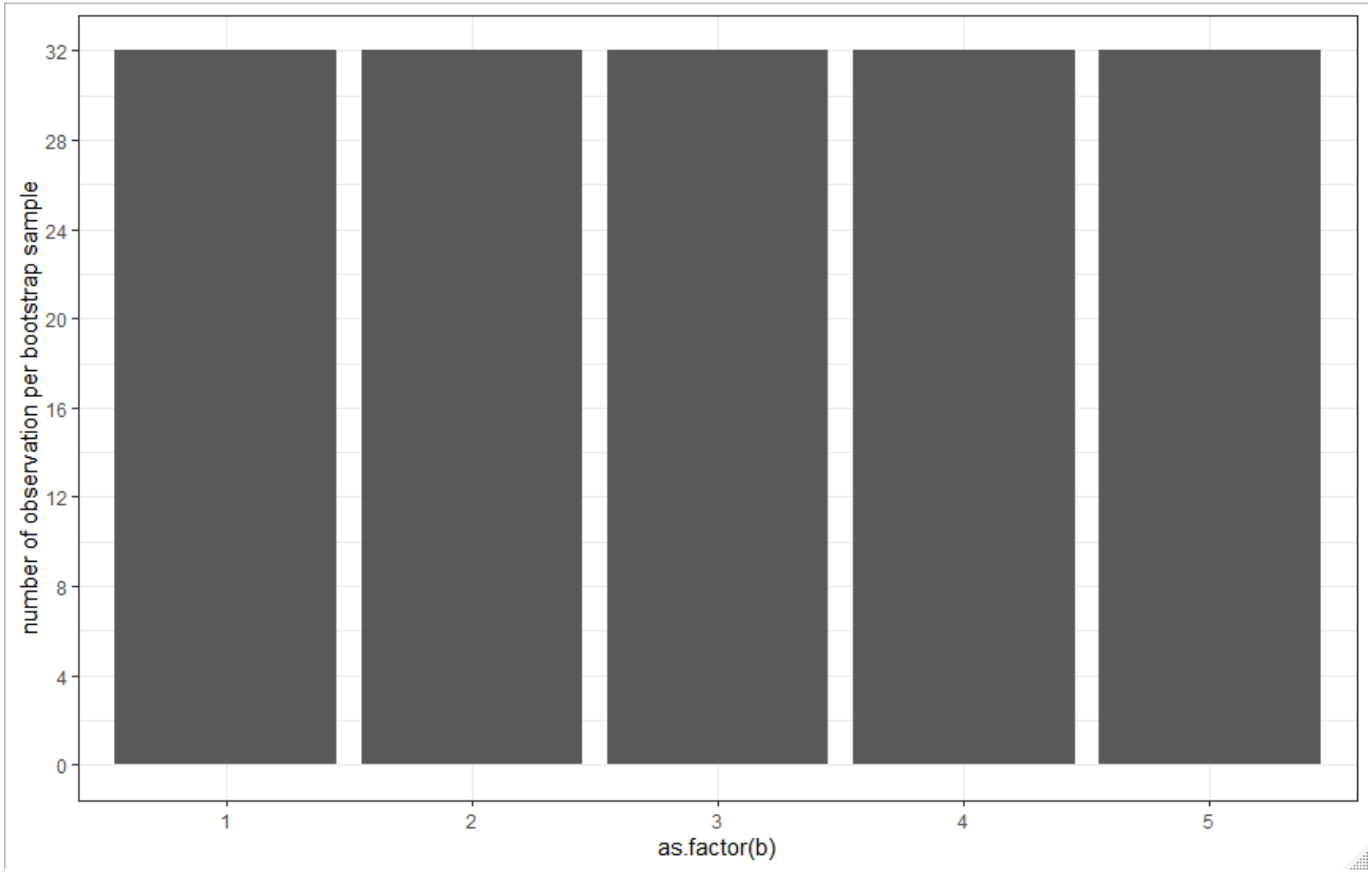
A blue rectangle represents the observation (row) is within a particular bootstrap sample.

A white space therefore means the row is **NOT** within a particular bootstrap sample.

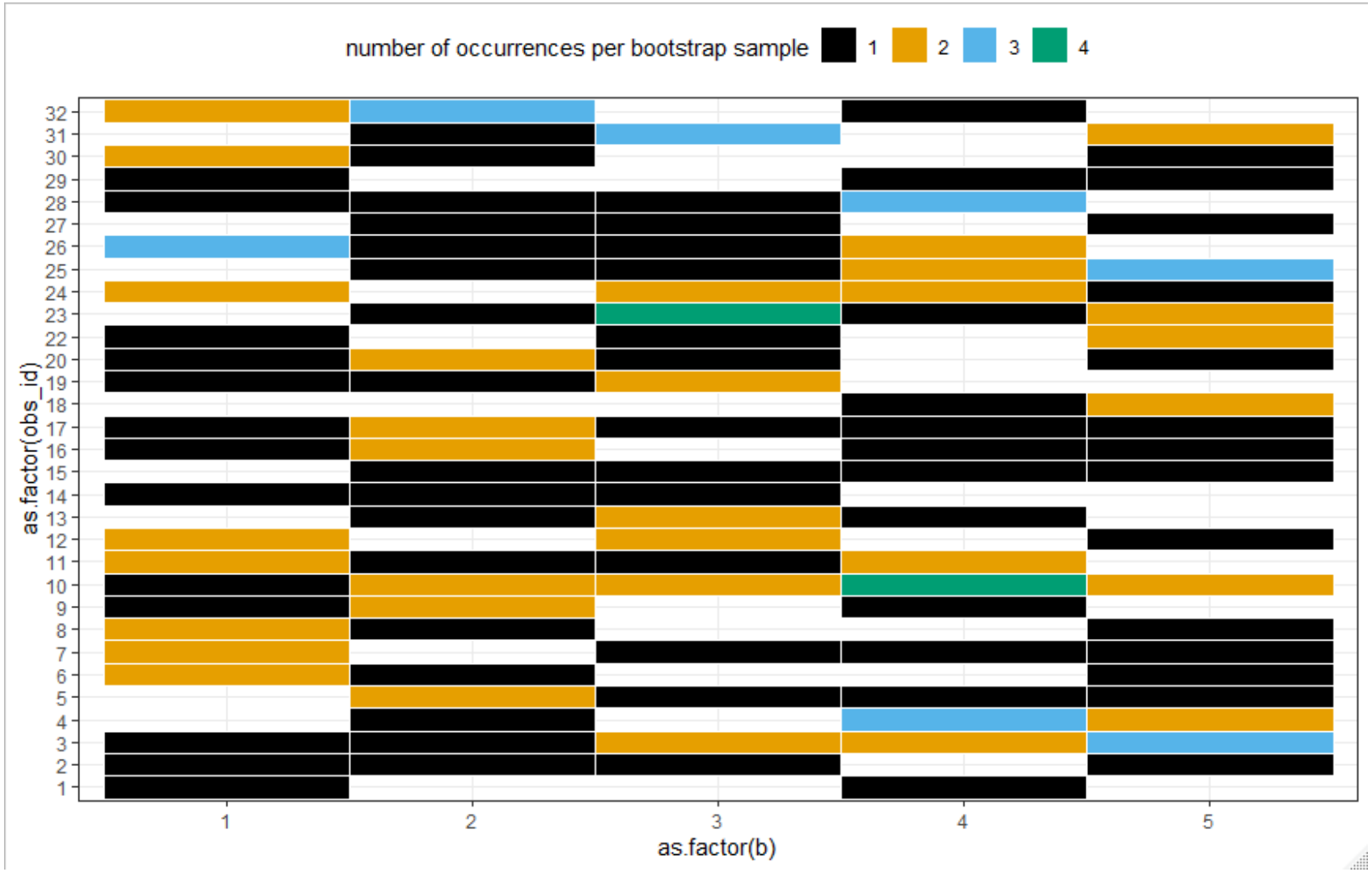


Are the number of observations different in each bootstrapped sample?

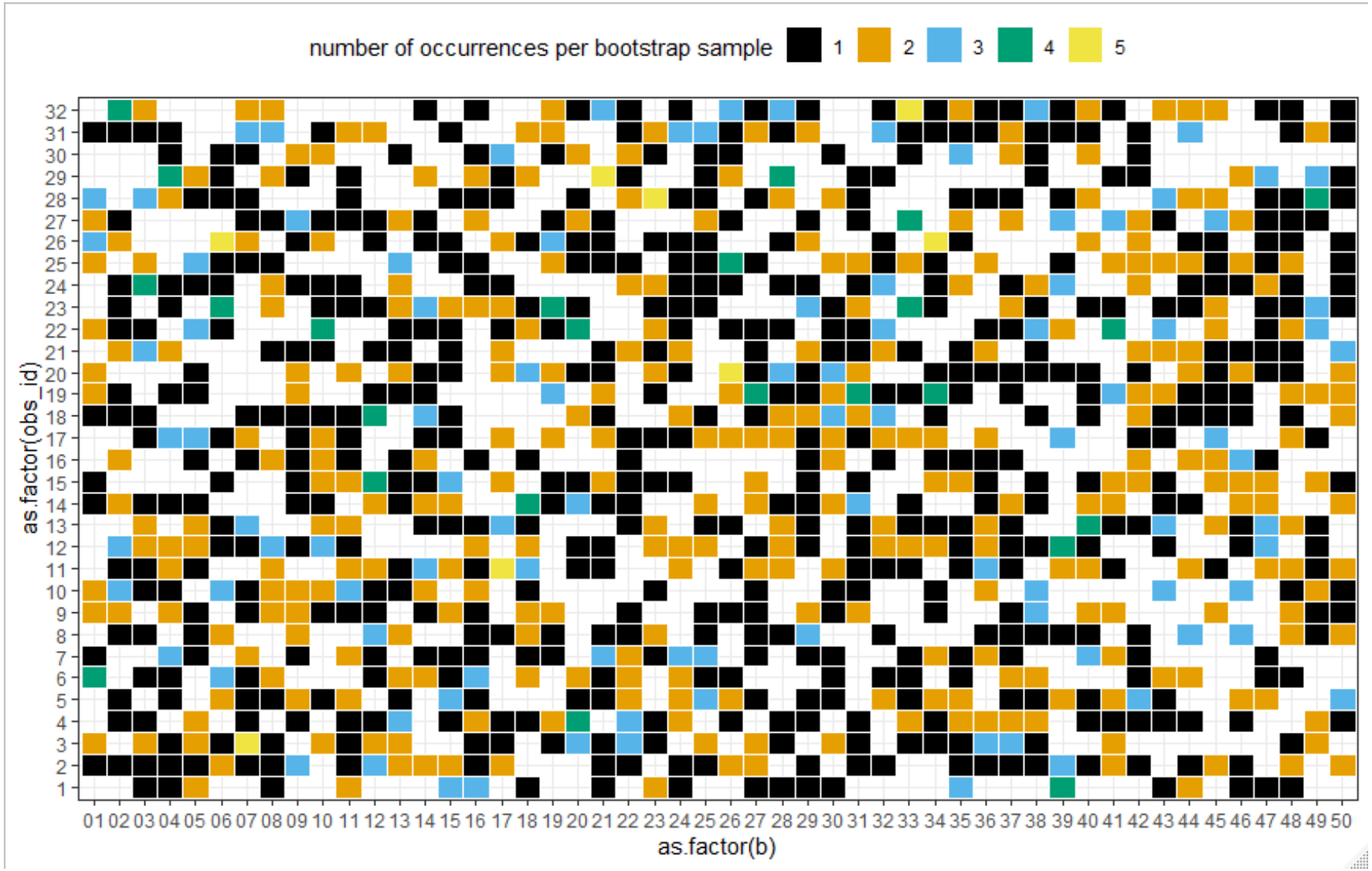
No! All bootstrapped samples have the same number of samples…



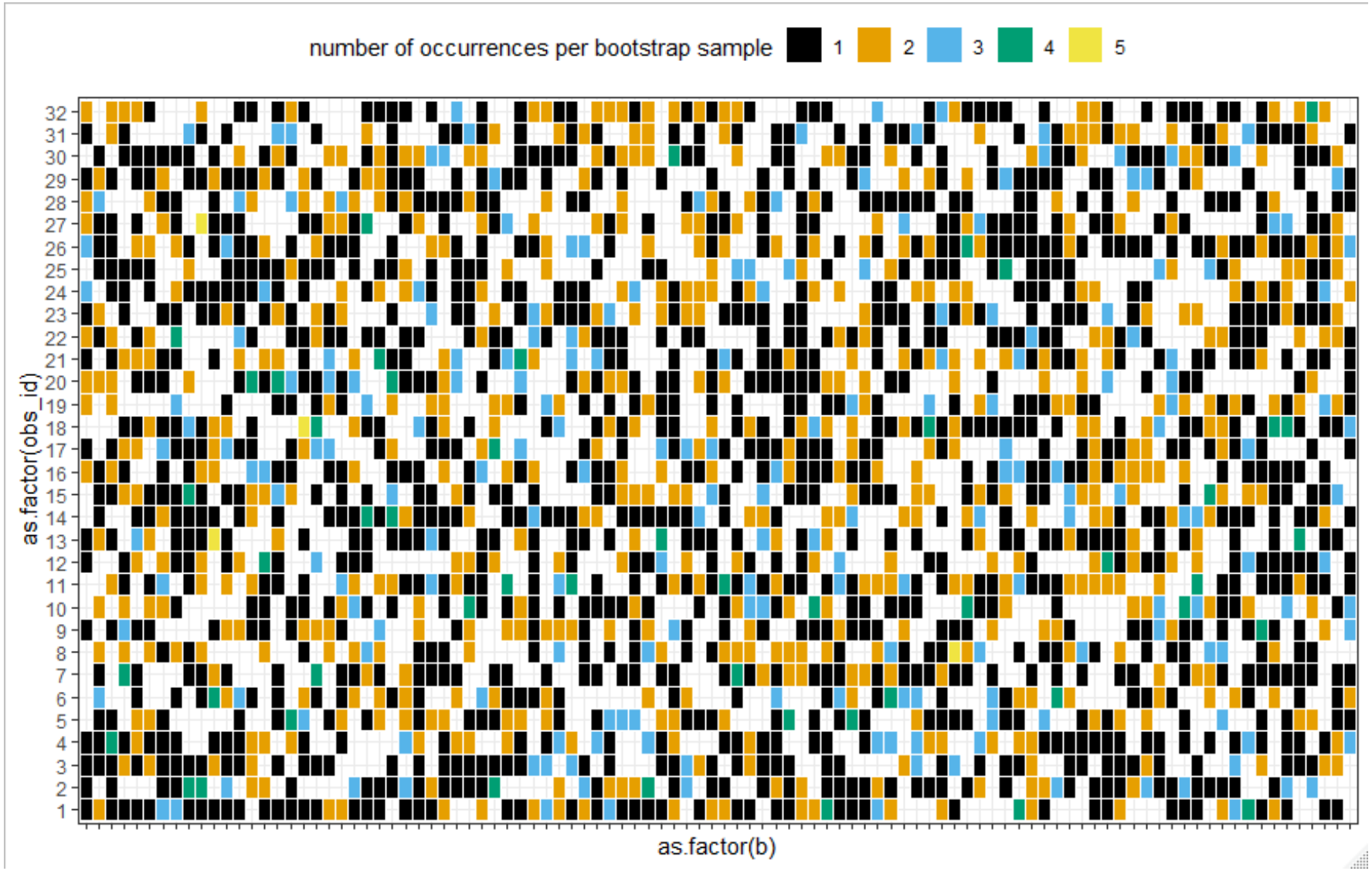
By sampling with replacement, an observation can be selected multiple times!



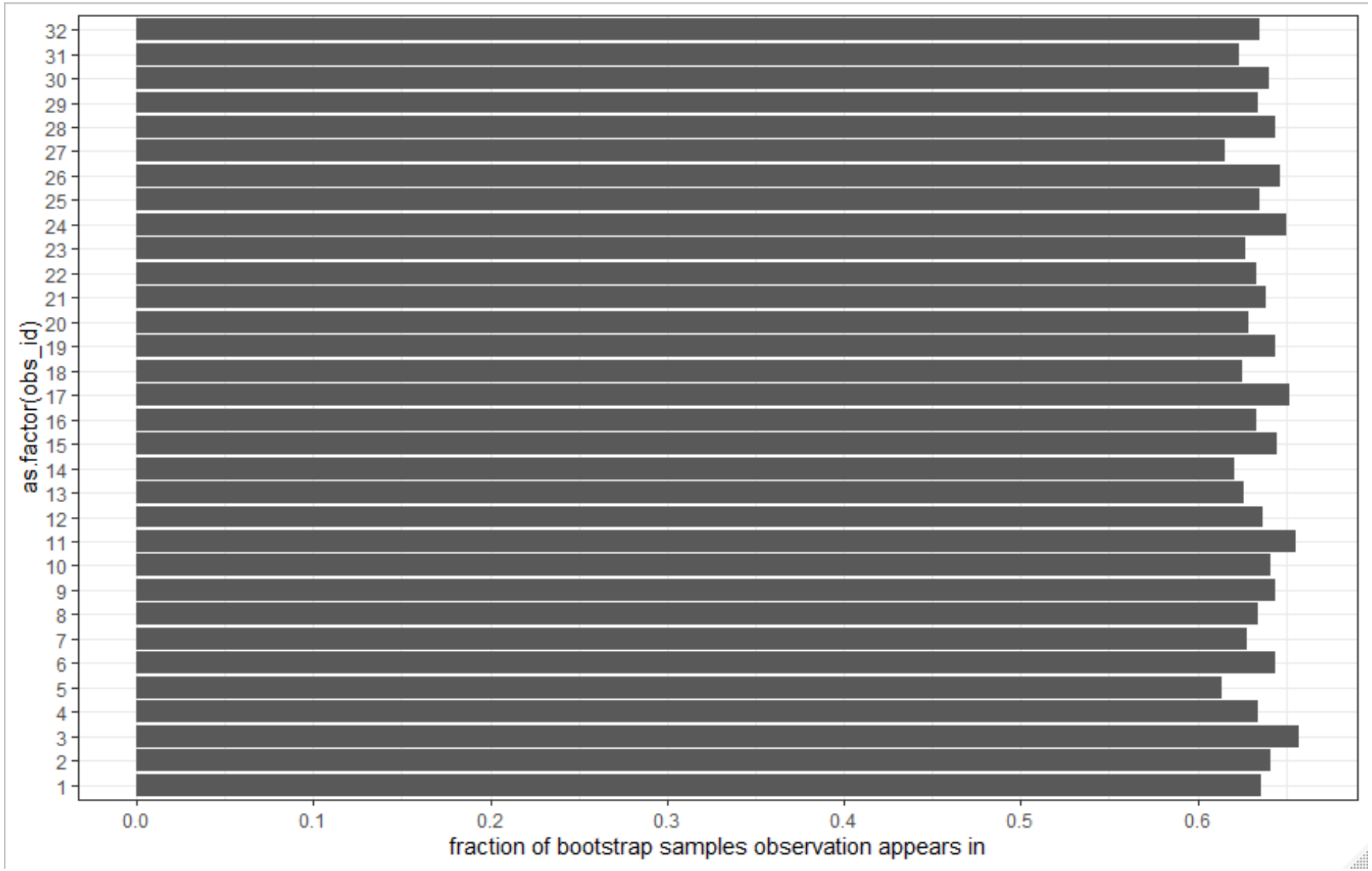
Increase the number of bootstrap samples



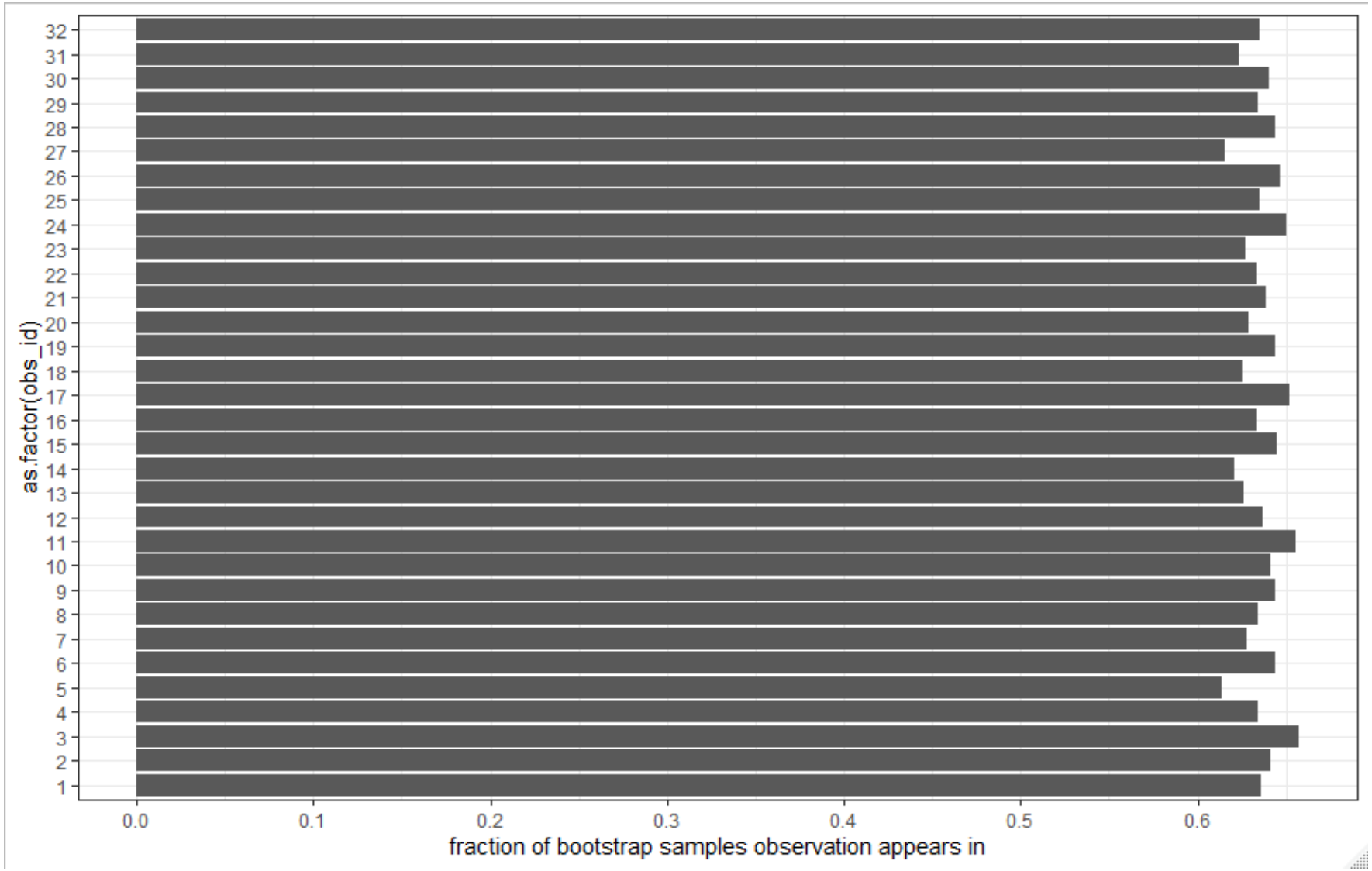
Increase to 100 bootstrap samples



How often is each observation selected?



How often is each observation selected?



We will return to this later…

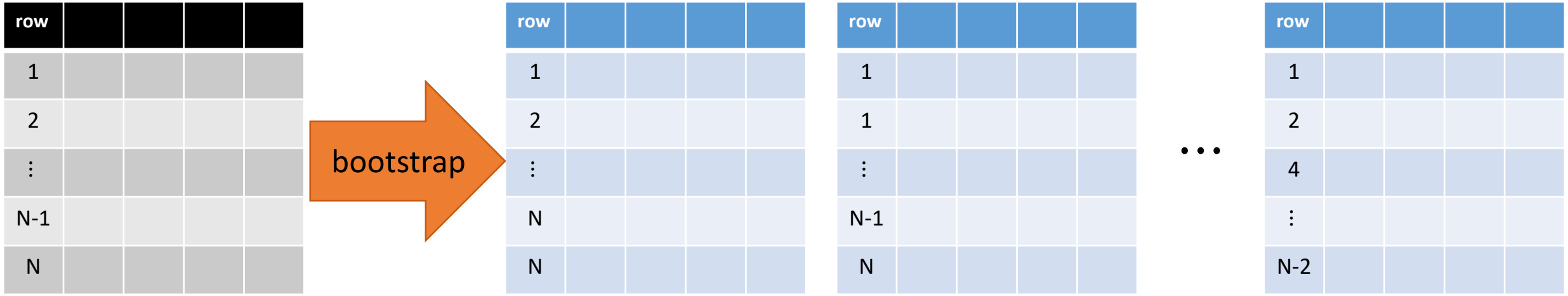
Bootstrapping is a general-purpose tool

- Bootstrapping is very useful when it's challenging to estimate uncertainty in non-Bayesian models.

- Can be used to estimate standard errors and confidence intervals on complex parameters and quantities.

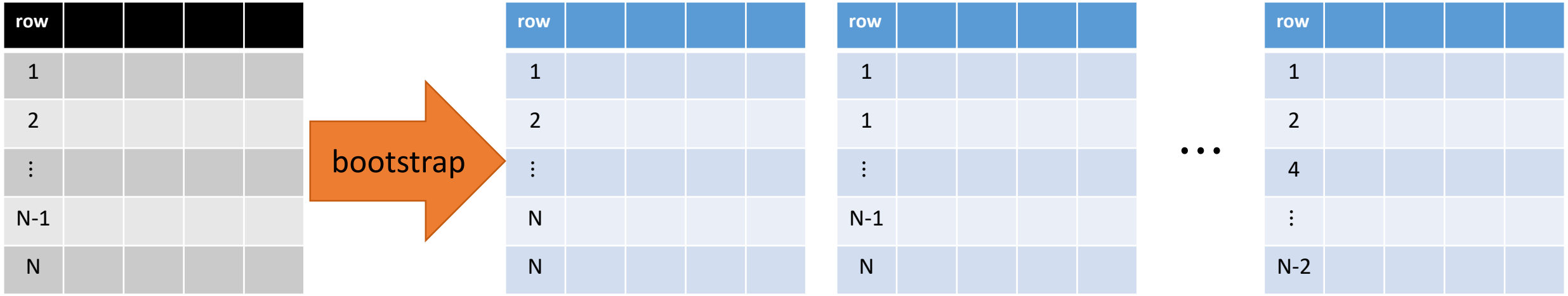
In the context of modeling and prediction, we wish to generate bootstrapped samples of the training data.

For example, generate 100 bootstrapped samples of the data set.



In the context of modeling and prediction, we wish to generate bootstrapped samples of the training data.

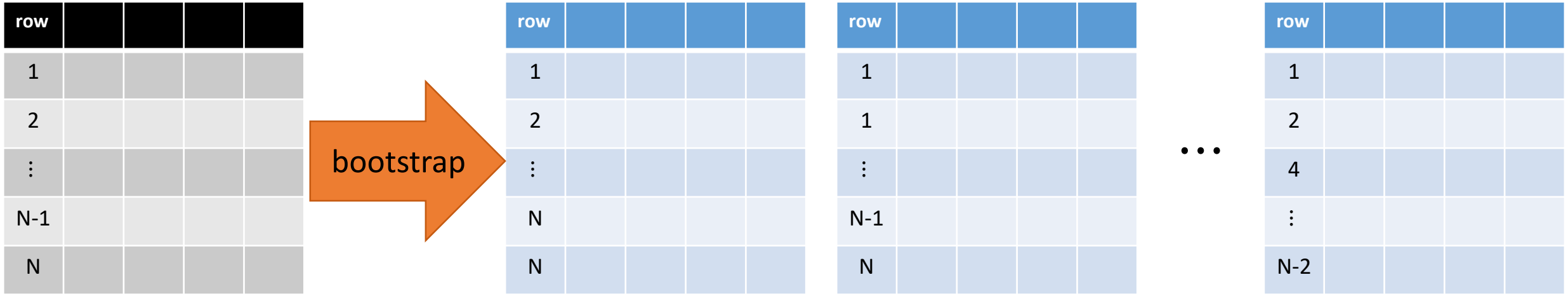
For example, generate 100 bootstrapped samples of the data set.



Bootstrapping effectively generates RANDOM “SYNTHETIC” training sets!

In the context of modeling and prediction, we wish to generate bootstrapped samples of the training data.

For example, generate 100 bootstrapped samples of the data set.



$\hat{f}_1(\mathbf{x})$ $\hat{f}_2(\mathbf{x})$ $\hat{f}_{100}(\mathbf{x})$

Train a separate model for each bootstrapped data set.

What can we do with the 100 models?

Bootstrap Aggregation - BAGGING

- With 100 models, we can make 100 different predictions of a test input, $\mathbf{x}_*$
- The 100 separate predictions are: $\hat{f}_1(\mathbf{x}_*), \hat{f}_2(\mathbf{x}_*), \ldots, \hat{f}_{100}(\mathbf{x}_*)$
- Aggregating all 100 predictions together and AVERAGING them yields the "final" bagged model prediction:

$$\hat{f}_{bag}(\mathbf{x}_*) = \frac{1}{100}\sum_{b=1}^{100} \hat{f}_b(\mathbf{x}_*)$$

Bootstrap Aggregation - BAGGING

- In general, if we generate B bootstrapped samples of the training data, the bagged model prediction is:

$$\hat{f}_{bag}(\mathbf{x}_*) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b(\mathbf{x}_*)$$

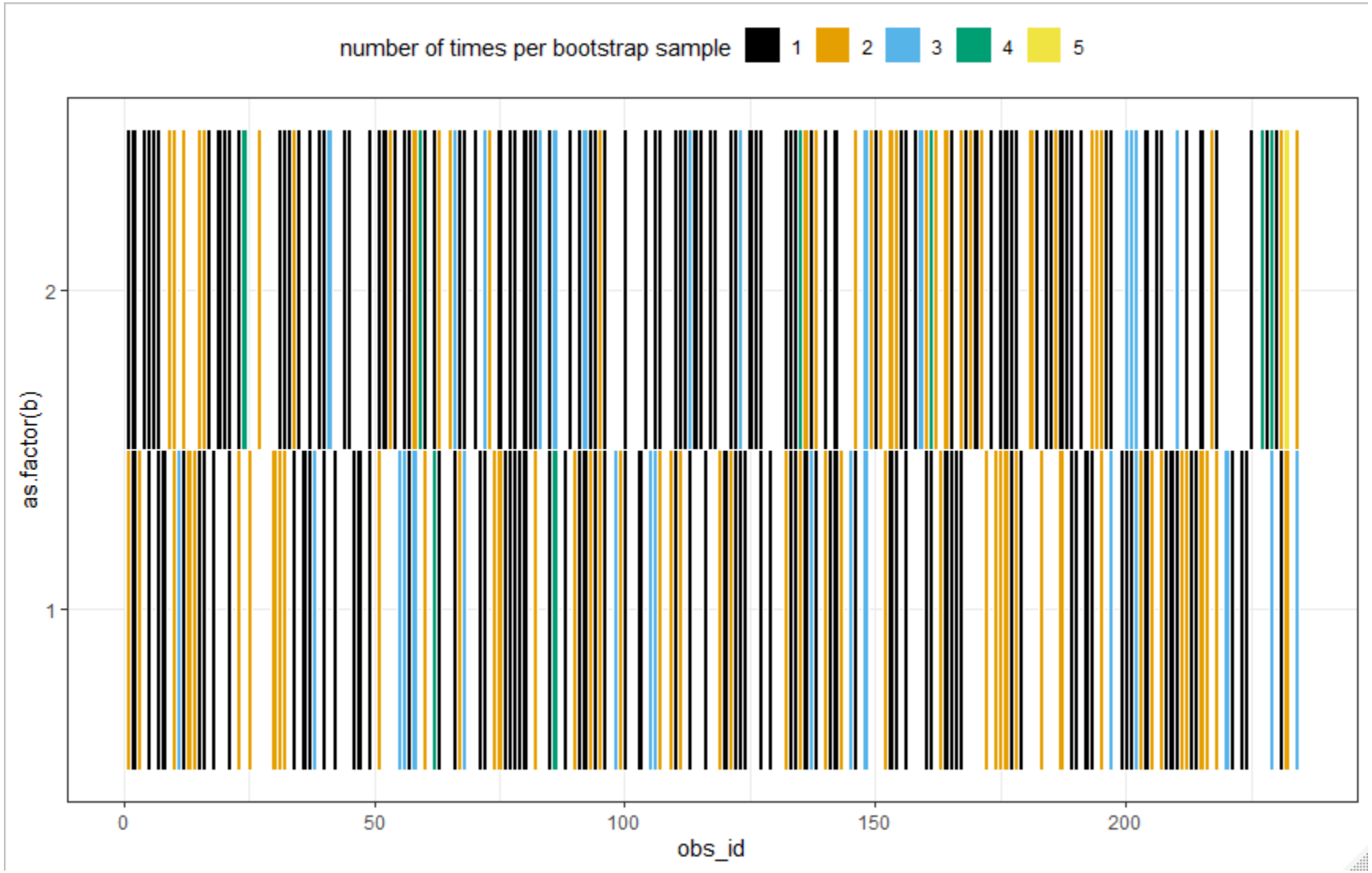
Bagging is general and so can be applied to any modeling framework

- However, bagging is frequently used in the context of decision trees.

- Consider the "**base model**" to be a decision tree. We will train B different trees for each of the B different bootstrapped training sets.

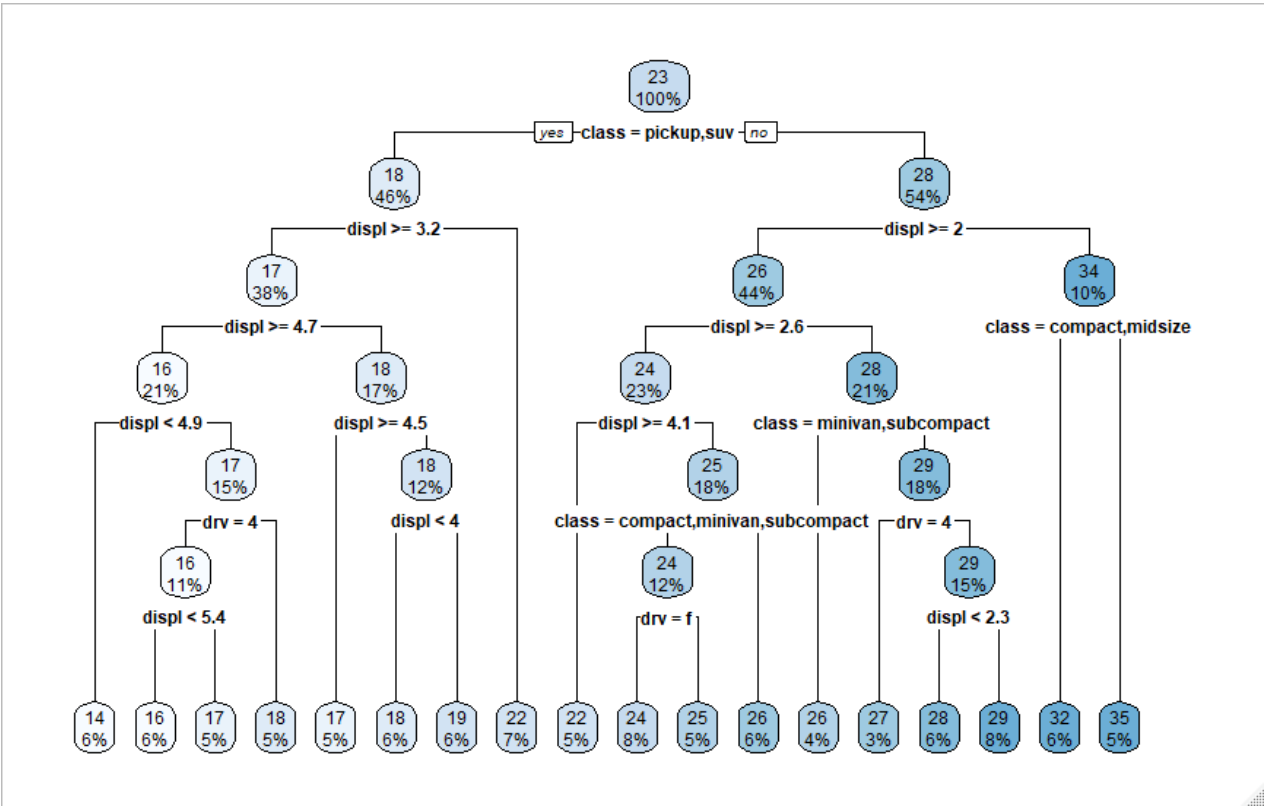
- **QUESTION**: would we want to grow shallow stumps or deep trees?

Consider the two bootstrapped samples below for the `mpg` example

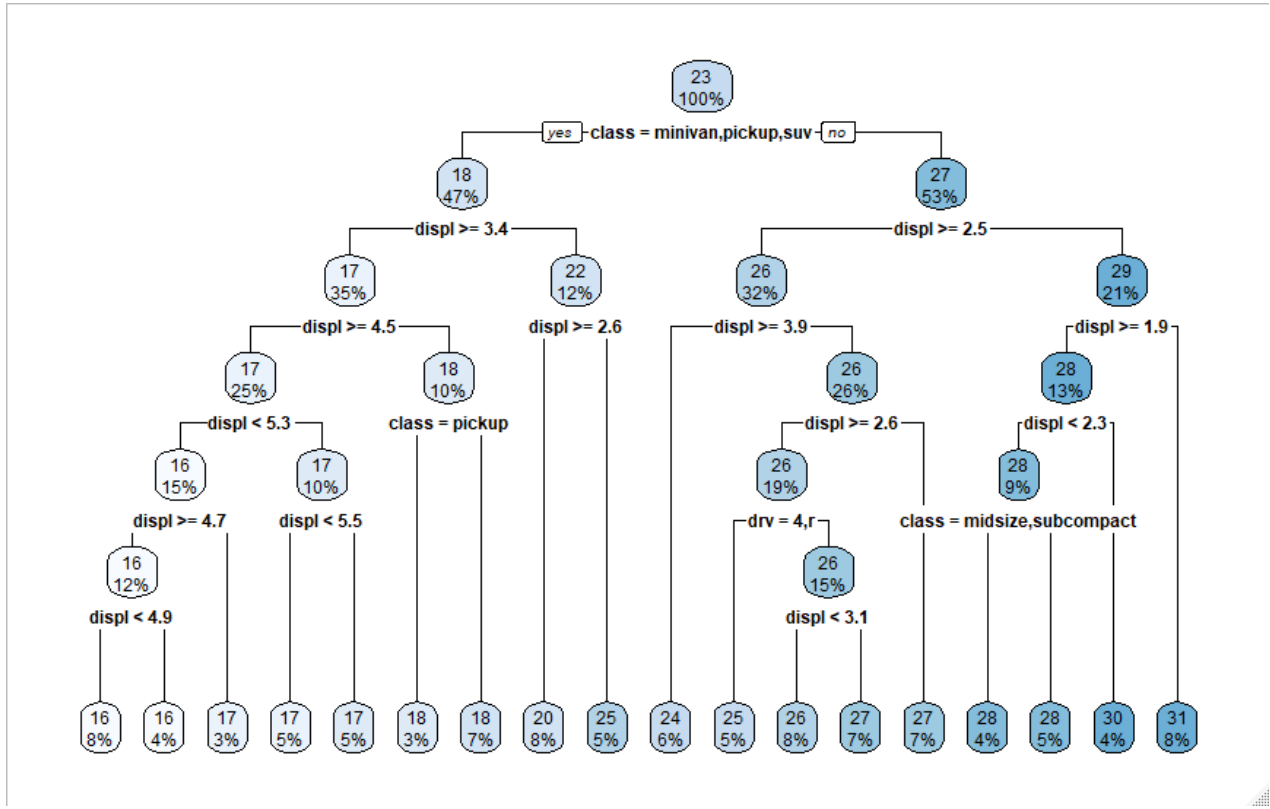


Fit trees to each bootstrapped sample

Bootstrap sample 1

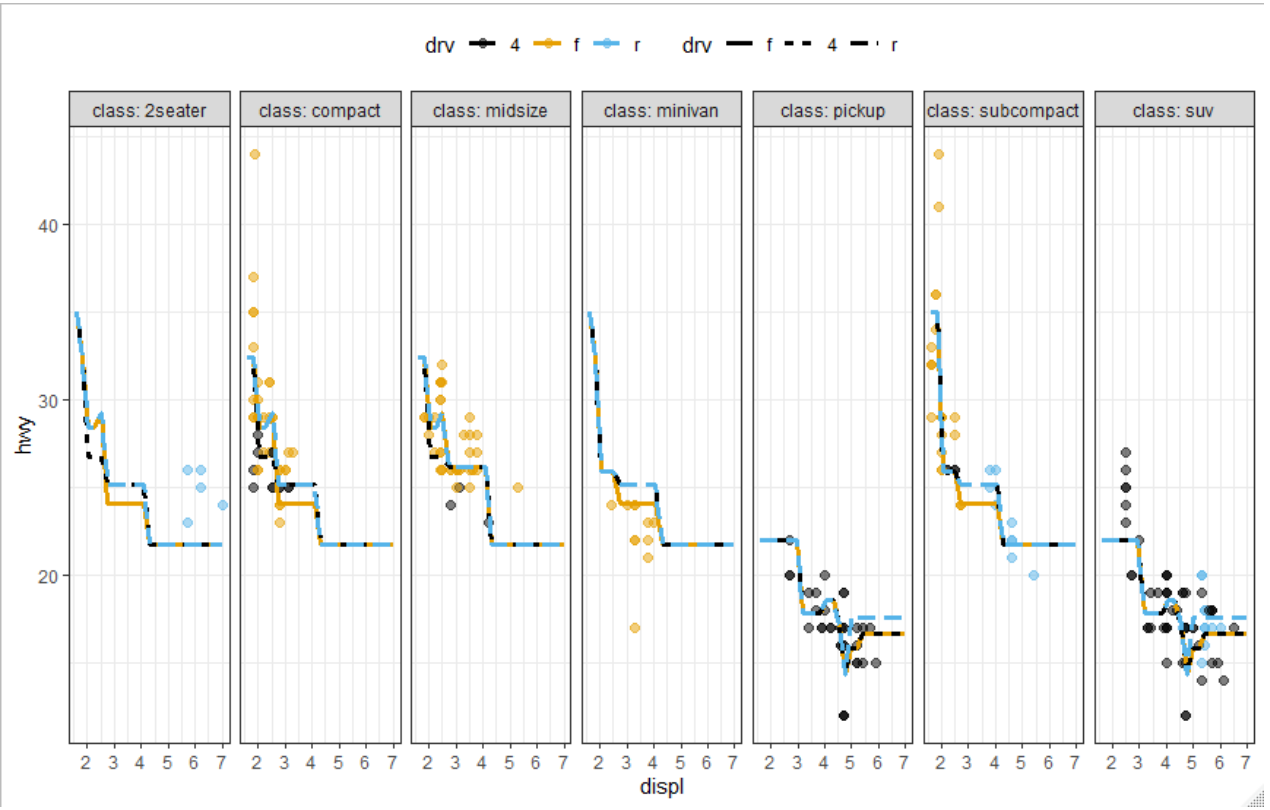


Bootstrap sample 2

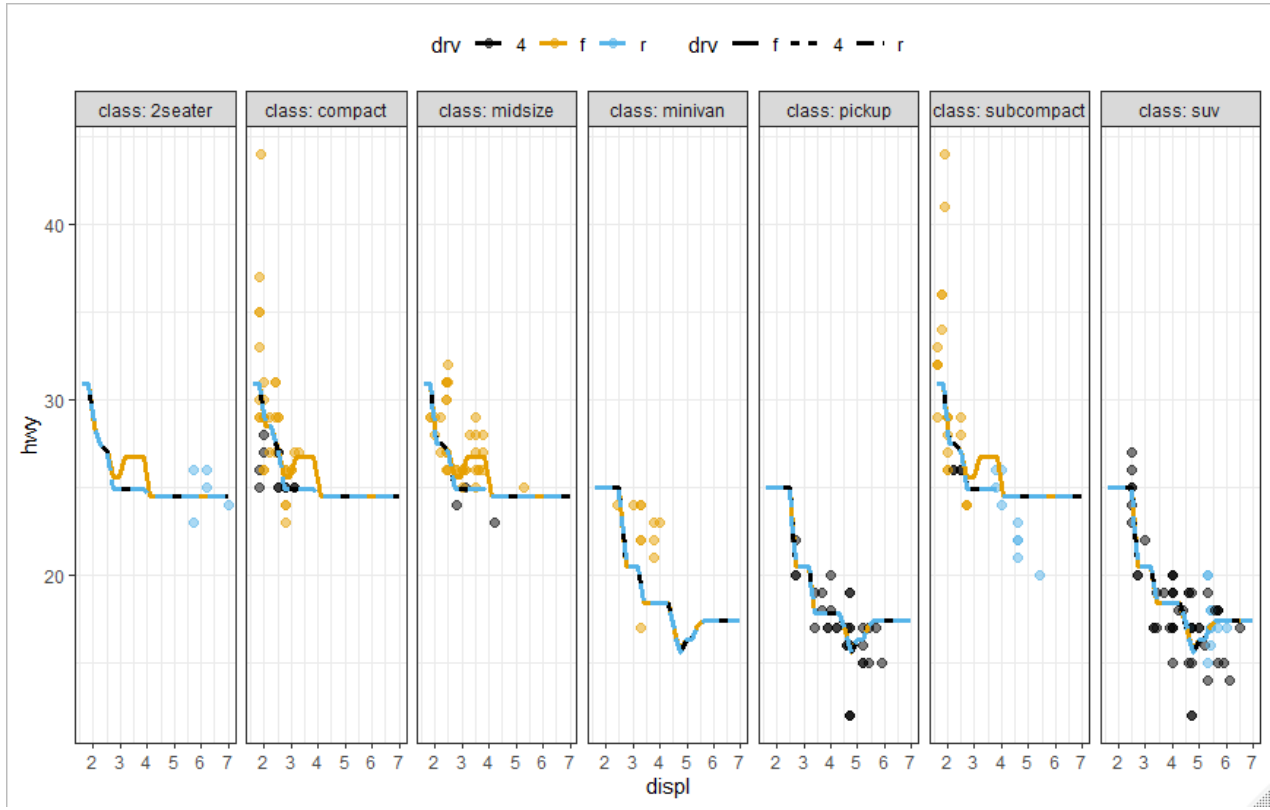


Predictions from each model

Bootstrap sample 1



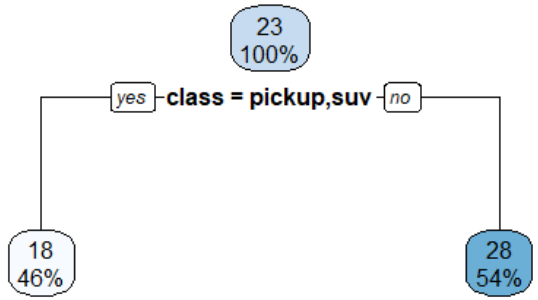
Bootstrap sample 2



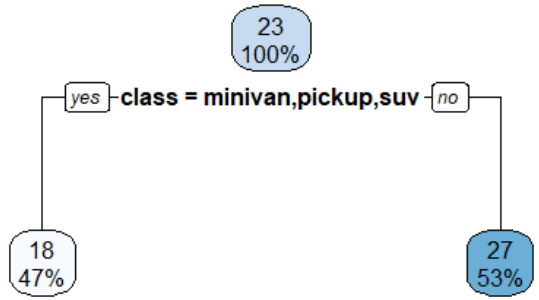
The two models yield different predicted behavior ⟹ represents high variance of the model!

What if we used simpler, less deep, trees?

Bootstrap sample 1

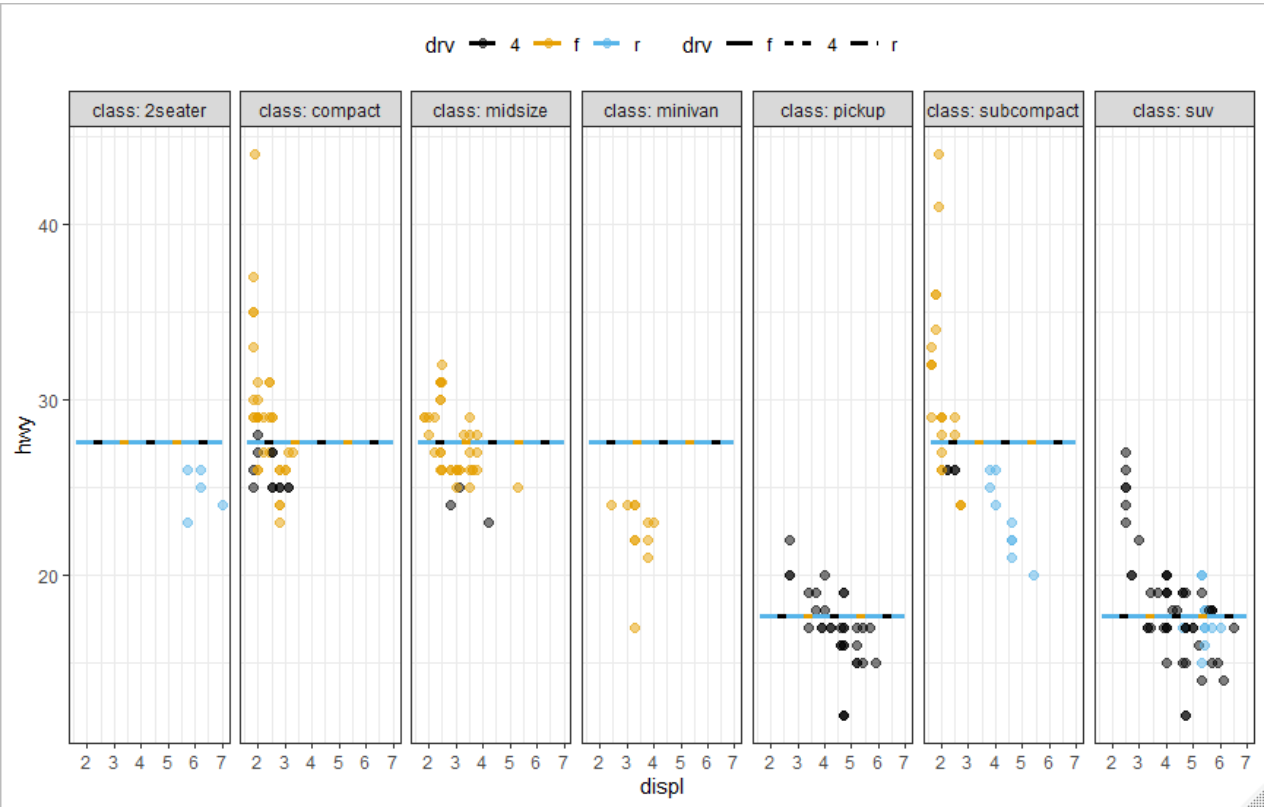


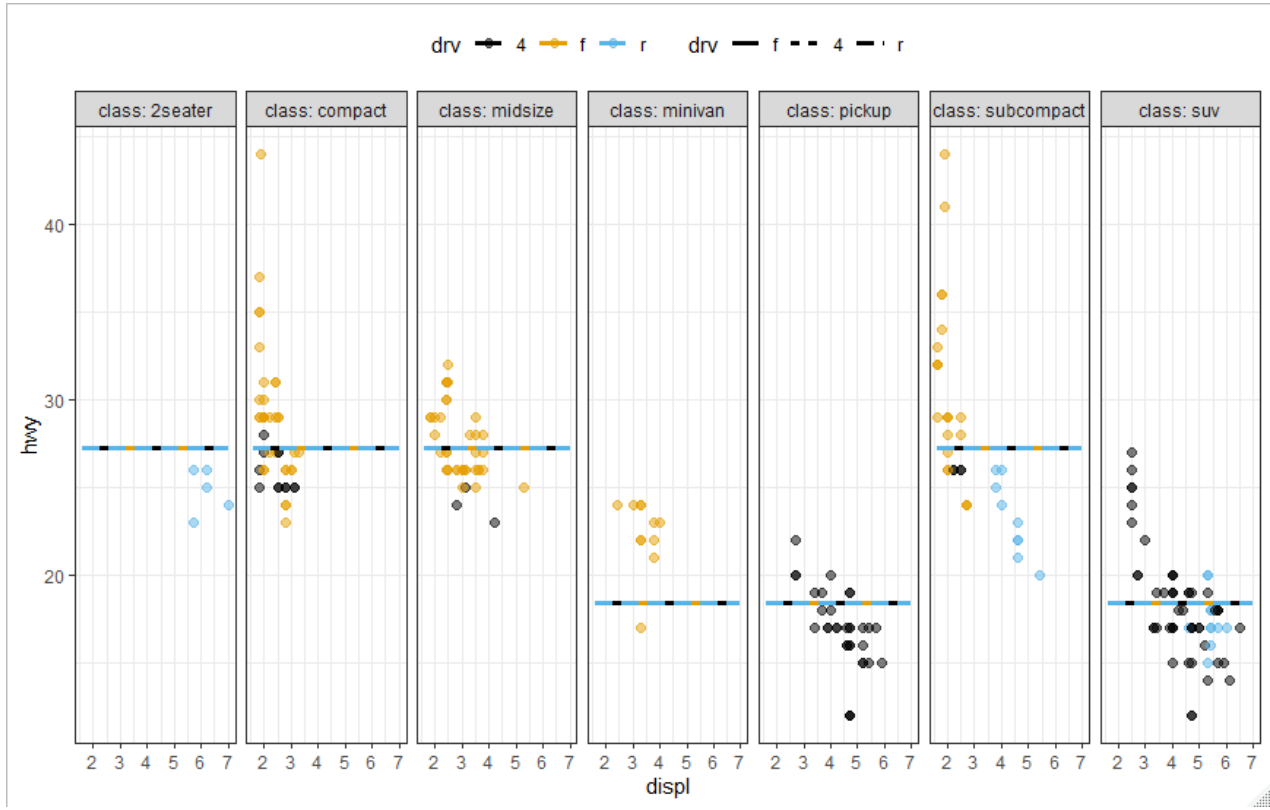
Bootstrap sample 2



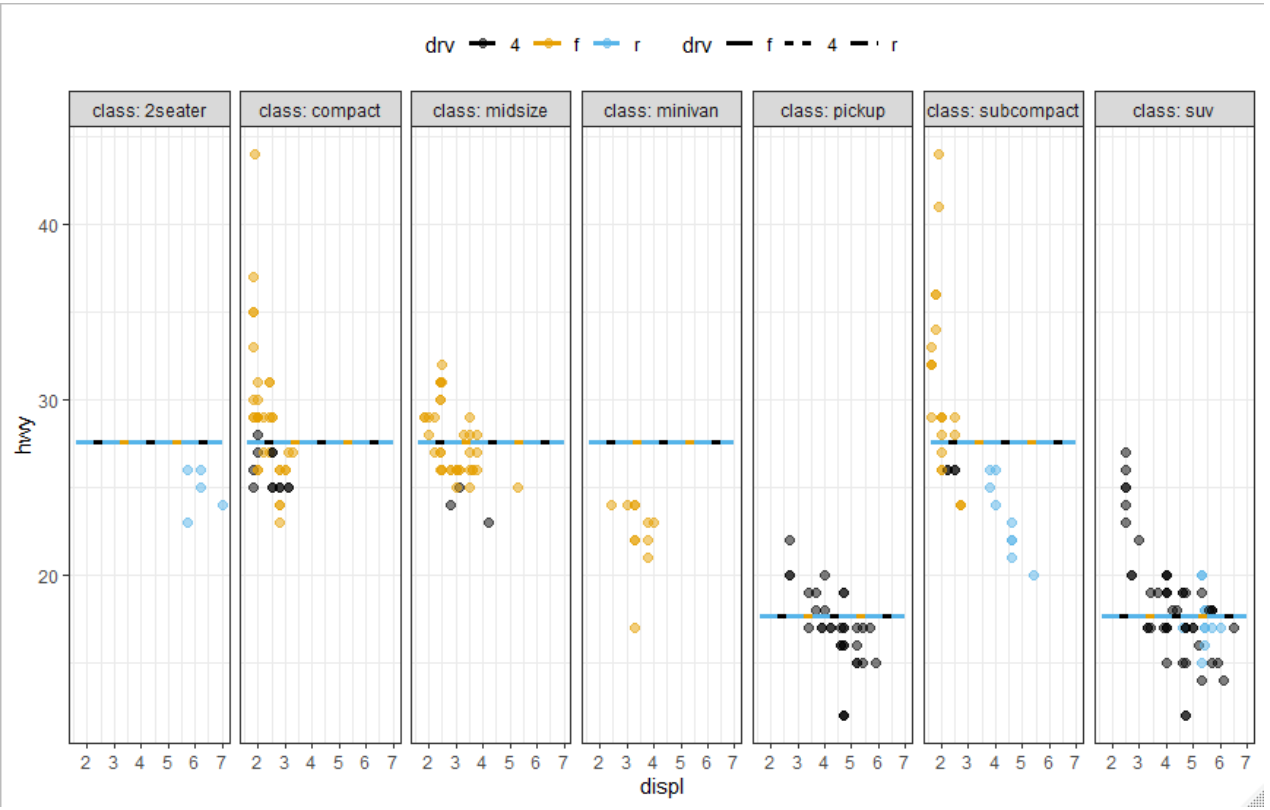
The trees appear to be very similar! The exact values for the splits are different but the splitting variables are the same!

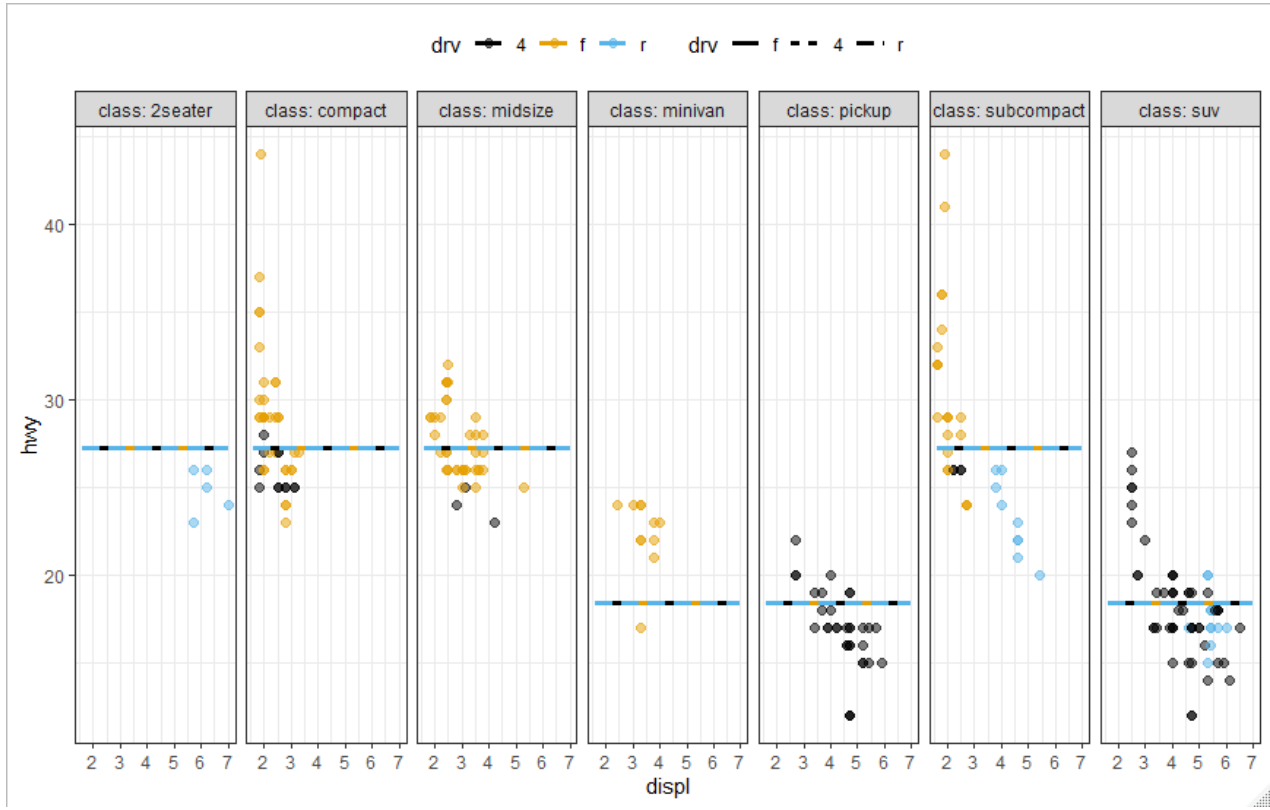
Predictions from the simpler models





Predictions from the simpler models





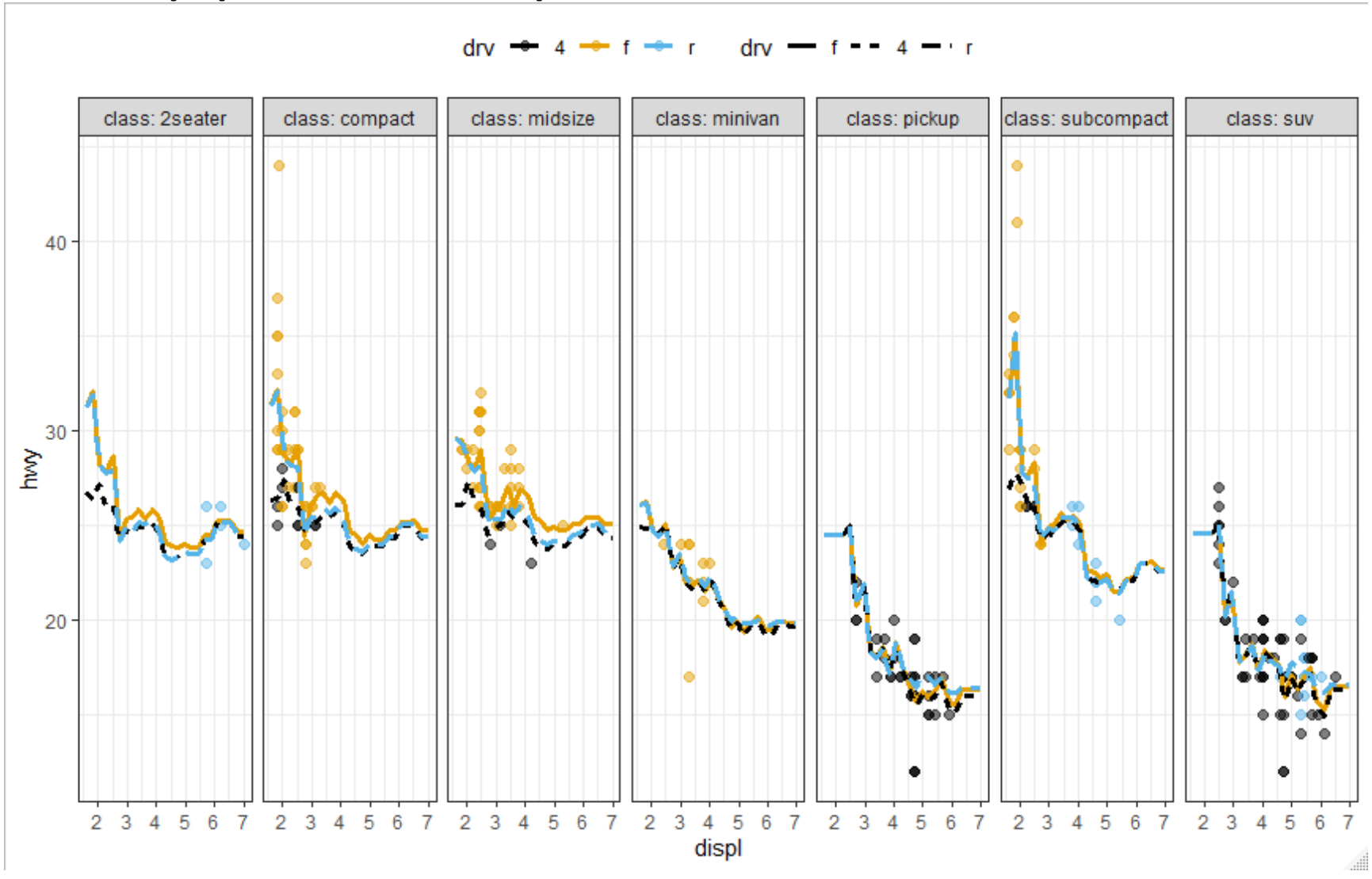
The simpler models are more consistent with each other, even though the training sets are different. They are less sensitive to the training set!

Higher bias!

To “maximize” the benefit of having multiple models, we want to have different “answers”

- Thus, **bagging is best suited when the base model has high variance**.
- We essentially want to overfit each tree to the bootstrapped training sample.
- Each tree’s prediction is therefore highly variable.
- **Averaging the trees together reduces the overall variance**.

Predictions from a bagged tree model with 50 bootstrapped samples



If we are interested in a classification problem instead, the general concepts are the same

- We want to aggregate many “high variance” models, each “overfit” to their respective bootstrapped training set.

- Simplest approach to decide the bagged prediction is to use Majority Vote.

Bagging has a straightforward approach to estimating the model performance

- Recall that each observation is selected in approximately 2/3 of the bootstrapped samples.
- That means, in about 1/3 of the models the observations are effectively held out and are not used to the train the models!
- These are referred to as the **out-of-bag** (**OOB**) observations.
- When B is large, the OOB error is like the LOOCV error.

A drawback of bagging is that we no longer have a single tree, which limits interpretability

- We cannot simply “read” the tree to see which variables decide the splits.
- However, each of the B trees still selects the splitting variable by trying to achieve the “best fit”.
  - Minimizing the SSE for regression and minimizing cross-entropy or the Gini index for classification.
- We can therefore look at the total reduction in the loss per variable and average over all trees.
- The larger the value, the more important the variable is compared to other variables!

This idea should be straightforward to think about for regression

- Reducing the SSE, increases the accuracy!

- What about the Gini index? What does it represent?

- For a binary outcome, and with the proportion of the event at a node equal to p, the Gini index reduces to:

Reducing the Gini index reduces the variance between classes at the node!

$$2p(1-p)$$

Twice the variance of a Bernoulli random variable with probability p!

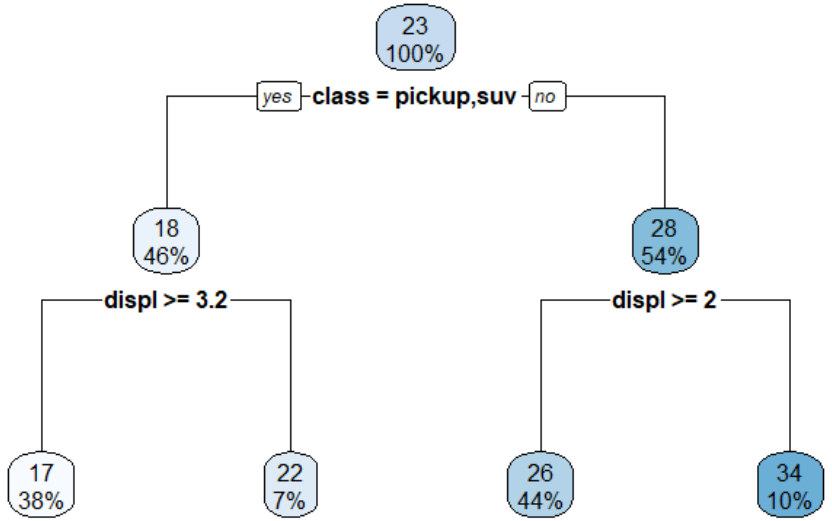
What does this look like?

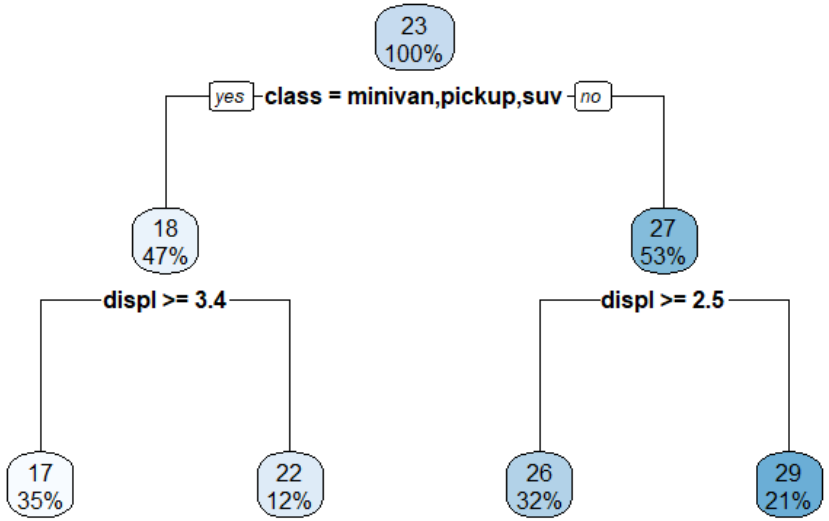
Bagging can be powerful, but the benefits can be limited if the models are “similar”

- We discussed how we want the individual models to be “overfit” to the separate bootstrapped training sets.

- Even though the bootstrapped samples are generated independently of each other, the separate trees might still be similar.

Consider the two bootstrapped shallow trees below



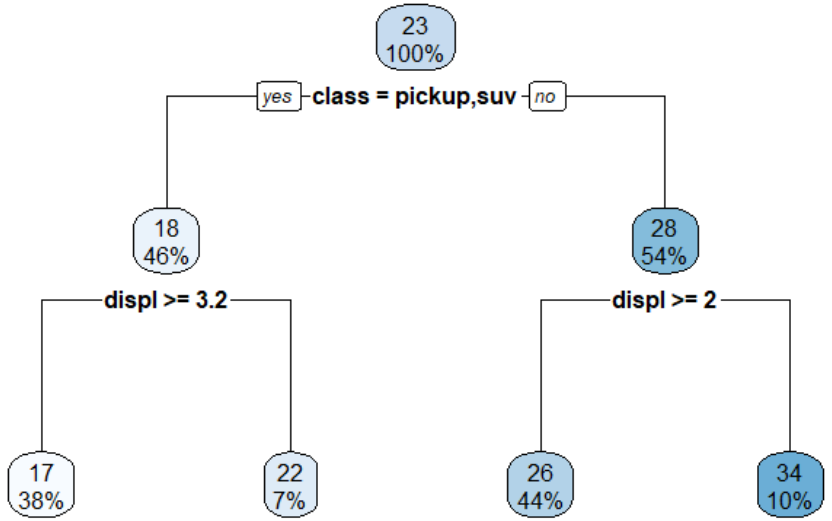


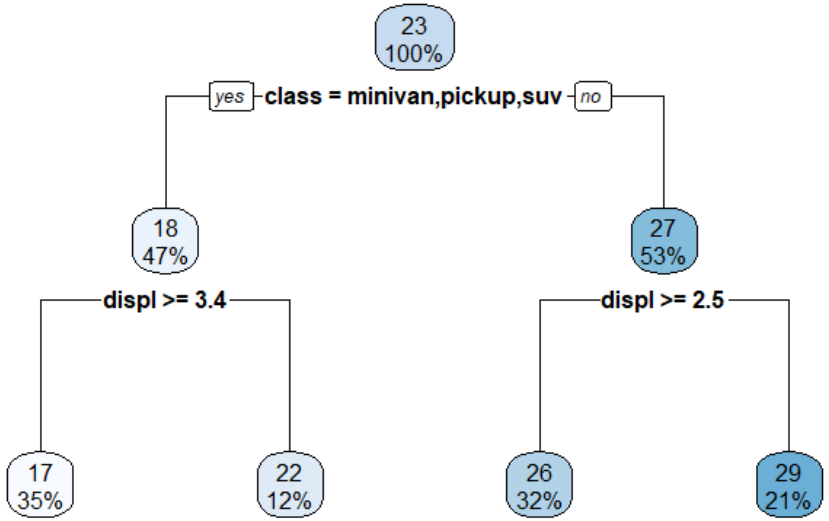
The overall structure of the two are similar.

The splitting variables are identical, though the exact splitting values are different.

This "degree" of similarity limits the potential improvement from bagging.

If we grow the trees further, the structure of the "top" of the trees are the same!





So the structure of the "early" splits will not change!

Similarity represents correlation between the trees.

- Even though each tree “sees” a different bootstrapped data set, they all consider the **same variables at each split**.
- A “strong predictor” will almost always dominate the behavior.
- That’s why the top splits appear similar between the trees.
- Causes the trees to be “correlated”.

To overcome this, we need to **decorrelate** the trees.

- Force the trees to consider a random subset of the variables when splitting.

- Some splits will therefore, by random chance, not include the strong predictor. Forces the trees to associate behavior with the other variables!

- We are forcing the trees to be different!

Combining bagging with randomly selecting predictors with trees is known as the **Random Forest**

Basic outline for a Random Forest model:

- Given the training data with N observations and D inputs: $\mathbf{X}$, $\mathbf{y}$
- Specify B: the number of trees
- Specify the number of inputs to randomly select per split: m_{try}
- Fit the model:
  - Generate B bootstrap samples of the training data.
  - Grow a separate decision tree associated with each bootstrap sample
    - When splitting, randomly select m_{try} inputs out of the total D inputs.
    - **Do not prune the trees**.

Bagging gives us the OOB error without cross-validation…

- However, the Random Forest requires specifying the number of variables to randomly select per split, m_{try}.

- Default values are:
  - Regression: $m_{try} = \text{floor}(D/3)$
  - Classification: $m_{try} = \text{floor}(\sqrt{D})$

- The “best” value for m_{try} is tuned with cross-validation!

Bagging and Random Forest summary

- We extended the basic decision tree by creating MULTIPLE trees.
- Predictions are made by aggregating the ENSEMBLE of models together.
- The ensemble consists of many separate models each trained **independently** of the others.

In bagging, we worked with an **ensemble** of high variance overfit models

- Let's now consider the opposite case.

- Let's work with an ensemble of high bias or underfit models!

- As discussed previously, the underfit (high bias) models are similar…so averaging them together will not provide much benefit…

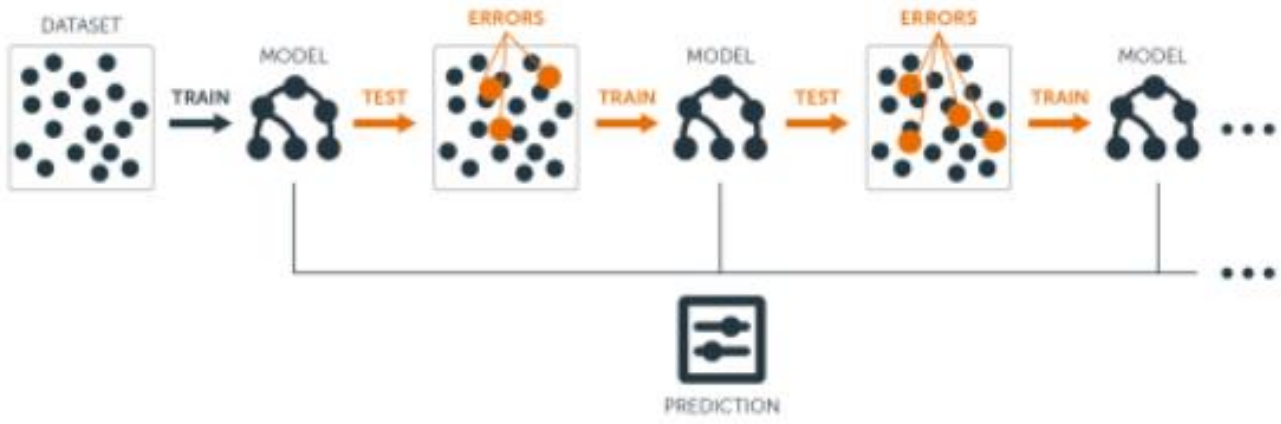
Instead of averaging separate models…we will **SEQUENTIALLY** update or **BOOST** a simple model!

- Boosting is the concept of “slowly” improving a “weak learner”.

- Originally developed for classification problems.
  - Weak learner specifically referred to being slightly better than random guessing.

- In the context of regression, think of a “weak learner” as a simple model such as a constant.

We will sequentially build simple models to correct the errors of previous models



From: https://bradleyboehmke.github.io/HOML/gbm.html

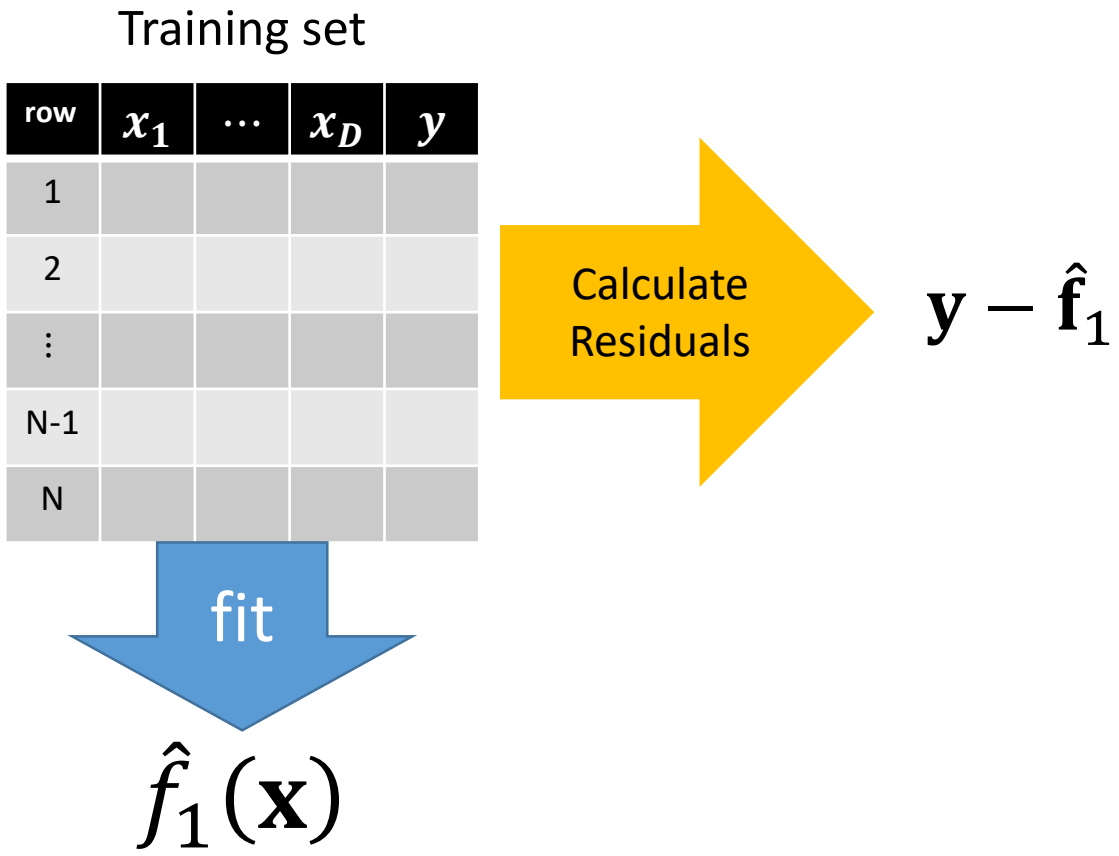
We will sequentially build simple models to correct the errors of the previous model

Training set

row	x_1	⋯	x_D	y
1				
2				
⋮				
N-1				
N				

$$\hat{f}_1(\mathbf{x})$$

We will sequentially build simple models to correct the errors of the previous model



We will sequentially build simple models to correct the errors of the previous model

Training set

row	x_1	⋯	x_D	y
1				
2				
⋮				
N-1				
N				

fit

$\hat{f}_1(\mathbf{x})$

Calculate Residuals

$\mathbf{y} - \hat{\mathbf{f}}_1$

Modify training set

The response is now the RESIDUAL associated with the first model

row	x_1	⋯	x_D	$y - \hat{f}_1$
1				
2				
⋮				
N-1				
N				

We will sequentially build simple models to correct the errors of the previous model

Training set

row	x_1	⋯	x_D	y
1				
2				
⋮				
N-1				
N				

fit

$\hat{f}_1(\mathbf{x})$

Calculate Residuals

$\mathbf{y} - \hat{\mathbf{f}}_1$

Modify training set

The response is now the RESIDUAL associated with the first model

row	x_1	⋯	x_D	$y - \hat{f}_1$
1				
2				
⋮				
N-1				
N				

fit

$\hat{f}_2(\mathbf{x})$

We will sequentially build simple models to correct the errors of the previous model

The response is the RESIDUAL associated with the first model

row	x_1	⋯	x_D	$y - \hat{f}_1$
1				
2				
⋮				
N-1				
N				

fit

$\hat{f}_2(\mathbf{x})$

Calculate Residuals

$(\mathbf{y} - \hat{\mathbf{f}}_1) - \hat{\mathbf{f}}_2$

Modify training set

The response is now the RESIDUAL associated with the second model

row	x_1	⋯	x_D	$y - \hat{f}_1 - \hat{f}_2$
1				
2				
⋮				
N-1				
N				

fit

$\hat{f}_3(\mathbf{x})$

We will sequentially build simple models to correct the errors of the previous model

The response is the RESIDUAL associated with the first model

row	x_1	⋯	x_D	$y - \hat{f}_1$
1				
2				
⋮				
N-1				
N				

fit

$\hat{f}_2(\mathbf{x})$

Calculate Residuals

$(\mathbf{y} - \hat{\mathbf{f}}_1) - \hat{\mathbf{f}}_2$

Modify training set

REPEAT

The response is now the RESIDUAL associated with the second model

row	x_1	⋯	x_D	$y - \hat{f}_1 - \hat{f}_2$
1				
2				
⋮				
N-1				
N				

fit

$\hat{f}_3(\mathbf{x})$

Equivalently we can think of boosting as fitting a model to the **gradient** of the loss with respect to the model prediction

- To see that, consider the Sum of Squared Error (*SSE*) loss:

$$SSE = \sum_{n=1}^{N} R_n = \sum_{n=1}^{N} \left((y_n - f_n)^2 \right)$$

- The derivative of the R_n with respect to f_n is:

$$\frac{\partial R_n}{\partial f_n} = -2(y_n - f_n)$$

Equivalently we can think of boosting as fitting a model to the **gradient** of the loss with respect to the model prediction

- To see that, consider the Sum of Squared Error (*SSE*) loss:

$$SSE = \sum_{n=1}^{N} R_n = \sum_{n=1}^{N} \left((y_n - f_n)^2 \right)$$

- The derivative of the R_n with respect to f_n is:

$$\frac{\partial R_n}{\partial f_n} = -2(y_n - f_n)$$

The residual!!!!

General outline of a boosting algorithm for regression trees

- Given training data: $\mathbf{X}$, $\mathbf{y}$
- Set $\hat{f}(\mathbf{x}) = 0$ and $r_n = y_n$
- For $b = 1, \ldots, B$ repeat
  - Fit a decision tree $\hat{f}_b(\mathbf{x})$ with d splits to (modified) training data: $\mathbf{X}$, $\mathbf{r}$
  - Update $\hat{f}(\mathbf{x})$ by adding in a "shrunken" version of the new tree:

$$\hat{f}(\mathbf{x}) \leftarrow \hat{f}(\mathbf{x}) + \lambda \hat{f}_b(\mathbf{x})$$

  - Update the residuals:

$$r_n \leftarrow r_n - \lambda \hat{f}_b(\mathbf{x})$$

- Final boosted model is:

$$\hat{f}(\mathbf{x}) = \sum_{b=1}^{B} \lambda \, \hat{f}_b(\mathbf{x})$$

General outline of a boosting algorithm for regression trees

- Given training data: $\mathbf{X}$, $\mathbf{y}$
- Set $\hat{f}(\mathbf{x}) = 0$ and $r_n = y_n$
- For $b = 1, \ldots, B$ repeat Number of trees = Number of iterations
  - Fit a decision tree $\hat{f}_b(\mathbf{x})$ with d splits to (modified) training data: $\mathbf{X}$, $\mathbf{r}$
  - Update $\hat{f}(\mathbf{x})$ by adding in a "shrunken" version of the new tree:

$$\hat{f}(\mathbf{x}) \leftarrow \hat{f}(\mathbf{x}) + \lambda \hat{f}_b(\mathbf{x})$$

  - Update the residuals:

$$r_n \leftarrow r_n - \lambda \hat{f}_b(\mathbf{x})$$

- Final boosted model is:

$$\hat{f}(\mathbf{x}) = \sum_{b=1}^{B} \lambda \, \hat{f}_b(\mathbf{x})$$

General outline of a boosting algorithm for regression trees

- Given training data: $\mathbf{X}$, $\mathbf{y}$
- Set $\hat{f}(\mathbf{x}) = 0$ and $r_n = y_n$
- For $b = 1, \ldots, B$ repeat
  - Fit a decision tree $\hat{f}_b(\mathbf{x})$ with d splits to (modified) training data: $\mathbf{X}$, $\mathbf{r}$
  - Update $\hat{f}(\mathbf{x})$ by adding in a "shrunken" version of the new tree:

$$\hat{f}(\mathbf{x}) \leftarrow \hat{f}(\mathbf{x}) + \lambda \hat{f}_b(\mathbf{x})$$

  - Update the residuals:

$$r_n \leftarrow r_n - \lambda \hat{f}_b(\mathbf{x})$$

- Final boosted model is:

$$\hat{f}(\mathbf{x}) = \sum_{b=1}^{B} \lambda \, \hat{f}_b(\mathbf{x})$$

Interaction depth

General outline of a boosting algorithm for regression trees

- Given training data: $\mathbf{X}$, $\mathbf{y}$
- Set $\hat{f}(\mathbf{x}) = 0$ and $r_n = y_n$
- For $b = 1, \ldots, B$ repeat
  - Fit a decision tree $\hat{f}_b(\mathbf{x})$ with d splits to (modified) training data: $\mathbf{X}$, $\mathbf{r}$
  - Update $\hat{f}(\mathbf{x})$ by adding in a "shrunken" version of the new tree:

$$\hat{f}(\mathbf{x}) \leftarrow \hat{f}(\mathbf{x}) + \lambda \hat{f}_b(\mathbf{x})$$

Learning rate

  - Update the residuals:

$$r_n \leftarrow r_n - \lambda \hat{f}_b(\mathbf{x})$$

- Final boosted model is:

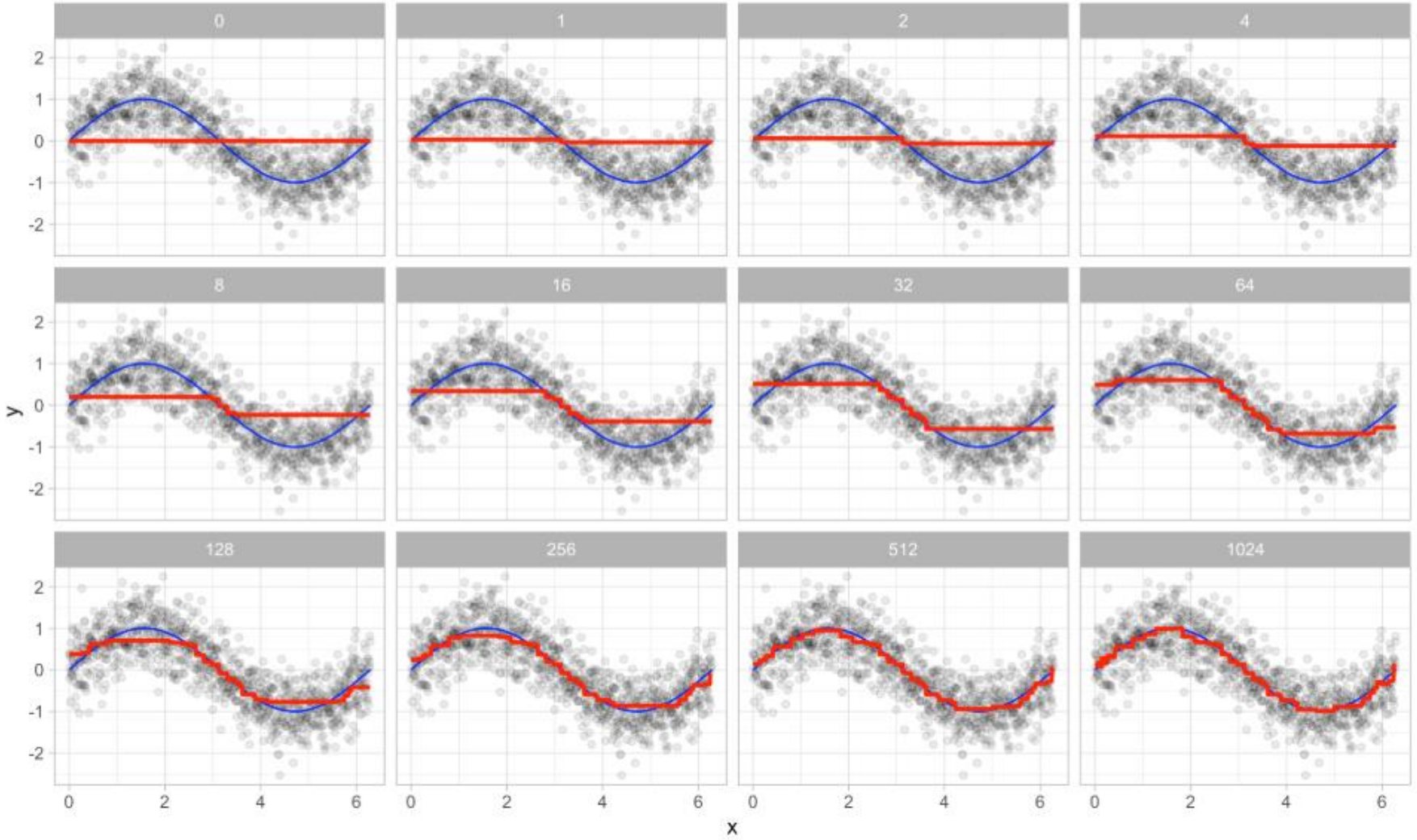
$$\hat{f}(\mathbf{x}) = \sum_{b=1}^{B} \lambda \hat{f}_b(\mathbf{x})$$

The previous algorithm is associated with regression models

- Boosting classification algorithms, such as AdaBoost, are similar in spirit.

- The new models improve the overall learner by focusing on the errors from the previous iteration.

Regression example – noisy sine wave



From: https://bradleyboehmke.github.io/HOML/gbm.html

Tree based methods in practice

- Tree based methods do not require much preprocessing of inputs.

- Random forests are fairly robust methods.
  - Typically you can just go with default settings.
  - Tuning m_{try} via cross-validation can yield further improvements.
  - I rarely change the number of trees from the default setting of 500.

- I almost always use random forest models as one of the candidate models I consider in a predictive modeling application.

Tree based methods in practice

- Boosted trees require more effort to tune and setup.
- Validation is critical to ensure the method does not overfit.
  - The number of trees corresponds to the number of iterations.
  - We can keep driving down the training set error by just using thousands and thousands of trees.
  - Essential to assess performance on a hold-out set, or preferably via cross-validation to identify the minimum number of trees required.

- The learning rate, λ, controls how quickly the errors are improved.
  - Smaller values require more trees, which increases the training time, but can improve performance.

Boosted tree implementations

- Original implementation of gradient boosted trees is the gbm package in `R`.

- Section 12.3.3 of the Hands on Machine Learning in R book provides a nice strategy for tuning `gbm` models.

- However, `gbm` can be quite slow and more recently the `xgboost` implementation of boosted trees has become more popular.

XGBoost stands for eXtreme Gradient Boosting and is a highly efficient implementation of boosted trees

- XGBoost burrows some concepts from random forests to help improve model performance.
  - Randomly select a fraction of the columns to use per tree. Similar to m_{try} to decorrelate trees.
  - Randomly subsample the observations used to train the trees. Similar to randomly creating a hold-out set and similar to Stochastic Gradient Descent!
  - Includes several additional parameters to help regularize the behavior and limit overfitting.

- `xgboost` therefore has more tuning parameters than the original `gbm` implementation. However, it typically outperforms `gbm` in both model performance and speed.

XGBoost is interfaced via APIs to multiple programming languages

- Please see the XGBoost documentation to see all of the languages it can interface with.
- Please see the xgboost CRAN site for specific documentation for the `R` interface.
- This github page provides a lot of useful references and material about XGBoost uses.
- The Hands on Machine Learning in R book also discusses tuning strategies.